ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

------oOo------

BÁO CÁO MÔN HỌC

CSC10012 - Cơ sở lập trình

Đồ án Caro

Giảng viên hướng dẫn : TS. Trương Toàn Thịnh
Nhóm sinh viên thực hiện : Nhóm 3 - Lớp 25CTT7

STT	MSSV	Họ và tên
1	24120260	Trương Tuấn Anh
2	24120421	Lê Văn Quốc
3	24120428	Đặng Quang Sang
4	24120451	Trương Đoàn Công Thành

TP. Hồ Chí Minh, tháng 04 năm 2026

BẢNG PHÂN CÔNG

STT	MSSV	Họ và tên	Nhiệm vụ
1	24120260	Trương Tuấn Anh	- Lên template source - Thiết kế giao diện
2	24120421	Lê Văn Quốc	- Thiết kế Game Logic
3	24120428	Đặng Quang Sang	- Quản lý âm thanh, ngôn ngữ, thời gian
4	24120451	Trương Đoàn Công Thành	- Thiết kế Game Logic

LỜI NÓI ĐẦU

Đồ án Caro được xây dựng trong khuôn khổ môn học Cơ sở lập trình tại Trường Đại học Khoa học Tự nhiên TP.HCM. Thay vì chỉ dừng lại ở các giao diện console cơ bản, dự án của nhóm đã tập trung ứng dụng ngôn ngữ C++17 kết hợp cùng Win32 API và thư viện đồ họa GDI+ để tạo ra một thế giới bóng đá Pixel Art sinh động.

Đồ án không chỉ là một trò chơi trí tuệ, mà còn là sự kết hợp độc đáo giữa logic cờ caro và không khí cuồng nhiệt của các sân vận động hàng đầu thế giới. Dự án tích hợp nhiều tính năng hiện đại như:

- Hệ thống AI thông minh: Sử dụng thuật toán Alpha-Beta Pruning và Zobrist Hashing để tạo ra các cấp độ thử thách từ cơ bản đến chuyên nghiệp.
- Giao diện & Âm thanh sống động: Hiệu ứng Glassmorphism hiện đại, hệ thống âm thanh đa luồng không gây gián đoạn và các nhân vật biểu tượng như Ronaldo, Messi, Neymar dưới dạng Pixel Art.
- Cơ chế chuyên sâu: Thống kê tỷ lệ kiểm soát bóng, số nước đi, hệ thống lưu/tải trò chơi nhị phân đảm bảo tính ổn định và bảo mật dữ liệu.

Thông qua đồ án này, nhóm chúng em không chỉ củng cố kỹ thuật xử lý mảng, tập tin và con trỏ mà còn học cách tổ chức mã nguồn theo mô-đun chuyên nghiệp và tối ưu hóa hiệu suất ứng dụng.

Chúng em xin gửi lời cảm ơn chân thành đến thầy Trương Toàn Thịnh đã tận tình hướng dẫn và tạo điều kiện để nhóm có thể hoàn thiện đồ án này một cách tốt nhất.

MỤC LỤC

BẢNG PHÂN CÔNG 1

LỜI NÓI ĐẦU 2

MỤC LỤC 3

DANH SÁCH HÌNH ẢNH 8

DANH SÁCH BẢNG 9

DANH SÁCH MÃ NGUỒN 12

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI VÀ CÔNG NGHỆ 13

1.1. Giới thiệu đề tài 13

1.1.1. Bối cảnh, lý do chọn đề tài 13

1.1.2. Mục tiêu kỹ thuật và giá trị giải trí 13

1.2. Phạm vi và ràng buộc 14

1.2.1. Phạm vi chức năng chính 14

1.2.2. Ràng buộc hiệu năng và nền tảng 15

1.3. Công nghệ và môi trường triển khai 16

1.3.1. C++17 và tư duy lập trình mô-đun 16

1.3.2. Win32 API + GDI+ cho đồ họa 2D 17

1.3.3. Visual Studio, cấu hình build và quản lý phiên bản 17

1.4. Tổng quan chức năng hệ thống 18

1.4.1. PvP, PvE và các mức độ trí tuệ nhân tạo 18

1.4.2. Lưu/Tải, Hoàn tác/Thực hiện lại và Hệ thống Thống kê 18

CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG VÀ TỔ CHỨC MÃ NGUỒN 20

2.1. Kiến trúc phân tầng 20

2.1.1. Render Layer và Logic Layer 20
2.1.2. System Layer: Tài nguyên, cấu hình và thời gian 21
2.2. Luồng xử lý sự kiện 22
2.2.1. Vòng lặp trò chơi và Thời gian sai lệch 22
2.2.2. Kỹ thuật vùng đệm kép chống nhấp nháy 22
2.2.3. Biểu đồ phân rã kiến trúc hệ thống 23
2.2.4. Sơ đồ luồng chương trình 25
2.3. Cấu trúc thư mục và phân hệ 27
2.3.1. Chi tiết các phân hệ 29
2.3.2. Quy ước Header, Cấu trúc dữ liệu và Đóng gói dữ liệu 31

CHƯƠNG 3. THIẾT KẾ DỮ LIỆU VÀ CẤU TRÚC CỐT LÕI 33
3.1. Quản lý trạng thái ván đấu 33
3.1.1. Ma trận bàn cờ và tối ưu bộ nhớ 33
3.1.2. Turn-based logic và Match Status 34
3.1.3. Lịch sử nước đi và cơ chế Undo/Redo 35
3.1.4. Kiểm tra điều kiện thắng và trạng thái kết thúc 36
3.2. Mô hình người chơi 36
3.2.1. PlayerInfo2: Wins, Moves, Possession Time 36
3.2.2. PlayerEngineer và vòng đời thực thể 37
3.3. Cấu hình và môi trường 38
3.3.1. Nạp Config và thiết lập mặc định 38
3.3.2. Lưu/Tải trạng thái và metadata 39
3.3.3. Hệ thống màu sắc và theme động 40

CHƯƠNG 4. HỆ THỐNG ĐỒ HỌA VÀ UI/UX 42
4.1. Tổng quan pipeline đồ họa 42
4.2. Renderer và tối ưu hóa hiển thị 43
4.2.1. Khởi tạo GDI+ và quản lý font 43
4.2.2. Double Buffering chống nhấp nháy 44
4.2.3. Đồng bộ resize và tái tạo back buffer 45
4.2.4. GDI+ nâng cao: Alpha Blending và Linear Gradient 46
4.2.5. Bitmap Caching cho đối tượng phức tạp 47
4.3. Pixel Art Engine 48
4.3.1. Dựng hình từ ma trận dữ liệu số 48
4.3.2. Palette và ngữ nghĩa màu 52

4.3.3. Action Animation System 54
4.4. Thiết kế UI thích ứng (Responsive UI) 56
4.4.1. UIScaler và cơ chế tỉ lệ hóa 56
4.4.2. Hiệu ứng thị giác và nền động 57
4.5. Vẽ bàn cờ và các hiệu ứng gameplay 59
4.6. Case study UI/UX từng màn hình 61
4.6.1. MenuScreen 61
4.6.2. PlayScreen 63
4.6.3. Pause Game 65
4.6.4. MatchConfigScreen 67
4.6.5. SettingScreen 69
4.6.6. LoadGameScreen 70
4.6.7. AboutScreen 72
4.6.8. GuildScreen 73
4.6.9. SaveGameScreen 74
4.6.10. SummaryScreen 76
4.7. Tối ưu hóa và quản lý tài nguyên 77
4.8. Vòng lặp ứng dụng và điều phối input 78
4.8.1. Message pump, dt và giới hạn bước nhảy thời gian 78
4.8.2. Throttling input và chống autorepeat quá nhanh 78

CHƯƠNG 5. LUẬT CHƠI VÀ CƠ CHẾ KIỂM TRA 80
5.1. Luật và biến thể 80
5.1.1. Caro 5 quân và Tic-Tac-Toe 80
5.1.2. Kiểm tra hợp lệ và biên 80
5.1.3. Trạng thái tối thiểu phục vụ kiểm tra 81
5.2. Thuật toán xác định thắng 81
5.2.1. Quét 4 hướng tối ưu 81
5.2.3. Kiểm tra điều kiện hợp lệ trước khi quét 82
5.2.2. Truy vết winningCells cho highlight 82
5.3. Trạng thái kết thúc đặc biệt 83
5.3.1. Hòa và chặn nước đi 83
5.3.2. Cơ chế BO (Best-of) và thắng series 84
5.3.3. Timeout và đồng bộ TimeSystem 85

CHƯƠNG 6. BỘ MÁY ĐIỀU PHỐI TRẬN ĐẤU 86

6.1. Điều phối lượt đánh 86
6.1.1. Luồng xử lý: Nhập lệnh → Luật → Cập nhật trạng thái 86
6.1.2. Đồng bộ lượt và biến isP1Turn 87
6.2. Điều phối match series 88
6.2.1. BO1/BO3/BO5 và mục tiêu thắng 88
6.2.2. Reset hiệp và bảo toàn tỷ số 88
6.3. Undo/Redo và lịch sử 90
6.3.1. Stack lưu nước đi 90
6.3.2. Khôi phục trạng thái và đồng hồ 90
6.4. Hệ thống thời gian 91
6.4.1. Đếm ngược lượt và thời gian giữ bóng 91
6.4.2. Quỹ thời gian toàn trận và khởi tạo match 92
6.3.3. Phục hồi redo và phán xét lại trạng thái 93

CHƯƠNG 7. TRÍ TUỆ NHÂN TẠO 95
7.1. Tổng quan kiến trúc AI 95
7.1.2. Gọi AI bất đồng bộ để giữ mượt giao diện 97
7.2. Đánh giá heuristic (đánh giá cục bộ) 98
7.2.1. Nhận dạng mẫu công/thủ theo 4 hướng 98
7.2.2. Trọng số chuỗi mở và chuỗi bị chặn 99
7.2.3. Tính điểm một nước đi theo công/thủ 100
7.3. Tìm kiếm cây quyết định 101
7.3.1. Minimax và cắt tỉa alpha-beta 101
7.3.2. Sắp xếp nước đi 103
7.3.3. Ưu tiên thắng ngay và chặn thua 104
7.4. Tối ưu và tăng tốc 105
7.4.1. Zobrist hashing 105
7.4.2. Transposition Table 106
7.4.3. Lọc ứng cử viên và giới hạn nhánh 107

CHƯƠNG 8. MÔ-ĐUN HỆ THỐNG PHỤ TRỢ 108
8.1. Lưu trữ và serialization 108
8.1.1. Binary Save: magic, version và chuỗi Unicode 108
8.1.2. Serialize PlayerInfo2 và các vector động 109
8.1.3. Save/Load toàn bộ PlayState 111
8.1.4. Hệ thống slot và metadata 115

8.2. Đa phương tiện 116
8.2.1. AudioSystem: preload và phát âm thanh 116
8.2.2. SFX bất đồng bộ và cơ chế chống spam 117
8.2.3. BGM: lặp vô hạn và quản lý volume 119
8.3. Ngôn ngữ 120
8.4. Thời gian 122
8.4.1. Cập nhật bộ đếm theo dt 122
8.4.2. Reset theo lượt và cơ chế failsafe 123
8.4.3. Hiển thị thời gian dạng MM:SS 124
8.5. Cấu hình hệ thống 124

CHƯƠNG 9. TỔNG KẾT **126**
9.1. Kết quả đạt được 126
9.2. Các hạng mục đã hoàn thiện 126
9.3. Dấu ấn kỹ thuật nổi bật 127

TÀI LIỆU THAM KHẢO **130**

DANH SÁCH HÌNH ẢNH

Hình 2.1: Sơ đồ kiến trúc hệ thống 24
Hình 2.2: Sơ đồ luồng chương trình 26
Hình 4.1: Hình minh họa dữ liệu pixel model (ví dụ title_caro) 49
Hình 4.2: Hình minh họa chuỗi frame action run của avt_00 55
Hình 4.3: Hình minh họa MenuScreen 62
Hình 4.4: Hình minh họa PlayScreen 64
Hình 4.5: Hình minh họa Pause Game 66
Hình 4.6: Hình minh họa MatchConfigScreen - trang 1 67
Hình 4.7: Hình minh họa MatchConfigScreen - trang 2 68
Hình 4.8: Hình minh họa SettingScreen 69
Hình 4.9: Hình minh họa LoadGameScreen 71
Hình 4.10: Hình minh họa AboutScreen 72
Hình 4.11: Hình minh họa GuildScreen 74
Hình 4.12: Hình minh họa SaveGameScreen 75
Hình 4.13: Hình minh họa SummaryScreen 76

DANH SÁCH BẢNG

Bảng 4.1: Tổng hợp cache đồ họa và khóa nhận diện 48
Bảng 4.2: Kích thước các mô hình pixel art tiêu biểu 49
Bảng 8.1: Cấu hình slot lưu game và metadata 115
Bảng 8.2: Danh sách SFX preload và alias 117
Bảng 8.3: Cấu hình BGM và quy tắc cập nhật âm lượng 119
Bảng 8.4: Danh sách ngôn ngữ và file bản địa hóa 121
Bảng 9.1: Ma trận chức năng các mô-đun phụ trợ 126
Bảng 9.2: Tóm tắt các trường quan trọng trong PlayState 128

DANH SÁCH MÃ NGUỒN

1 Mã nguồn Hằng số và định nghĩa trạng thái ô bàn cờ 34
2 Mã nguồn Ma trận bàn cờ trong PlayState 34
3 Mã nguồn Enum MatchStatus trong PlayState 35
4 Mã nguồn Quy tắc Undo cho PvE (lùi 2 nước) 35
5 Mã nguồn Xác định độ dài thắng theo chế độ chơi 36
6 Mã nguồn Trả về toàn bộ chuỗi thắng để highlight 36
7 Mã nguồn Cấu trúc PlayerInfo2 37
8 Mã nguồn Reset thống kê theo round trong PlayerEngineer 38
9 Mã nguồn Thiết lập mặc định trong ConfigLoader 38
10 Mã nguồn Cấu trúc metadata cho slot lưu 39
11 Mã nguồn Magic number và phiên bản file save 39
12 Mã nguồn Cấu trúc SmartColor 40
13 Mã nguồn Một số màu chủ đạo trong Theme 40
14 Mã nguồn Khởi tạo font và tái tạo theo tỉ lệ UI 43
15 Mã nguồn Tạo bộ đệm kép theo kích thước cửa sổ 44
16 Mã nguồn Cập nhật tỉ lệ và back buffer khi resize 45
17 Mã nguồn Banner gradient và viền pulse 46
18 Ví dụ file pixel art `title_caro.txt` 49
19 Mã nguồn Load dữ liệu pixel từ file txt 49
20 Mã nguồn Dựng bitmap pixel art với cache và lưới 51
21 Mã nguồn Bảng màu nguyên thủy và màu ngữ nghĩa 52
22 Mã nguồn Tra mã màu avatar theo loại nhân vật 53
23 Mã nguồn Cập nhật frame và dựng animation 55
24 Mã nguồn Cập nhật tỉ lệ theo kích thước cửa sổ 56
25 Mã nguồn Cache nền sân và hiệu ứng flash 57
26 Mã nguồn Vẽ lưới bàn cờ bằng GDI trong DrawGameBoard 59
27 Mã nguồn Highlight thắng và con trỏ neon 60

28 Mã nguồn RenderMenuScreen và bố cục tiêu đề 62
29 Mã nguồn RenderPlayScreen: tính kích thước và vẽ bàn cờ 64
30 Mã nguồn Lớp phủ Pause Menu trong RenderPlayScreen 66
31 Mã nguồn RenderMatchConfigScreen: panel và banner 68
32 Mã nguồn RenderSettingScreen: panel và thanh volume 69
33 Mã nguồn Che do tuong tac va RenderLoadGameScreen 71
34 Mã nguồn RenderAboutScreen: banner va bang thanh vien 72
35 Mã nguồn RenderGuildScreen: banner va phan trang 74
36 Mã nguồn RenderSaveGameScreen: bố cục và banner 75
37 Mã nguồn RenderSummaryScreen: banner và vùng kết quả 76
38 Mã nguồn Dọn cache UI tập trung 77
39 Vòng lặp chính: xử lý message và giới hạn dt 78
40 Throttling phím điều hướng trong MenuScreen 79
41 Xác định độ dài chuỗi thắng theo chế độ chơi 80
42 Mã nguồn kiểm tra hợp lệ nước đi 80
43 Các trường dữ liệu phục vụ kiểm tra thắng và hòa 81
44 Mã nguồn quét 4 hướng từ nước đi cuối 81
45 Chặn sớm khi nước đi cuối không hợp lệ 82
46 Trả về toàn bộ chuỗi thắng và lưu trong PlayState 83
47 Highlight các ô thắng dựa trên winningCells 83
48 Kiểm tra điều kiện hòa bằng bộ đếm nước đi 83
49 Cập nhật trạng thái ván đấu sau khi kiểm tra thắng 84
50 Xác định thắng series BO trong processMove 84
51 Đếm ngược theo giây bằng bộ tích lũy thời gian 85
52 Xử lý timeout và đổi lượt trong vòng lặp chơi 85
53 Luồng xử lý một nước đi trong processMove 86
54 Đổi lượt và đồng bộ timer 87
55 Xác định thắng series trong processMove 88
56 Khởi tạo round mới, giữ nguyên tiến trình series 88
57 Reset thống kê lượt, giữ nguyên tiến trình BO 89
58 Ghi lịch sử và hủy nhánh redo khi có nước mới 90
59 Undo theo cơ chế 2 bước trong PvE 90
60 Cập nhật nước đi cuối sau Undo 91
61 Reopen ván đấu sau Undo 91
62 Cộng dồn thời gian giữ bóng và kiểm soát timeout 92
63 Khởi tạo trận mới và quỹ thời gian toàn trận 92

64 Phục hồi nước đi trong redoMove 93
65 Điểm vào chính của AI theo mức độ khó 95
66 Chạy AI bất đồng bộ và nhận kết quả an toàn 97
67 Phân tích chuỗi và đầu hở theo một hướng 98
68 Bảng trọng số đánh giá một chuỗi 100
69 Tính điểm một ô theo 4 hướng 100
70 Hàm alpha-beta đầy đủ (có TT và cắt tỉa) 102
71 Tạo và sắp xếp danh sách ứng cử viên 104
72 Ưu tiên nước thắng hoặc chặn thua tại gốc 104
73 Khởi tạo bảng Zobrist và sinh hash 105
74 Lưu và tra cứu bảng TT 106
75 Lọc ứng cử viên theo bán kính và giới hạn nhánh 107
76 Ghi/đọc chuỗi Unicode kèm độ dài 108
77 Serialize PlayerInfo2 theo phiên bản 109
78 Ghi trạng thái trận đấu ra file nhị phân 111
79 Đọc và khôi phục trạng thái trận đấu 113
80 Truy xuất metadata bản lưu để hiển thị danh sách 115
81 Preload các SFX và khởi tạo luồng nền 116
82 Hàng đợi SFX và xử lý luồng nền 117
83 Phát nhạc nền và cập nhật volume 119
84 Ví dụ định dạng `key=value` trong file ngôn ngữ 120
85 Nạp từ điển ngôn ngữ và chuyển UTF-8 121
86 Truy xuất text với cache khóa thiếu 122
87 Cập nhật đếm ngược và phát hiện hết lượt 122
88 Reset bộ đếm kèm kiểm tra an toàn 123
89 Định dạng thời gian hiển thị 124
90 Thiết lập mặc định và nạp cấu hình 124
91 Lưu cấu hình hiện hành 125
92 Các hằng số nền tảng cho bàn cờ và lưu game 127
93 Trạng thái ván đấu tập trung trong PlayState 128

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI VÀ CÔNG NGHỆ

1.1. Giới thiệu đề tài

1.1.1. Bối cảnh, lý do chọn đề tài

- **Bối cảnh thực hiện:** Đề tài được phát triển trong khuôn khổ môn học Cơ sở lập trình tại Trường Đại học Khoa học Tự nhiên TP.HCM. Dự án không chỉ là bài tập vận dụng kiến thức về xử lý mảng, con trỏ và tệp tin, mà còn là thử thách trong việc quản lý bộ nhớ và tối ưu hóa hiệu năng trong môi trường Windows.
- **Sự phổ biến của trò chơi:** Cờ Caro là một trò chơi trí tuệ kinh điển, sở hữu luật chơi đơn giản nhưng chứa đựng không gian trạng thái khổng lồ, là mảnh đất màu mỡ để thử nghiệm các thuật toán tìm kiếm và trí tuệ nhân tạo.
- **Lý do chọn đề tài:**
  - *Đổi mới sáng tạo:* Dự án xóa bỏ sự khô khan của bàn cờ truyền thống bằng việc "gamification" – kết hợp không khí kịch tính của bóng đá thế giới, tạo ra sự giao thoa độc đáo giữa trò chơi dân gian và thể thao hiện đại.
  - *Thử thách kỹ năng:* Việc xây dựng một ứng dụng Win32 hoàn chỉnh đòi hỏi khả năng tổ chức mã nguồn theo mô hình hướng mô-đun, xử lý sự kiện hệ thống và phối hợp đa luồng.

1.1.2. Mục tiêu kỹ thuật và giá trị giải trí

- **Mục tiêu kỹ thuật:**
  - *Đồ họa cao cấp:* Ứng dụng tiêu chuẩn ngôn ngữ C++17 cùng thư viện GDI+ để xây dựng hệ thống render. Áp dụng kỹ thuật **Caching System** cho tài

nguyên đồ họa và phương pháp **Flicker-free rendering** (Double Buffering) để đạt trải nghiệm FPS ổn định.

  - *Trí tuệ nhân tạo:* Cài đặt Bot AI với **Alpha-Beta Pruning** kết hợp **Zobrist Hashing** và **Transposition Table** (kèm sắp xếp nước đi), tối ưu không gian tìm kiếm ở mức độ khó.
  - *Xử lý song song:* Hệ thống âm thanh chạy trên luồng phụ với `std::thread` và `std::condition_variable`, xử lý hàng đợi hiệu ứng âm thanh để không gây nghẽn luồng giao diện chính.
  - *Kiến trúc dữ liệu:* Cơ chế lưu/khôi phục trạng thái trận đấu bằng tệp nhị phân, bảo toàn dữ liệu ván chơi.

- **Giá trị trải nghiệm và giải trí:**
  - *Hệ thống nhân vật:* Sử dụng phong cách đồ họa **Pixel Art** mô phỏng các huyền thoại bóng đá (Ronaldo, Messi, Neymar), được render từ các tệp dữ liệu cấu trúc tự định nghĩa thay vì ảnh bitmap tĩnh.
  - *Tính tương tác:* Giao diện hỗ trợ đa ngôn ngữ, tích hợp hệ thống **Match Statistics** (thống kê tỷ lệ kiểm soát, lịch sử nước đi) tạo cảm giác chuyên nghiệp như một giải đấu thực thụ.
  - *Thẩm mỹ hiện đại:* Áp dụng ngôn ngữ thiết kế **Glassmorphism** với các lớp nền kính mờ, gradient và hiệu ứng chuyển động nhẹ (nhấp nháy khán đài, nhấn nhá con trỏ, highlight ô thắng) giúp giao diện sống động và lôi cuốn.

1.2. Phạm vi và ràng buộc

1.2.1. Phạm vi chức năng chính

Đề tài tập trung xây dựng một hệ thống trò chơi hoàn chỉnh với các nhóm chức năng trọng tâm sau:

- **Cơ chế thi đấu:**
  - Hỗ trợ hai chế độ chơi cơ bản: Đối kháng trực tiếp giữa hai người chơi hoặc đối đầu với trí tuệ nhân tạo.
  - Cung cấp hai biến thể luật chơi: Cờ Caro truyền thống trên bàn cờ kích thước 15x15 và Cờ Caro rút gọn trên bàn cờ 3x3.
  - Hệ thống tính điểm theo thể thức chuỗi trận, cho phép người chơi thiết lập mục tiêu thắng trước một số ván nhất định để kết thúc trận đấu.

- **Hệ thống trí tuệ nhân tạo:**
  - Xây dựng ba cấp độ khó khác nhau: Cấp độ dễ sử dụng phương pháp chọn nước đi ngẫu nhiên; cấp độ trung bình và khó ứng dụng thuật toán tìm kiếm tối ưu kết hợp với các bộ hàm lượng giá bàn cờ chuyên sâu.
  - Tích hợp cơ chế lưu trữ các trạng thái bàn cờ đã tính toán để tăng tốc độ phản hồi trong các tình huống phức tạp.
- **Quản lý dữ liệu và trạng thái:**
  - Cho phép người chơi tạm dừng và lưu lại trạng thái trận đấu hiện tại vào bộ nhớ để tiếp tục vào lần sau.
  - Chức năng hoàn tác và thực hiện lại cho phép người chơi điều chỉnh các nước đi sai sót trong quá trình thi đấu.
  - Hệ thống lưu trữ thông tin người dùng bao gồm tên hiệu, hình đại diện và các thông số thống kê trận đấu.
- **Tương tác và giao diện:**
  - Giao diện người dùng hỗ trợ hoàn toàn hai ngôn ngữ: Tiếng Việt và Tiếng Anh.
  - Hệ thống cài đặt cho phép tinh chỉnh các thông số về âm thanh nền, hiệu ứng đặc biệt và thời gian giới hạn cho mỗi lượt đi.

1.2.2. Ràng buộc hiệu năng và nền tảng

Để đảm bảo tính ổn định và trải nghiệm mượt mà, dự án tuân thủ các ràng buộc kỹ thuật sau:

- **Nền tảng phát triển:**
  - Ứng dụng được thiết kế chạy trên hệ điều hành Windows, sử dụng giao diện lập trình ứng dụng Win32 và tiêu chuẩn ngôn ngữ chuyên dụng để đảm bảo hiệu suất tối đa ở mức phần cứng.
- **Hiệu năng xử lý đồ họa:**
  - Hoạt động hiển thị phải đạt tốc độ 60 khung hình trên giây để đảm bảo các hiệu ứng chuyển động và hoạt cảnh diễn ra mượt mà.
  - Áp dụng kỹ thuật vùng đệm kép và cơ chế bộ nhớ đệm tài nguyên để triệt tiêu hiện tượng nháy màn hình và giảm tải cho bộ vi xử lý trung tâm.

- **Xử lý đa luồng và phản hồi:**
  - Các tác vụ nặng như tính toán nước đi của máy hoặc tái tạo âm thanh phải được xử lý trên các luồng độc lập, không được phép gây nghẽn luồng xử lý giao diện chính.
  - Thời gian phản hồi của máy ở cấp độ khó nhất không được vượt quá 2 giây cho mỗi nước đi nhằm duy trì nhịp độ trận đấu.
- **Quản lý tài nguyên:**
  - Toàn bộ dữ liệu trận đấu và cấu hình người dùng được lưu dưới dạng tệp nhị phân nhằm tối ưu hóa không gian lưu trữ và hạn chế can thiệp chỉnh sửa thủ công từ bên ngoài.

1.3. Công nghệ và môi trường triển khai

Dự án được xây dựng dựa trên sự kết hợp giữa hiệu năng mạnh mẽ của ngôn ngữ lập trình hệ thống và tính ổn định của các thư viện đồ họa tích hợp sẵn trên nền tảng Windows.

1.3.1. C++17 và tư duy lập trình mô-đun

- **Tiêu chuẩn ngôn ngữ C++17:** Dự án khai thác các tính năng hiện đại của ngôn ngữ C++17 nhằm tăng cường độ an toàn và hiệu suất xử lý. Việc sử dụng các cấu trúc dữ liệu tối ưu giúp quản lý bộ nhớ hiệu quả và giảm thiểu sai sót trong quá trình thao tác với các thực thể phức tạp (như bảng băm cho AI hoặc bộ đệm tài nguyên đồ họa).
- **Tư duy lập trình mô-đun:** Mã nguồn được phân tách một cách khoa học thành các phân hệ chức năng độc lập, giúp việc bảo trì và mở rộng trở nên dễ dàng:
  - *Phân hệ Logic (`GameLogic`):* Chứa các quy tắc trò chơi và bộ não trí tuệ nhân tạo, hoàn toàn độc lập với phần hiển thị.
  - *Phân hệ Hiển thị (`RenderAPI`):* Quản lý các hàm vẽ cấp thấp, xử lý mô hình điểm ảnh và các hiệu ứng hình ảnh đạo cụ.
  - *Phân hệ Quản lý màn hình (`ScreenModules`):* Điều khiển luồng chuyển động giữa các trạng thái của ứng dụng theo mô hình máy trạng thái hữu hạn.
  - *Phân hệ Hệ thống (`SystemModules`):* Đảm nhiệm các tác vụ phụ trợ như âm thanh, cấu hình tệp tin và đa ngôn ngữ.

1.3.2. Win32 API + GDI+ cho đồ họa 2D

Sự kết hợp giữa Win32 API và thư viện GDI+ mang lại khả năng kiểm soát tuyệt đối trên kiến trúc Windows mà không cần phụ thuộc vào bên thứ ba:

- **Win32 API:** Được sử dụng để thiết lập cửa sổ ứng dụng, quản lý hàng chờ thông điệp hệ thống và xử lý các sự kiện từ bàn phím. Hệ thống sử dụng bộ hẹn giờ độ phân giải cao để đảm bảo nhịp độ logic của trò chơi không bị biến động theo cấu hình phần cứng.
- **GDI+ (Graphics Device Interface Plus):** Đóng vai trò là công cụ render chính cho toàn bộ trò chơi:
  - Hỗ trợ các chế độ trộn màu phức tạp, độ trong suốt cho các thành phần kính mờ và kỹ thuật khử răng cưa giúp hình ảnh Pixel Art sắc nét nhưng vẫn mềm mại.
  - Khả năng xử lý các loại bút vẽ dải màu giúp tạo nên không gian sân vận động có chiều sâu, vượt qua giới hạn của các ứng dụng vẽ cửa sổ thông thường.

1.3.3. Visual Studio, cấu hình build và quản lý phiên bản

- **Môi trường phát triển tích hợp (IDE):** Dự án sử dụng Microsoft Visual Studio 2022. Đây là công cụ hỗ trợ mạnh mẽ nhất cho lập trình Win32 với các tính năng gỡ lỗi trực quan, phân tích hiệu năng và quản lý tài nguyên hệ thống đồng bộ.
- **Cấu hình Build:**
  - *Bộ biên dịch:* Tối ưu hóa cho kiến trúc 64-bit nhằm khai thác tối đa sức mạnh của các bộ vi xử lý hiện đại.
  - *Chế độ tối ưu hóa:* Sử dụng cấu hình phát hành để nén mã nguồn, loại bỏ các chỉ thị gỡ lỗi thừa, giúp ứng dụng có dung lượng nhỏ gọn và tốc độ khởi chạy tức thì.
- **Quản lý phiên bản:** Toàn bộ quá trình phát triển được kiểm soát thông qua hệ thống quản lý phiên bản Git. Điều này giúp nhóm lưu trữ lịch sử thay đổi, phối hợp làm việc trên các nhánh chức năng khác nhau và đảm bảo mã nguồn luôn ở trạng thái ổn định trước khi tích hợp các tính năng mới.

1.4. Tổng quan chức năng hệ thống

Hệ thống được thiết kế nhằm cung cấp một trải nghiệm trò chơi Caro hoàn chỉnh, từ việc thi đấu, quản lý dữ liệu đến bộ máy phân tích số liệu trận đấu chuyên nghiệp.

1.4.1. PvP, PvE và các mức độ trí tuệ nhân tạo

Trò chơi cung cấp khả năng tùy biến linh hoạt giữa các hình thức đối đầu, đáp ứng nhu cầu giải trí cá nhân hoặc thi đấu nhóm:

- **Chế độ Người đấu Người (PvP):** Hai người chơi có thể thi đấu trực tiếp trên cùng một thiết bị. Hệ thống cho phép cá nhân hóa tên hiệu, hình đại diện (theo phong cách các cầu thủ huyền thoại) và chọn quân cờ đại diện.
- **Chế độ Người đấu Máy (PvE):** Người chơi thử thách kỹ năng trước hệ thống AI được xây dựng theo ba cấp độ mang tính phân loại cao:
  - *Cấp độ Đồng (Dễ):* Máy thực hiện các nước đi cơ bản, ưu tiên chặn các chuỗi ngắn và tạo cơ hội cho người chơi làm quen với giao diện.
  - *Cấp độ Vàng (Trung bình):* AI bắt đầu biết dàn trận, nhận diện các thế cờ nguy hiểm và thực hiện các chiến thuật tấn công dồn dập khi có lỗ hổng.
  - *Cấp độ Thách đấu (Khó):* Đây là mức độ cao nhất ứng dụng thuật toán tìm kiếm theo chiều sâu. AI có khả năng đọc trước nhiều nước đi, sử dụng bộ nhớ trạng thái để đưa ra các phản hồi mang tính chiến lược cao, thách thức ngay cả những người chơi chuyên nghiệp.

1.4.2. Lưu/Tải, Hoàn tác/Thực hiện lại và Hệ thống Thống kê

Bên cạnh lõi trò chơi, các chức năng quản trị được thiết kế đồng bộ nhằm tối ưu hóa trải nghiệm người dùng và cung cấp cái nhìn chi tiết về hiệu suất thi đấu:

- **Quản lý trạng thái trận đấu:**
  - *Lưu/Tải:* Hệ thống cung cấp 05 vị trí lưu trữ độc lập (Slot). Dữ liệu được đóng gói vào tệp tin nhị phân (.bin) bao gồm: Toàn bộ ma trận bàn cờ, thông tin chi tiết của hai tuyển thủ (tên, ảnh đại diện, số bàn thắng), lịch sử nước đi và các cấu hình trận đấu (thể thức BoX, độ khó AI). Chức năng này cho phép người chơi không chỉ tải lại mà còn có thể đổi tên nhãn bản lưu hoặc xóa bỏ để tối ưu không gian.

  - *Hoàn tác/Thực hiện lại:* Tính năng này được lập trình thông minh để thích ứng với từng chế độ. Trong chế độ Người đấu Máy, khi thực hiện Hoàn tác, hệ thống sẽ lùi lại 02 bước đi cùng lúc để trả lượt về đúng vị trí của người chơi; trong khi ở chế độ Đối kháng, việc lùi bước diễn ra tuần tự. Toàn bộ các chỉ số về lượt đánh và thời gian đếm ngược sẽ được khôi phục tương ứng.

- **Hệ thống Thống kê:** Phản ánh đúng tinh thần bóng đá, hệ thống ghi nhận và phân tích các chỉ số vận động thực tế của từng tuyển thủ tại màn hình Tổng kết ván đấu:
  - *Dẫn bóng:* Thống kê tổng số nước đi mà cầu thủ đã thực hiện trên sân cỏ, phản ánh cường độ điều phối trận đấu.
  - *Giữ bóng:* Tính toán tổng thời gian tích lũy (theo giây) mà người chơi nắm quyền kiểm soát lượt đi. Đây là chỉ số quan trọng để đánh giá khả năng làm chủ nhịp độ trận đấu.
  - *Bàn thắng:* Số ván thắng mà cầu thủ đã ghi được trong loạt trận hiện tại để tiến tới Cup vô địch.
  - *Thủng lưới:* Số ván thua phản ánh sơ hở trong hệ thống phòng ngự của cầu thủ trước đối phương.
  - *Thể thức chuỗi trận:* Theo dõi số trận thắng chung cuộc (Bo1, Bo3, Bo5) để vinh danh nhà vô địch sau cùng của giải đấu.
  - *Thời lượng trận đấu:* Tổng thời gian thực tế diễn ra cuộc đối đầu, giúp đánh giá độ kịch tính và dai sức của các tuyển thủ.

CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG VÀ TỔ CHỨC MÃ NGUỒN

Dự án được xây dựng theo hai hướng phân rã bổ trợ nhau: phân hệ và phân tầng. Phân hệ dùng để nhóm các thành phần theo chức năng (RenderAPI, GameLogic, ScreenModules, SystemModules, ApplicationTypes, Asset), từ đó cô lập trách nhiệm, giảm phụ thuộc chéo và giúp mở rộng từng mảng độc lập. Phân tầng là cách tổ chức theo luồng xử lý (hiển thị, logic, hệ thống), làm rõ ranh giới dữ liệu đi qua các tầng và kiểm soát chiều phụ thuộc từ trên xuống. Nhờ kết hợp cả hai, mã nguồn vừa có cấu trúc rõ ràng ở cấp thư mục/chức năng, vừa có kỷ luật kiến trúc ở cấp luồng xử lý, giúp dễ đọc, dễ gỡ lỗi và thuận tiện mở rộng.

2.1. Kiến trúc phân tầng

Kiến trúc tổng thể của hệ thống được chia thành ba tầng chính, hoạt động phối hợp thông qua một bộ điều phối trung tâm tại lớp màn hình.

2.1.1. Render Layer và Logic Layer

Đây là hai tầng quan trọng nhất, đảm nhiệm vai trò hiển thị và xử lý ”linh hồn” của trò chơi:

- **Render Layer (Tầng hiển thị - Thư mục RenderAPI):**
  - *Vai trò:* Chịu trách nhiệm vẽ toàn bộ các thành phần hình ảnh lên màn hình bằng thư viện GDI+.

  - *Thành phần cốt lõi:* Các mô-đun như `Renderer` (quản lý vùng đệm), `UIComponents` (vẽ các thành phần giao diện, Pixel Art) và `UIScaler` (tính toán tỷ lệ màn hình động).
  - *Kỹ thuật đặc trưng:* Sử dụng hệ thống bộ nhớ đệm để lưu trữ các đối tượng đồ họa nặng (Bitmap, Brush), giúp tốc độ phản hồi đồ họa cực nhanh và mượt mà. Tầng này chỉ nhận dữ liệu thô và thực hiện việc vẽ, không can thiệp vào logic thắng thua.

- **Logic Layer (Tầng xử lý - Thư mục GameLogic):**
  - *Vai trò:* Đảm nhiệm việc kiểm soát luật chơi, trạng thái bàn cờ và bộ não trí tuệ nhân tạo.
  - *Thành phần cốt lõi:* `GameRules` (kiểm tra điều kiện thắng), `GameEngine` (đầu não điều phối trận đấu) và `BotAI` (xử lý thuật toán tìm kiếm nước đi).
  - *Đặc điểm:* Tách biệt hoàn toàn với môi trường đồ họa. Tầng Logic làm việc trực tiếp trên các cấu hình dữ liệu và ma trận trạng thái, cho phép hệ thống tính toán hàng triệu trạng thái mà không bị ảnh hưởng bởi tốc độ render của giao diện.

2.1.2. System Layer: Tài nguyên, cấu hình và thời gian

Tầng hệ thống (Thư mục SystemModules) đóng vai trò là ”hậu cần”, cung cấp các dịch vụ nền tảng cho toàn bộ ứng dụng:

- **Quản lý Tài nguyên và Đa ngôn ngữ (Localization):**
  - Mô-đun `Localization` chịu trách nhiệm tải và ánh xạ các chuỗi văn bản từ tệp tin bên ngoài vào ứng dụng. Điều này cho phép chuyển đổi ngôn ngữ (Vietnamese/English) ngay lập tức mà không cần biên dịch lại mã nguồn.
- **Hệ thống cấu hình (ConfigLoader):**
  - Sử dụng tệp `config.ini` để lưu trữ các tùy chỉnh của người dùng như âm lượng, ngôn ngữ, trạng thái bật/tắt nhạc nền. Cơ chế này giúp trò chơi ”ghi nhớ” được trạng thái của người chơi qua các lần khởi động khác nhau.
- **Hệ thống Âm thanh (AudioSystem):**
  - Đây là một dịch vụ đặc biệt được vận hành theo cơ chế không đồng bộ. Âm thanh được quản lý bởi một luồng tách biệt, giúp các hiệu ứng tiếng vang, nhạc nền hay tiếng hò reo của khán giả có thể phát ra liên tục mà không làm gián đoạn luồng xử lý nước đi của người chơi.

- **Quản lý Thời gian (TimeSystem):**
  - Sử dụng thư viện `chrono` để tính toán chính xác thời gian đếm ngược và tổng thời lượng trận đấu. Hệ thống đảm bảo sự công bằng tuyệt đối trong thi đấu, đặc biệt là khi áp dụng các ràng buộc về thời gian suy nghĩ cho mỗi tuyển thủ.

2.2. Luồng xử lý sự kiện

Để đảm bảo hiệu năng và sự mượt mà trong các thao tác tương tác, hệ thống ứng dụng một cơ chế vận hành theo thời gian thực thay vì mô hình hướng sự kiện thụ động thông thường.

2.2.1. Vòng lặp trò chơi và Thời gian sai lệch

- **Vòng lặp trò chơi:** Tại tệp hệ thống chính (`main.cpp`), ứng dụng sử dụng cơ chế kiểm tra thông điệp không chặn thông qua hàm `PeekMessage`. Điều này cho phép ứng dụng liên tục thực hiện các tác vụ cập nhật logic và dựng hình ngay cả khi không có tác động từ người dùng.
- **Thời gian sai lệch:** Để đảm bảo các hiệu ứng chuyển động và hoạt cảnh diễn ra với tốc độ đồng nhất trên mọi cấu hình máy tính, hệ thống sử dụng thư viện `chrono` để tính toán chính xác khoảng thời gian giữa hai khung hình liên tiếp. Chỉ số này được dùng để nhân với vận tốc của các hiệu ứng (như xung nhịp của bộ đệm, hiệu ứng sóng rung), giúp trò chơi vận hành ổn định tại mức khung hình mục tiêu là 60 FPS.

2.2.2. Kỹ thuật vùng đệm kép chống nhấp nháy

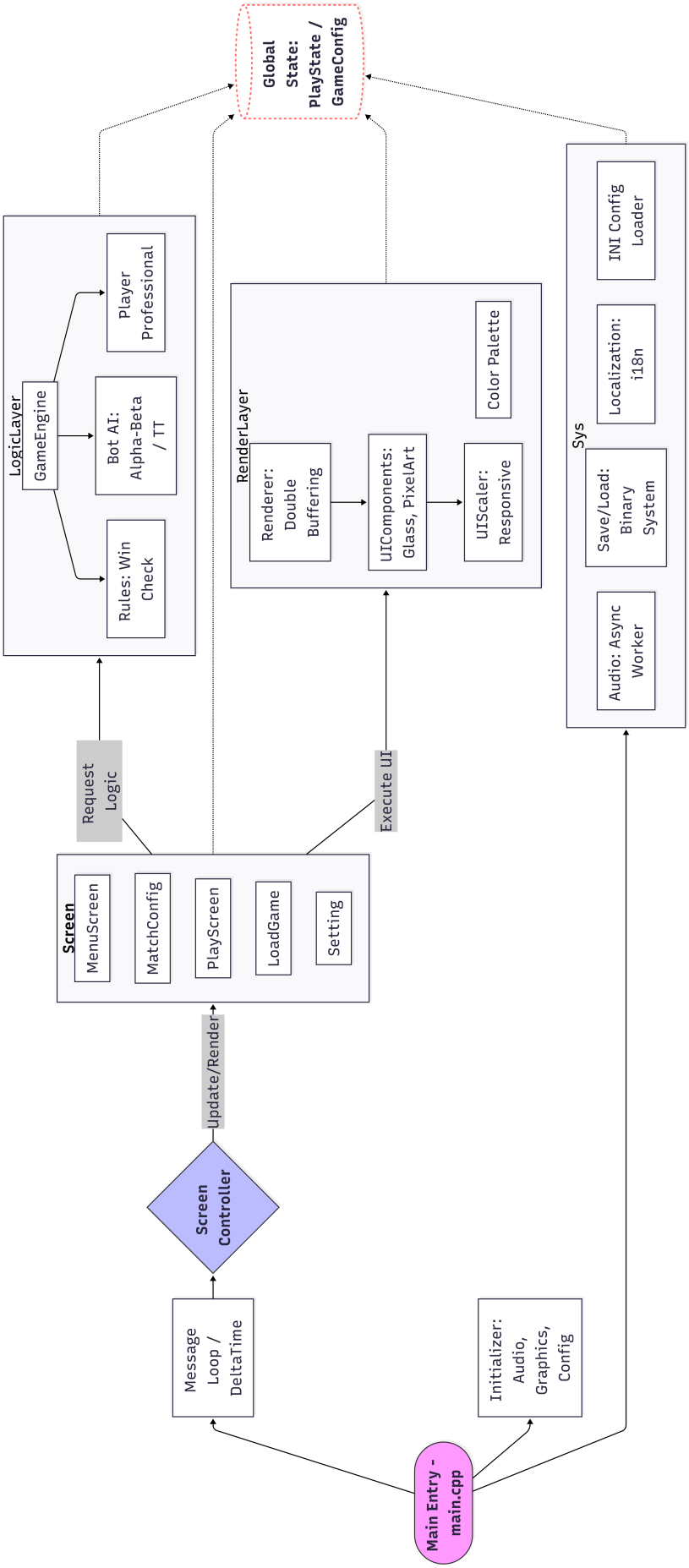
Nhược điểm của việc dựng hình trực tiếp lên màn hình trong Windows là hiện tượng nhấp nháy do quá trình xóa và vẽ lại diễn ra liên tục. Dự án giải quyết triệt để vấn đề này bằng kỹ thuật vùng đệm kép:

- **Cấu trúc vùng đệm:** Toàn bộ quá trình vẽ từ thư viện GDI+ được thực hiện lên một ngữ cảnh thiết bị ảo trong bộ nhớ (Back Buffer) thay vì trực tiếp lên cửa sổ.
- **Cơ chế hoán đổi:** Chỉ khi tất cả các thành phần đồ họa (bàn cờ, quân cờ, hiệu ứng) đã được vẽ hoàn tất lên vùng đệm, hệ thống mới thực hiện một lệnh sao chép khối lượng lớn (`BitBlt`) duy nhất để đẩy toàn bộ hình ảnh lên màn hình.

- **Tối ưu hóa:** Ứng dụng can thiệp vào sự kiện xóa nền hệ thống để ngăn chặn Windows tự ý xóa màn hình trước mỗi khung hình, từ đó tạo ra trải nghiệm hình ảnh cực kỳ ổn định và không có độ trễ.

2.2.3. Sơ đồ kiến trúc hệ thống

Dưới đây là sơ đồ chi tiết về mối liên kết giữa các mô-đun trong hệ thống. Kiến trúc được thiết kế theo mô hình ”Trung tâm trạng thái”, nơi thông tin về trạng thái trận đấu (`PlayState`) được chia sẻ giữa các tầng xử lý và tầng hiển thị:



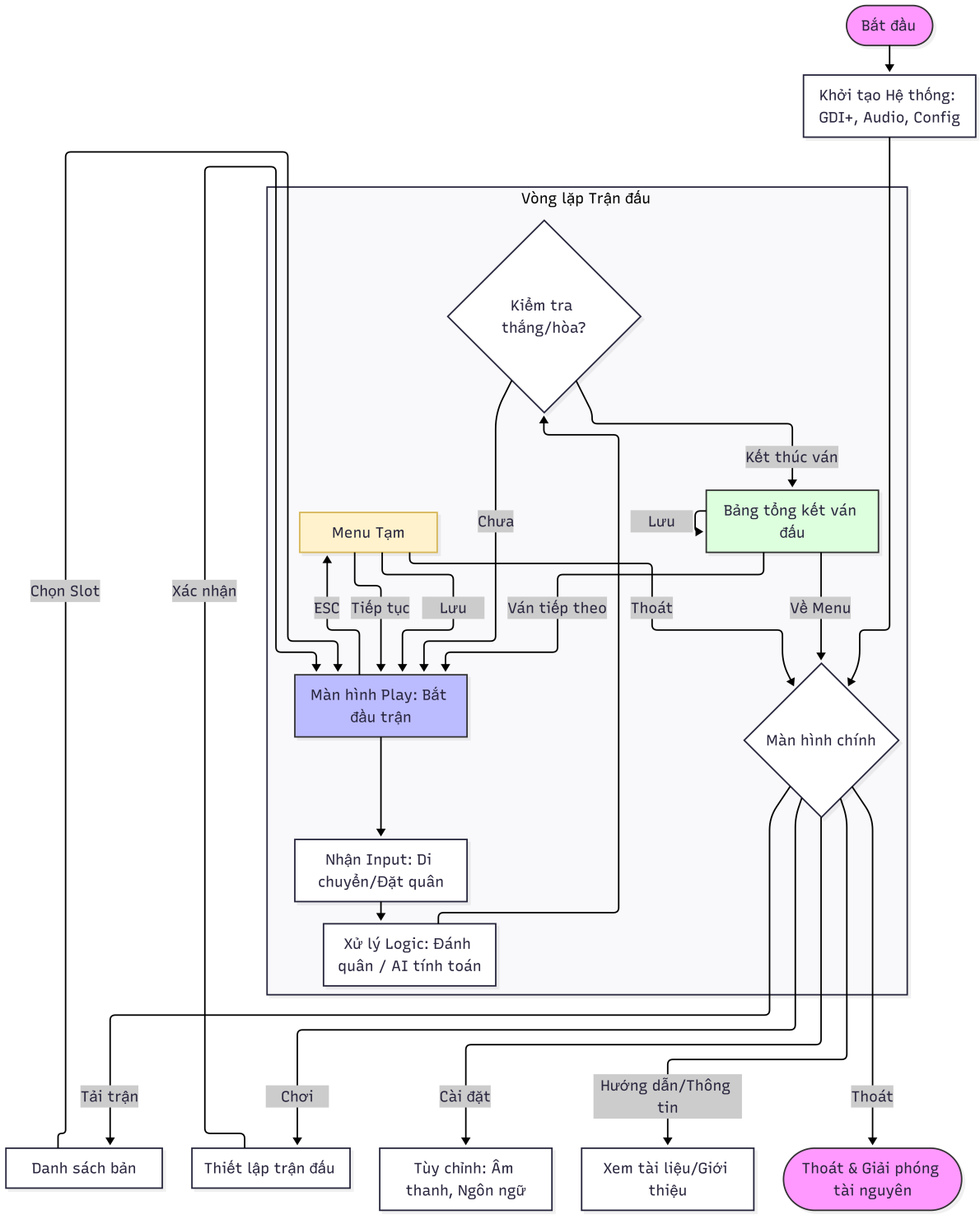
Hình 2.1: Sơ đồ kiến trúc hệ thống

Diễn giải chi tiết các mối quan hệ:

- **Luồng khởi tạo và vòng lặp chính:** Từ `main.cpp`, khối khởi tạo nạp Audio/-Graphics/Config, sau đó đi vào *Message Loop/DeltaTime*. Nhịp cập nhật này là nguồn kích hoạt cho *Screen Controller*.
- **Tầng điều phối (Screen Controller):** Bộ điều phối chọn màn hình hiện tại (`MenuScreen`, `MatchConfig`, `PlayScreen`, `LoadGame`, `Setting`) và gọi *Update/Render*. Khi cần xử lý nước đi hoặc luật, nó gửi *Request Logic* sang `GameEngine`.
- **LogicLayer:** `GameEngine` điều phối luật và lượt chơi, gọi `GameRules` để kiểm tra thắng, dùng `BotAI` (Alpha-Beta/TT) cho PvE và `PlayerEngineer` để xử lý thao tác người chơi. Tầng này chỉ làm việc với dữ liệu bàn cờ/trạng thái.
- **RenderLayer:** `Renderer` dựng khung hình bằng double buffering, `UIComponents` vẽ Glass/PixelArt, `UIScaler` đảm bảo responsive, và `Colours` cung cấp bảng màu đồng nhất.
- **Sys:** Các dịch vụ nền gồm Audio Async Worker, Save/Load Binary, Localization (i18n) và INI Config Loader, được các màn hình và logic gọi theo nhu cầu.
- **Global State:** `PlayState` và `GameConfig` là nguồn dữ liệu trung tâm. Logic cập nhật, Render đọc để vẽ, còn Sys đọc/ghi khi lưu/tải hoặc nạp cấu hình, bảo đảm tính nhất quán giữa các tầng.

2.2.4. Sơ đồ luồng chương trình

Sơ đồ luồng dưới đây mô tả quá trình từ khi khởi chạy ứng dụng, điều hướng qua các màn hình chức năng cho đến khi kết thúc một vòng đời của trận đấu và thoát chương trình:



Hình 2.2: Sơ đồ luồng chương trình

Mô tả các nút thắt quyết định:

- **Khởi tạo hệ thống:** Bắt đầu → khởi tạo GDI+, Audio, Config. Sau khi sẵn sàng, chương trình chuyển vào luồng điều hướng chính.
- **Điều hướng từ Màn hình chính:**

  - *Chơi* → Thiết lập trận đấu → Màn hình Play (bắt đầu trận).
  - *Tải trận* → Danh sách bản lưu → Chọn slot → Xác nhận → Màn hình Play.
  - *Cài đặt* → Tùy chỉnh âm thanh/ngôn ngữ.
  - *Hướng dẫn/Thông tin* → Xem tài liệu/giới thiệu.
  - *Thoát* → Giải phóng tài nguyên và kết thúc.

- **Vòng lặp trận đấu (Match Loop):** Màn hình Play nhận input di chuyển/đặt quân → logic xử lý (đánh quân hoặc AI tính toán) → kiểm tra thắng/hòa. Nếu *chưa* kết thúc, quay lại vòng lặp; nếu *kết thúc ván*, chuyển sang Bảng tổng kết.
- **Menu tạm và Lưu:** Trong khi chơi, nhấn *ESC* mở Menu tạm, cho phép tiếp tục, lưu, hoặc thoát về màn hình chính. Lưu sẽ ghi `PlayState` ra file để có thể tải lại ở lần sau.
- **Tổng kết và điều hướng sau ván:** Bảng tổng kết ván đấu cho phép lưu, chơi ván tiếp theo (quay lại Match Loop), hoặc trở về Màn hình chính/thoát.

2.3. Cấu trúc thư mục và phân hệ

Dự án được tổ chức theo cấu trúc phân rã chức năng nghiêm ngặt, giúp việc quản lý hàng chục tệp tin mã nguồn trở nên khoa học và tránh hiện tượng chồng chéo định nghĩa.

Cấu trúc thư mục tổng quát của mã nguồn được thể hiện như sau:

```
src/
|---- ApplicationTypes/
|     |---- GameConfig.h
|     |---- GameConstants.h
|     |---- GameState.h
|     |---- PlayState.h
|---- GameLogic/
|     |---- BotAI.cpp
|     |---- BotAI.h
|     |---- GameEngine.cpp
|     |---- GameEngine.h
|     |---- GameRules.cpp
|     |---- GameRules.h
|     |---- PlayerEngineer.cpp
|     |---- PlayerEngineer.h
|---- RenderAPI/
|     |---- Colours.cpp
|     |---- Colours.h
```

```
|     |---- Renderer.cpp
|     |---- Renderer.h
|     |---- UIComponents.cpp
|     |---- UIComponents.h
|     |---- UIScaler.cpp
|     |---- UIScaler.h
|---- ScreenModules/
|     |---- AboutScreen.cpp
|     |---- AboutScreen.h
|     |---- GuildScreen.cpp
|     |---- GuildScreen.h
|     |---- LoadGameScreen.cpp
|     |---- LoadGameScreen.h
|     |---- MatchConfigScreen.cpp
|     |---- MatchConfigScreen.h
|     |---- MenuScreen.cpp
|     |---- MenuScreen.h
|     |---- PlayScreen.cpp
|     |---- PlayScreen.h
|     |---- SettingScreen.cpp
|     |---- SettingScreen.h
|---- SystemModules/
|     |---- AudioSystem.cpp
|     |---- AudioSystem.h
|     |---- ConfigLoader.cpp
|     |---- ConfigLoader.h
|     |---- Localization.cpp
|     |---- Localization.h
|     |---- SaveLoadSystem.cpp
|     |---- SaveLoadSystem.h
|     |---- TimeSystem.cpp
|     |---- TimeSystem.h
|---- Asset/
|     |---- config.ini
|     |---- audio/
|     |---- font/
|     |---- lang/
|     |---- models/
|     |---- save/
|---- main.cpp
|---- src.sln
|---- src.vcxproj
|---- src.vcxproj.filters
```

2.3.1. Chi tiết các phân hệ: RenderAPI, GameLogic, ScreenModules và SystemModules

Mã nguồn và tài nguyên được chia thành 6 thư mục cốt lõi, mỗi thư mục đảm nhiệm một vai trò chuyên biệt trong vòng đời của ứng dụng. Dưới đây là mô tả đầy đủ theo từng phân hệ, nêu rõ trách nhiệm, phạm vi dữ liệu và các mô-đun chính:

- **Phân hệ ApplicationTypes (Nền tảng dữ liệu):** Đây là nơi định nghĩa các kiểu dữ liệu và hằng số dùng chung cho toàn bộ dự án, làm "hợp đồng dữ liệu" giữa các tầng. Các tệp tiêu biểu gồm:
  - `GameConfig.h`: lưu cấu hình trận đấu (chế độ chơi, kích thước bàn cờ, thời gian, âm lượng) để các phân hệ khác đọc thống nhất.
  - `GameConstants.h`: tập hợp các hằng số hệ thống (kích thước ô, giới hạn mảng, mã trạng thái) nhằm tránh rải rác magic number.
  - `GameState.h`: định nghĩa trạng thái tổng quát của ứng dụng và trận đấu (ví dụ: đang ở menu, đang chơi, đang tổng kết).
  - `PlayState.h`: cấu trúc trung tâm lưu trạng thái bàn cờ, lượt chơi, lịch sử nước đi, thông số người chơi/AI.

  Việc tập trung dữ liệu tại đây giúp đảm bảo tính nhất quán khi các phân hệ khác cùng truy cập và cập nhật.

- **Phân hệ RenderAPI (Thư viện đồ họa nội bộ):** Đóng vai trò là lớp vỏ bọc cho GDI+, cung cấp các công cụ vẽ bậc cao và nhất quán giao diện:
  - `Colours`: quản lý bảng màu, đảm bảo toàn bộ giao diện dùng cùng hệ màu (nút, nền, hiệu ứng).
  - `Renderer`: khởi tạo ngữ cảnh đồ họa, quản lý back buffer và thực hiện quy trình vẽ khung hình.
  - `UIComponents`: dựng các thành phần giao diện (bàn cờ, quân cờ, nút) từ dữ liệu mô tả.
  - `UIScaler`: tính toán tỷ lệ và chuyển đổi tọa độ nhằm hỗ trợ đa độ phân giải và tỷ lệ màn hình.

  Tầng này chỉ nhận dữ liệu đã được chuẩn hóa và không tự ý thay đổi logic trận đấu.

- **Phân hệ GameLogic (Đầu não xử lý):** Tập trung vào quy tắc, điều phối trận đấu và trí tuệ nhân tạo:

  - `GameEngine`: điều phối vòng đời trận đấu, xử lý đặt quân, chuyển lượt, kiểm soát trạng thái và lịch sử nước đi.
  - `GameRules`: kiểm tra điều kiện thắng, hòa, hợp lệ nước đi theo hàng ngang, dọc, chéo.
  - `BotAI`: tính toán nước đi tối ưu cho chế độ PvE, tách biệt khỏi phần đồ họa.
  - `PlayerEngineer`: xử lý thao tác người chơi và chuyển dữ liệu đầu vào thành nước đi trước khi chuyển sang `GameEngine`.

  Mọi xử lý trong phân hệ này chỉ làm việc với dữ liệu thuần (ma trận bàn cờ, trạng thái), không phụ thuộc GDI+.

- **Phân hệ ScreenModules (Quản lý màn hình):** Mỗi tệp trong thư mục này tương ứng một màn hình cụ thể, chịu trách nhiệm xử lý input, điều hướng và cập nhật trạng thái hiển thị:
  - `MenuScreen`: màn hình chính, điều hướng đến các chế độ chơi và thiết lập.
  - `PlayScreen`: màn hình trận đấu, kết nối trực tiếp với `GameEngine` và `Renderer`.
  - `MatchConfigScreen`: cấu hình trận đấu (chế độ, nhân vật, luật chơi).
  - `LoadGameScreen`: tải trận đã lưu, khôi phục `PlayState`.
  - `SettingScreen`: điều chỉnh âm lượng, ngôn ngữ, tùy chọn hiển thị.
  - `AboutScreen` và `GuildScreen`: các màn hình thông tin/bảng thành tích.

  Cách tổ chức này giúp mỗi màn hình có vòng đời rõ ràng, tránh chồng chéo xử lý.

- **Phân hệ SystemModules (Dịch vụ nền):** Cung cấp các dịch vụ hỗ trợ cho toàn ứng dụng:
  - `AudioSystem`: quản lý âm thanh đa luồng, phát nhạc nền và hiệu ứng.
  - `ConfigLoader`: tải và lưu cấu hình người dùng từ `config.ini`.
  - `Localization`: nạp và ánh xạ chuỗi đa ngôn ngữ từ tệp tài nguyên.
  - `SaveLoadSystem`: lưu/khôi phục `PlayState` xuống ổ đĩa.
  - `TimeSystem`: cung cấp bộ đếm thời gian và đo độ trễ khung hình.

  Các dịch vụ này độc lập với logic trận đấu và được gọi theo nhu cầu của các màn hình.

- **Phân hệ Asset (Tài nguyên dữ liệu):** Lưu trữ toàn bộ tài nguyên đầu vào của trò chơi, tách biệt khỏi mã nguồn để dễ cập nhật:
  - `config.ini`: cấu hình người dùng và thiết lập mặc định.

- `audio/`: tệp nhạc nền và hiệu ứng âm thanh.
- `font/`: phông chữ hiển thị trong giao diện.
- `lang/`: tệp ngôn ngữ (Việt/Anh) dùng cho `Localization`.
- `models/`: dữ liệu pixel art, hình nền, vật thể.
- `save/`: dữ liệu lưu trận và lịch sử chơi.

Thư mục này giúp tách biệt nội dung và logic, cho phép thay đổi tài nguyên mà không cần biên dịch lại.

2.3.2. Quy ước Header, Cấu trúc dữ liệu và Đóng gói dữ liệu

Để duy trì tính ổn định của mã nguồn, dự án áp dụng các quy chuẩn lập trình đồng bộ, đảm bảo khả năng mở rộng và giảm lỗi phát sinh khi nhiều mô-đun cùng phát triển:

- **Quy ước tệp tiêu đề (Header Guards):** Tất cả các tệp `.h` đều dùng `#ifndef / #define / #endif` để chống khai báo trùng lặp. Tên guard được đặt theo tên file và phân hệ để tránh va chạm (ví dụ: `GAMELOGIC_GAMEENGINE_H`).
- **Tổ chức Cấu trúc và Kiểu liệt kê (Struct/Enum):**
  - Enum được dùng để mô tả trạng thái màn hình, chế độ chơi và trạng thái trận đấu, giúp logic điều phối rõ ràng và tránh dùng số "ẩn".
  - Struct được thiết kế theo hướng dữ liệu thuần (POD), bao gồm các trường cần thiết cho hiển thị và xử lý (bàn cờ, lượt chơi, điểm, thời gian), không nhúng thuật toán bên trong.
  - Các struct trong `ApplicationTypes` là nguồn dữ liệu chuẩn cho cả `GameLogic` và `RenderAPI`, tránh trùng lặp định nghĩa.
- **Cơ chế đóng gói và Quản lý biến toàn cục:**
  - Các biến trạng thái dùng chung (như cấu hình và trạng thái trận) được khai báo `extern` trong header và định nghĩa duy nhất tại tệp triển khai để mọi mô-đun truy cập đồng nhất.
  - Biến nội bộ của từng màn hình được giới hạn phạm vi bằng `static` trong file `.cpp` để tránh va chạm tên và giảm rủi ro chỉnh sửa ngoài ý muốn.
  - Mọi thay đổi lên dữ liệu dùng chung đều đi qua các hàm điều phối (ví dụ: từ `GameEngine` hoặc `ScreenModules`) để đảm bảo thứ tự cập nhật nhất quán.
- **Tách biệt giao diện và thực thi:** Các tệp `.h` chỉ khai báo lớp/hàm cần thiết, còn phần triển khai đặt trong `.cpp`. Điều này giúp giảm phụ thuộc chéo, rút ngắn thời gian biên dịch và tạo giao diện lập trình rõ ràng giữa các phân hệ.

- **Quy ước lưu trữ và tuần tự hóa dữ liệu:** Dữ liệu trận đấu được lưu dưới dạng nhị phân thông qua `SaveLoadSystem`, đảm bảo tốc độ đọc/ghi cao và khôi phục đúng trạng thái. Cấu hình người dùng được lưu trong `config.ini` để dễ chỉnh sửa và tương thích lâu dài.

CHƯƠNG 3. THIẾT KẾ DỮ LIỆU VÀ CẤU TRÚC CỐT LÕI

3.1. Quản lý trạng thái ván đấu

Lõi dữ liệu của toàn bộ hệ thống được gom vào cấu trúc `PlayState`. Cấu trúc này lưu tất cả thông tin cần thiết để khôi phục ván đấu, điều phối lượt đi, hiển thị giao diện và hỗ trợ lưu/tải. Dữ liệu được thiết kế theo hướng POD để tối ưu hiệu năng và đơn giản hóa việc tuần tự hóa.

3.1.1. Ma trận bàn cờ và tối ưu bộ nhớ

Bàn cờ được lưu dưới dạng mảng tĩnh hai chiều, kết hợp các hằng số hệ thống trong `GameConstants.h`. Thiết kế này tránh cấp phát động lặp lại, giúp truy cập theo chỉ số O(1) và giảm overhead khi kiểm tra luật thắng.

- **Kích thước và giới hạn:** `GOMOKU_SIZE = 15`, `WIN_CONDITION = 5`, `MAX_BOARD_SIZE = 20` đảm bảo đủ vùng đệm cho mọi chế độ.
- **Giá trị ô:** mỗi ô mang một trong ba trạng thái `CELL_EMPTY`, `CELL_PLAYER1`, `CELL_PLAYER2`.
- **Kích thước thực tế:** `boardSize` cho phép chuyển giữa Caro (15x15) và TTT (3x3) mà không đổi cấu trúc lưu trữ.

Mã nguồn 1: Mã nguồn Hằng số và định nghĩa trạng thái ô bàn cờ

STT	Mã nguồn
1	`#define GOMOKU_SIZE 15`
2	`#define WIN_CONDITION 5`
3	`#define MAX_BOARD_SIZE 20`
4	
5	`#define CELL_EMPTY 0`
6	`#define CELL_PLAYER1 1`
7	`#define CELL_PLAYER2 2`
8	
9	`#define MAX_SAVE_SLOTS 5 // Giới hạn 5 slot lưu game`
10	`#define MAX_SAVE_NAME_LEN 15 // Giới hạn 15 ký tự cho tên/thông tin hiển thị`

Mã nguồn 2: Mã nguồn Ma trận bàn cờ trong PlayState

STT	Mã nguồn
1	`int boardSize = 15;`
2	`// Ma trận lưu giá trị: CELL_EMPTY, CELL_PLAYER1, hoặc CELL_PLAYER2`
3	`int board[MAX_BOARD_SIZE][MAX_BOARD_SIZE] = 0;`

3.1.2. Turn-based logic và Match Status

Vòng đời ván đấu được điều phối bằng các trường trạng thái trong `PlayState` và enum `MatchStatus`. Trạng thái này là cơ sở để GameEngine quyết định xử lý nước đi, tạm dừng hoặc tổng kết.

- **Lượt chơi:** `isP1Turn` quyết định quân cờ đang được đặt và cập nhật đếm lượt.
- **Trạng thái ván:** `MATCH_PLAYING`, `MATCH_PAUSED`, `MATCH_FINISHED`, `MATCH_SUMMARY`.
- **Kết quả:** `winner` lưu giá trị `-1` (đang chơi), `0` (hòa), `1` (P1), `2` (P2).

- **Thời gian:** `countdownTime` và `timeRemaining` quản lý thời gian lượt; `isMatchTimed`, `p1TotalTimeLeft`, `p2TotalTimeLeft` quản lý thời gian toàn trận.

Mã nguồn 3: Mã nguồn Enum MatchStatus trong PlayState

STT	Mã nguồn
1	`enum MatchStatus`
2	`{`
3	`  MATCH_PLAYING,`
4	`  MATCH_PAUSED,`
5	`  MATCH_FINISHED,`
6	`  MATCH_SUMMARY`
7	`};`

3.1.3. Lịch sử nước đi và cơ chế Undo/Redo

Dữ liệu lịch sử được lưu trong `matchHistory` và `redoStack`. Điều này cho phép hoàn tác và thực hiện lại một cách nhất quán giữa PvP và PvE.

- **matchHistory:** chuỗi các tọa độ đã đánh, dùng để khôi phục và tính toán kết quả.
- **redoStack:** lưu các nước đã hoàn tác để có thể thực hiện lại.
- **lastMoveRow/Col và winningCells:** phục vụ hiển thị nước đi cuối và highlight chuỗi thắng.

Mã nguồn 4: Mã nguồn Quy tắc Undo cho PvE (lùi 2 nước)

STT	Mã nguồn
1	`void undoMove(PlayState* state) {`
2	`  if (state->matchHistory.empty()) return;`
3	
4	`  // PVE: lùi 2 bước để trả lượt về người chơi`
5	`  int popCount = (state->matchType == MATCH_PVE`
6	`    && state->matchHistory.size() >= 2) ? 2 : 1;`

3.1.4. Kiểm tra điều kiện thắng và trạng thái kết thúc

Luật thắng được xử lý trong `GameRules::checkWinCondition`. Hệ thống tự động chọn độ dài thắng theo chế độ (Caro: 5, TTT: 3), quét 4 hướng và trả về toàn bộ chuỗi thắng để hiển thị.

Mã nguồn 5: Mã nguồn Xác định độ dài thắng theo chế độ chơi

STT	Mã nguồn
1	`const int winLength = (state->gameMode == MODE_CARO) ? 5 : 3;`
2	`const int size = state->boardSize;`

Mã nguồn 6: Mã nguồn Trả về toàn bộ chuỗi thắng để highlight

STT	Mã nguồn
1	`if ((int)line.size() >= winLength) {`
2	`  if (outWinCells)`
3	`  *outWinCells = line; // Trả về toàn bộ chuỗi, không cắt`
4	`  return currentPiece;`
5	`}`

3.2. Mô hình người chơi

Thông tin người chơi được chuẩn hóa qua `PlayerInfo2`. Cấu trúc này tập trung các thống kê phục vụ hiển thị, tính điểm và lưu/tải.

3.2.1. PlayerInfo2: Wins, Moves, Possession Time

- **name, avatarPath, piece:** thông tin định danh và quân cờ đại diện.
- **totalWins:** số ván thắng trong series BO hiện tại.
- **matchWins:** số series BO đã thắng.
- **movesCount:** tổng số nước đã đi trong ván.

- **maxTurnTime, totalTimePossessed:** giới hạn thời gian lượt và tổng thời gian giữ lượt.

Mã nguồn 7: Mã nguồn Cấu trúc PlayerInfo2

STT	Mã nguồn
1	`struct PlayerInfo2`
2	`{`
3	`  wstring name;`
4	`  string avatarPath;`
5	`  char piece = ' '; // 'X' hoặc 'O'`
6	`  int totalWins = 0; // Số Bàn Thắng (trong series BO hiện tại)`
7	`  int matchWins = 0; // Số Trận Thắng BO (đã thắng trọn 1 serie BO)`
8	`  int movesCount = 0;`
9	`  float maxTurnTime = 0.0f; // Thời gian tối đa cho 1 lượt`
10	`  float totalTimePossessed = 0.0f; // Tổng thời gian giữ bóng (giây), cộng dồn theo phiên`
11	`};`

3.2.2. PlayerEngineer và vòng đời thực thể

`PlayerEngineer` đóng vai trò khởi tạo và reset chỉ số theo vòng đời ván đấu. Các thao tác này được GameEngine gọi khi bắt đầu match hoặc round mới.

- **initPlayer:** gán tên, avatar, quân cờ và reset các thống kê cần thiết.
- **resetPlayerForRound:** chỉ reset thống kê theo round (ví dụ `movesCount`), giữ nguyên tiến trình series BO.

Mã nguồn 8: Mã nguồn Reset thống kê theo round trong PlayerEngineer

STT	Mã nguồn
1	`void resetPlayerForRound(PlayerInfo2& player) {`
2	`  player.movesCount = 0;`
3	`  // Lưu ý: totalWins và matchWins được giữ nguyên để tính tiến trình series BO`
4	`}`

3.3. Cấu hình và môi trường

3.3.1. Nạp Config và thiết lập mặc định

Thông số hệ thống (âm thanh, ngôn ngữ) được gói trong `GameConfig`. `ConfigLoader` đọc/ghi trực tiếp cấu trúc này theo dạng nhị phân và tự tạo mặc định nếu thiếu tệp.

- **Audio:** `isBgmEnabled`, `bgmVolume`, `isSfxEnabled`, `sfxVolume`.
- **Language:** `currentLang` với hai lựa chọn `APP_LANG_VI` và `APP_LANG_EN`.

Mã nguồn 9: Mã nguồn Thiết lập mặc định trong ConfigLoader

STT	Mã nguồn
1	`static void SetDefaultConfig(GameConfig* config) {`
2	`  config->isBgmEnabled = true;`
3	`  config->bgmVolume = 50;`
4	`  config->isSfxEnabled = true;`
5	`  config->sfxVolume = 50;`
6	`  config->currentLang = APP_LANG_VI;`
7	`}`

3.3.2. Lưu/Tải trạng thái và metadata

Hệ thống lưu/tải được triển khai trong `SaveLoadSystem`. Mỗi bản lưu được quản lý theo slot, kèm metadata để hiển thị ở màn hình Load Game. File save có magic number và version để kiểm tra tính hợp lệ.

- **MAX_SAVE_SLOTS = 5:** giới hạn số slot lưu.
- **SaveMetadata:** lưu tên, thời gian, chế độ, điểm và độ khó.
- **SAVE_MAGIC / SAVE_VERSION:** dùng để kiểm tra định dạng file và tương thích phiên bản.

Mã nguồn 10: Mã nguồn Cấu trúc metadata cho slot lưu

STT	Mã nguồn
1	`struct SaveMetadata {`
2	`  std::wstring name;`
3	`  std::wstring timestamp;`
4	`  int mode; // 0: Caro, 1: TTT`
5	`  int type; // 0: PvP, 1: PvE`
6	`  int p1Wins;`
7	`  int p2Wins;`
8	`  int difficulty;`
9	`  int boardSize;`
10	`  bool exists;`
11	`  uint32_t version;`
12	`};`

Mã nguồn 11: Mã nguồn Magic number và phiên bản file save

STT	Mã nguồn
1	`static const uint32_t SAVE_MAGIC = 0xCA05A1E2;`
2	`static const uint32_t SAVE_VERSION = 5;`

3.3.3. Hệ thống màu sắc và theme động

Hệ thống màu được chuẩn hóa qua `SmartColor`, `Palette` và `Theme`. Điều này giúp giao diện đồng nhất, dễ mở rộng, và hỗ trợ pha trộn alpha cho hiệu ứng kính mờ và highlight.

- **SmartColor:** cấu trúc ARGB dạng POD, dùng chung cho GDI và GDI+.
- **Palette:** tập hợp màu cơ bản (đen, trắng, xám, đỏ, xanh, cyan...) và các dải màu phụ trợ.
- **Theme:** màu theo ngữ nghĩa (panel, pitch, highlight, border, turn pulse).

Mã nguồn 12: Mã nguồn Cấu trúc SmartColor

STT	Mã nguồn
1	`struct SmartColor`
2	`{`
3	`  BYTE a;`
4	`  BYTE r;`
5	`  BYTE g;`
6	`  BYTE b;`
7	`};`

Mã nguồn 13: Mã nguồn Một số màu chủ đạo trong Theme

STT	Mã nguồn
1	`constexpr SmartColor GlassWhite = {220, 255, 255, 255}; // Panel trắng mờ`
2	`constexpr SmartColor GlassDark = {200, 15, 20, 30}; // Panel tối (pause)`
3	`constexpr SmartColor GlassGleam = {150, 255, 255, 255}; // Viền trắng mờ`
4	
5	`constexpr SmartColor P1TurnPulse = {30, 255, 150, 0};`
6	`constexpr SmartColor P1TurnBorder = {200, 255, 150, 0};`
7	`constexpr SmartColor P1Watermark = {60, 255, 150, 0};`
8	

STT	Mã nguồn
9	`constexpr SmartColor P2TurnPulse = {30, 0, 255, 255};`
10	`constexpr SmartColor P2TurnBorder = {200, 0, 255, 255};`
11	`constexpr SmartColor P2Watermark = {60, 0, 255, 255};`

CHƯƠNG 4. HỆ THỐNG ĐỒ HỌA VÀ UI/UX

4.1. Tổng quan pipeline đồ họa

Chương này trình bày toàn bộ kiến trúc đồ họa và UI/UX của trò chơi, dựa trên cấu trúc triển khai và các mô-đun hệ thống. Hệ thống được xây dựng trên Win32 API kết hợp GDI/GDI+ theo hướng "raster + procedural", nhấn mạnh tốc độ, độ mượt và cảm giác thị giác retro [1]. Phần này không chỉ mô tả cách vẽ mà còn lý giải các tối ưu quan trọng (double buffering, cache bitmap, palette động) và cách các màn hình thực tế ghép các thành phần đó thành trải nghiệm người dùng nhất quán.

Về mặt kỹ thuật, vòng đời đồ họa bắt đầu từ `InitGraphics` (khởi động GDI+), kế tiếp là
`GlobalFont::Initialize` để nạp font VT323, sau đó tạo `DoubleBuffer` để vẽ toàn bộ frame lên HDC bộ nhớ. Khi thoát game, `GlobalFont::Cleanup` gỡ font đã nạp và `ShutdownGraphics` giải phóng token GDI+ nhằm tránh rò rỉ tài nguyên hệ thống.

Pipeline hiển thị chia làm hai lớp:

- **Lớp khởi tạo và tài nguyên toàn cục**: GDI+ được khởi tạo, font riêng (VT323) được nạp vào môi trường chạy, sau đó tạo các font theo tỉ lệ UI hiện tại.
- **Lớp vẽ theo khung hình**: các màn hình gọi renderer/utility để vẽ nền, panel, text, pixel art và hiệu ứng động. GDI được dùng cho các primitive đơn giản (text, line) trong khi GDI+ chịu trách nhiệm phần hiệu ứng và bitmap.

Các tệp cốt lõi trong RenderAPI gồm: `Renderer.*`, `UIScaler.*`, `Colours.*`, `UIComponents.*`. Những tệp này cung cấp API cấp cao để các mô-đun màn hình (`ScreenModule`) chỉ cần gọi hàm vẽ theo ngữ cảnh.

Luồng gọi chính vì vậy rất rõ ràng: mô-đun màn hình (`ScreenModule`) tính bố cục (theo `UIScaler`), gọi các hàm vẽ nền/đối tượng (trong `UIComponents`), sau cùng "đổ" nội dung ra màn hình thông qua buffer kép. Việc chia lớp giúp tránh trộn lẫn logic UI với xử lý tài nguyên mức thấp.

4.2. Renderer và tối ưu hóa hiển thị

4.2.1. Khởi tạo GDI+ và quản lý font

GDI+ được bật/tắt có kiểm soát, tránh rò rỉ tài nguyên. Font VT323 được load từ thư mục Asset và tạo 4 biến thể: Default, Bold, Title, Note. Điểm quan trọng là kích thước font phụ thuộc vào `UIScaler::S()`, đảm bảo phóng to/thu nhỏ đồng bộ với tỉ lệ màn hình.

Cụ thể, `AddFontResourceExW` dùng `FR_PRIVATE` để font chỉ có hiệu lực trong tiến trình hiện tại, không ảnh hưởng hệ thống. Khi resize, `GlobalFont::RebuildFonts` xóa font cũ trước khi tạo mới, tránh leak GDI objects. Đây là phần quan trọng vì GDI giới hạn số handle, nếu không dọn đúng sẽ dẫn tới lỗi hiển thị sau thời gian dài.

Mã nguồn 14: Mã nguồn Khởi tạo font và tái tạo theo tỉ lệ UI

STT	Mã nguồn
1	`void Initialize()`
2	`{`
3	`  // Nạp file Font từ thư mục Asset vào môi trường chạy tạm thời của Windows`
4	`  AddFontResourceExW(L"Asset/font/VT323/VT323-Regular.ttf", FR_PRIVATE, 0);`
5	
6	`  RebuildFonts();`
7	`}`
8	
9	`void RebuildFonts()`
10	`{`
11	`  if (Default) { DeleteObject(Default); }`
12	`  if (Bold)    { DeleteObject(Bold); }`
13	`  if (Title)   { DeleteObject(Title); }`
14	`  if (Note)    { DeleteObject(Note); }`
15	
16	`  Default = CreateFontW(UIScaler::S(36), 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE,`
17	`  DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,`

18	NONANTIALIASED_QUALITY, DEFAULT_PITCH | FF_DONTCARE, L"VT323");
19	Bold = CreateFontW(UIScaler::S(42), 0, 0, 0, FW_BOLD, FALSE, FALSE, FALSE,
20	DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
21	NONANTIALIASED_QUALITY, DEFAULT_PITCH | FF_DONTCARE, L"VT323");
22	Title = CreateFontW(UIScaler::S(64), 0, 0, 0, FW_HEAVY, FALSE, FALSE, FALSE,
23	DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
24	NONANTIALIASED_QUALITY, DEFAULT_PITCH | FF_DONTCARE, L"VT323");
25	Note = CreateFontW(UIScaler::S(28), 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE,
26	DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
27	NONANTIALIASED_QUALITY, DEFAULT_PITCH | FF_DONTCARE, L"VT323");
28	}

Việc dùng `NONANTIALIASED_QUALITY` giúp font giữ độ sắc và phù hợp phong cách pixel. Khi thay đổi kích thước cửa sổ, `UIScaler::Update()` sẽ cập nhật tỉ lệ, sau đó font được rebuild để đảm bảo tính nhất quán kích thước.

4.2.2. Double Buffering chống nhấp nháy

Hiện tượng nhấp nháy (flicker) rất dễ xảy ra khi vẽ nhiều lớp. Để xử lý, renderer sử dụng bộ đệm kép: tạo một HDC bộ nhớ, vẽ toàn bộ frame lên đó, sau cùng mới blit ra màn hình. Tối ưu quan trọng là đảm bảo kích thước buffer tối thiểu 1x1 để tránh lỗi khi cửa sổ bị thu nhỏ.

Trong `CreateBuffer`, việc gọi `CreateCompatibleDC` và `CreateCompatibleBitmap` theo kích thước hiện tại giúp đảm bảo bề mặt vẽ tương thích với HDC cửa sổ. Sau khi tạo, buffer được xóa nền trắng bằng `FillRect` để tránh "rác" hình ảnh trong khung đầu tiên.

Mã nguồn 15: Mã nguồn Tạo bộ đệm kép theo kích thước cửa sổ

STT	Mã nguồn

```
void CreateBuffer(HWND hwnd, HDC hdc, DoubleBuffer &buffer)
{
  RECT rect;
  GetClientRect(hwnd, &rect);
  int w = rect.right - rect.left;
  int h = rect.bottom - rect.top;

  if (w <= 0) { w = 1; }
  if (h <= 0) { h = 1; }
  buffer.width = w;
  buffer.height = h;

  buffer.hdcMem = CreateCompatibleDC(hdc);
  buffer.hBitmap = CreateCompatibleBitmap(hdc, w, h);
  buffer.hOldBitmap = (HBITMAP)SelectObject(buffer.hdcMem,
buffer.hBitmap);

  RECT fillRect = {0, 0, w, h};
  HBRUSH hBrush = CreateSolidBrush(RGB(255, 255, 255));
  FillRect(buffer.hdcMem, &fillRect, hBrush);
  DeleteObject(hBrush);
}
```

4.2.3. Đồng bộ resize và tái tạo back buffer

Khi kích thước cửa sổ thay đổi, hệ thống cập nhật tỉ lệ UI, rebuild font và tái tạo back buffer để đảm bảo vùng vẽ luôn đúng kích thước. Cơ chế này đồng bộ trong xử lý `WM_SIZE` để tránh tình trạng render vào buffer sai kích thước.

Việc `DeleteBuffer` luôn chọn lại `hOldBitmap` trước khi giải phóng nhằm tránh trạng thái HDC "treo" vào bitmap cũ. Đây là lỗi phổ biến trong GDI nếu không hoàn nguyên object trước khi `DeleteObject`.

Mã nguồn 16: Mã nguồn Cập nhật tỉ lệ và back buffer khi resize

STT	Mã nguồn
1	`case WM_SIZE:`
2	`{`
3	`  int w = LOWORD(lParam);`
4	`  int h = HIWORD(lParam);`
5	`  UIScaler::Update(w, h);`
6	`  GlobalFont::RebuildFonts();`
7	`  HDC hdc = GetDC(hWnd);`
8	`  DeleteBuffer(g_BackBuffer);`
9	`  CreateBuffer(hWnd, hdc, g_BackBuffer);`
10	`  ReleaseDC(hWnd, hdc);`
11	`  InvalidateRect(hWnd, NULL, FALSE);`
12	`  return DefWindowProc(hWnd, message, wParam, lParam);`
13	`}`

4.2.4. GDI+ nâng cao: Alpha Blending và Linear Gradient

GDI+ được dùng để tạo các hiệu ứng thị giác "mềm" mà GDI thuần khó làm được: nền kính mờ (glassmorphism), viền phát sáng, gradient nền banner. Ví dụ `DrawPixelBanner` sử dụng `LinearGradientBrush` cho nền, thêm viền sáng theo nhịp sin để tạo cảm giác động.

Hàm `DrawPixelBanner` còn hỗ trợ icon pixel art ở hai đầu banner. Khi không cung cấp icon, hệ thống vẽ bóng (football) mặc định; khi có đường dẫn icon, hàm tự tạo palette động dựa trên màu nhấn của màn hình để đảm bảo icon hòa hợp với theme hiện hành.

Mã nguồn 17: Mã nguồn Banner gradient và viền pulse

STT	Mã nguồn
1	`Gdiplus::LinearGradientBrush gradBrush(`
2	`  Gdiplus::Point(bannerX, bannerY),`
3	`  Gdiplus::Point(bannerX + bannerW, bannerY),`
4	`  Gdiplus::Color(230, 10, 15, 25),`
5	`  Gdiplus::Color(230, 30, 60, 90));`

```
g.FillRectangle(&gradBrush, bannerX, bannerY, bannerW, bannerH);

float pulse = 0.6f + sin(g_GlobalAnimTime * 6.0f) * 0.4f;
BYTE lineA = (BYTE)(180 + pulse * 75);
Gdiplus::Pen topLine(Gdiplus::Color(lineA, gr, gg, gb), 2.5f);
Gdiplus::Pen botLine(Gdiplus::Color((BYTE)(lineA * 0.6f), gr, gg,
gb), 1.5f);
g.DrawLine(&topLine, bannerX, bannerY, bannerX + bannerW, bannerY);
g.DrawLine(&botLine, bannerX, bannerY + bannerH, bannerX + bannerW,
bannerY + bannerH);
```

Nhờ alpha blending, các lớp kính và glow chồng lên nền sân vận động vẫn giữ độ rõ của nền phía sau, qua đó tạo chiều sâu cho UI.

4.2.5. Bitmap Caching cho đối tượng phức tạp

Các đối tượng vẽ bằng pixel art thường nặng nếu vẽ từng pixel mỗi frame. Hệ thống sử dụng nhiều cache độc lập:

- `g_RawModelCache`: cache dữ liệu ma trận số đọc từ file txt.
- `g_ModelCache`: cache bitmap đã dựng từ ma trận + palette.
- `g_AvatarCache`, `g_ActionCache`, `g_FootballCache`, `g_TrophyCache`, `g_ClockCache`: cache theo loại đối tượng.
- `g_BrushCache`: cache các brush GDI+ theo màu ARGB để tránh new/delete liên tục.

Bảng 4.1: Tổng hợp cache đồ họa và khóa nhận diện

Cache	Dữ liệu	Khóa (key)	Ghi chú
`g_RawModelCache`	PixelModel thô	Đường dẫn file txt	Tránh đọc file lặp lại
`g_ModelCache`	Bitmap từ model + palette	Hash (model ptr, size, palette)	Dùng lại bitmap đã dựng
`g_AvatarCache`	Avatar bitmap	Hash (avatarType, size)	Fallback về avatar 0 nếu thiếu
`g_ActionCache`	Action frame bitmap	Hash (avatarType, action, frame, flipH, size)	Cache theo từng frame hoạt họa
`g_FootballCache`	Bitmap bóng	Size (int)	Vẽ pulse theo thời gian
`g_TrophyCache`	Bitmap cúp	Size (int)	Dựng theo palette cúp
`g_ClockCache`	Bitmap đồng hồ	Hash (size, color)	Màu phụ thuộc chủ đề
`g_BrushCache`	GDI+ SolidBrush	ARGB value	Tránh new/delete liên tục

Khi một model hoặc action đã được dựng, các frame sau chỉ cần vẽ lại bitmap đã cache bằng `DrawImage`, giảm đáng kể chi phí CPU.

Cơ chế cache dựa trên khóa hash (model pointer + size + palette) giúp hệ thống không cần ghép chuỗi tên file trong vòng lặp nóng. Điều này vừa tăng hiệu suất, vừa giảm phân mảnh bộ nhớ do tạo/huỷ string liên tục.

4.3. Pixel Art Engine

4.3.1. Dựng hình từ ma trận dữ liệu số

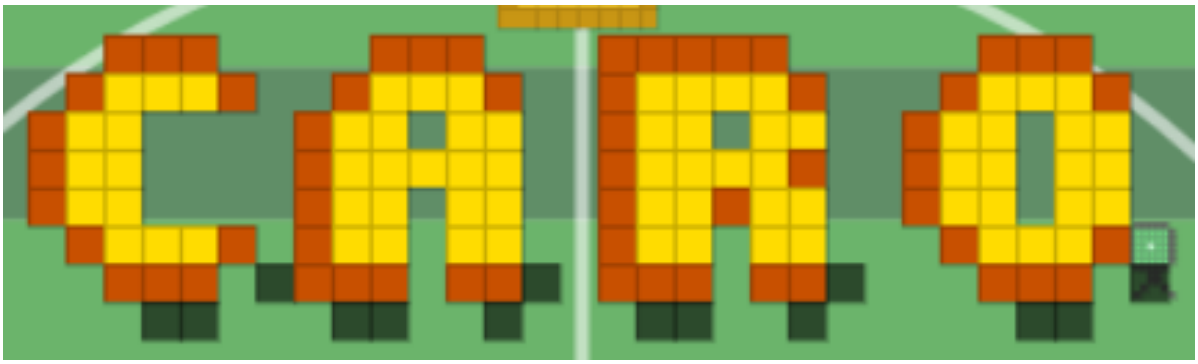
Pixel art được lưu dưới dạng file txt chứa kích thước và ma trận số (mỗi số là một mã màu). Ví dụ một file model được đọc bằng `LoadPixelModel` và dựng ảnh trong `DrawPixelModel`.

`LoadPixelModel` có cache cấp dữ liệu thô (`g_RawModelCache`) để tránh đọc file lặp lại. Sau khi đọc, `model.isLoaded` được bật nhằm cho phép các bước dựng ảnh phía sau kiểm tra nhanh tính hợp lệ.

Bảng 4.2: Kích thước các mô hình pixel art tiêu biểu

File	Kích thước	Sử dụng
`Asset/models/bg/title_caro.txt`	31×8	Tiêu đề banner Caro
`Asset/models/bg/badge.txt`	12×14	Huy hiệu thắng
`Asset/models/bg/trophy.txt`	12×12	Cúp tổng kết
`Asset/models/bg/clock.txt`	8×8	Đồng hồ đếm ngược
`Asset/models/bg/cloud.txt`	16×6	Mây nền sân
`Asset/models/bg/football.txt`	8×8	Bóng khán đài
`Asset/models/avt_00/avt.txt`	48×48	Avatar cầu thủ

Hình 4.1: Hình minh họa dữ liệu pixel model (ví dụ title_caro)



Ví dụ rút gọn nội dung file `Asset/models/bg/title_caro.txt` (dòng đầu là kích thước, các dòng sau là ma trận mã màu):

Mã nguồn 18: Ví dụ file pixel art `title_caro.txt`

STT	Mã nguồn
1	31 8
2	0 0 0 1 1 1 0 0 0 0 1 1 1 0 0 0 1 1 1 1 1 0 0 0 0 0 1 1 1 0 0
3	0 0 1 2 2 2 1 0 0 1 2 2 2 1 0 0 1 2 2 2 2 1 0 0 0 1 2 2 2 1 0
4	0 1 2 2 0 0 0 0 1 2 2 0 2 2 0 0 1 2 2 0 2 2 0 0 1 2 2 0 2 2 0
5	0 1 2 2 0 0 0 0 1 2 2 2 2 2 0 0 1 2 2 2 2 1 0 0 1 2 2 0 2 2 0
6	0 1 2 2 0 0 0 0 1 2 2 0 2 2 0 0 1 2 2 1 2 2 0 0 1 2 2 0 2 2 0
7	0 0 1 2 2 2 1 0 1 2 2 0 2 2 0 0 1 2 2 0 2 2 0 0 0 1 2 2 2 1 0
8	0 0 0 1 1 1 0 3 1 1 1 0 1 1 3 0 1 1 1 0 1 1 3 0 0 0 1 1 1 0 3
9	0 0 0 0 3 3 0 0 0 3 3 0 0 3 0 0 0 3 3 0 0 3 0 0 0 0 0 3 3 0 0

Mã nguồn 19: Mã nguồn Load dữ liệu pixel từ file txt

STT	Mã nguồn
1	`PixelModel LoadPixelModel(const std::string &filePath)`
2	`{`
3	`  auto it = g_RawModelCache.find(filePath);`
4	`  if (it != g_RawModelCache.end()) return it->second;`
5	
6	`  PixelModel model;`
7	`  std::ifstream file(filePath);`
8	`  if (!file.is_open()) { return model; }`
9	
10	`  std::string line;`
11	`  if (std::getline(file, line)) {`
12	`    std::stringstream ss(line);`
13	`    ss >> model.width >> model.height;`
14	`  }`
15	
16	`  model.data.resize(model.height, std::vector<int>(model.width, 0));`
17	`  for (int r = 0; r < model.height; ++r) {`
18	`    if (!std::getline(file, line)) break;`
19	`    std::stringstream ss(line);`
20	`    for (int c = 0; c < model.width; ++c) {`
21	`      int val = 0;`
22	`      if (ss >> val) model.data[r][c] = val;`
23	`    }`
24	`  }`
25	`  model.isLoaded = true;`
26	`  g_RawModelCache[filePath] = model;`
27	`  return model;`
28	`}`

Quá trình dựng bitmap từ ma trận được tối ưu bằng hashing key (không dùng string

concat trong vòng lặp nóng). Ngoài ra, mỗi pixel được vẽ kèm một đường lưới mờ để duy trì phong cách retro.

Trong `DrawPixelModel`, kích thước pixel `pSize` được suy ra từ `totalSize` và kích thước model, đảm bảo hình giữ đúng tỷ lệ. Khi vẽ, hệ thống đặt `InterpolationModeNearestNeighbor` và `PixelOffsetModeHalf` để tránh mờ biên, giữ đúng cảm giác pixel art.

Mã nguồn 20: Mã nguồn Dựng bitmap pixel art với cache và lưới

STT	Mã nguồn
1	`size_t key = (size_t)&model;`
2	`hash_combine(key, totalSize);`
3	`for (auto const &[id, col] : palette)`
4	`{`
5	`  hash_combine(key, id);`
6	`  hash_combine(key, col.GetValue());`
7	`}`
8	
9	`if (g_ModelCache.find(key) == g_ModelCache.end())`
10	`{`
11	`  int totalW = model.width * pSize;`
12	`  int totalH = model.height * pSize;`
13	`  Gdiplus::Bitmap *bmp = new Gdiplus::Bitmap(totalW + 2, totalH + 2, PixelFormat32bppARGB);`
14	`  Gdiplus::Graphics gBmp(bmp);`
15	`  gBmp.SetSmoothingMode(Gdiplus::SmoothingModeNone);`
16	
17	`  Gdiplus::Pen gridPen(Gdiplus::Color(60, 0, 0, 0), 1.0f);`
18	`  for (int r = 0; r < model.height; ++r)`
19	`  {`
20	`    for (int c = 0; c < model.width; ++c)`
21	`    {`
22	`      int val = model.data[r][c];`
23	`      if (val != 0)`

```
      {
        auto it = palette.find(val);
        if (it != palette.end())
        {
          Gdiplus::SolidBrush *b = GetCachedBrush(it->second);
          gBmp.FillRectangle(b, c * pSize, r * pSize, pSize, pSize);
          gBmp.DrawRectangle(&gridPen, c * pSize, r * pSize, pSize, pSize);
        }
      }
    }
  }
  g_ModelCache[key] = bmp;
}
```

Hệ thống này cho phép các biểu tượng như bóng, cúp, đồng hồ, banner, mây, bóng bay được tạo hoàn toàn bằng dữ liệu (không cần PNG). Cách tiếp cận này giảm phụ thuộc vào asset ảnh, đồng thời cho phép thay đổi palette theo chủ đề một cách linh hoạt.

4.3.2. Palette và ngữ nghĩa màu

Tệp `Colours.h` định nghĩa hai nhóm màu:

- **Palette**: màu nguyên thủy dùng chung (đen, trắng, xám, đỏ, xanh, v.v.).
- **Theme**: màu ngữ nghĩa (shadow, glass panel, pitch, viền panel, màu P1/P2, highlight, v.v.).

Điểm nổi bật là bảng màu avatar có mã số riêng cho từng cầu thủ (Ronaldo, Messi, Neymar). Mỗi mã số (2,3,4,6,8,15,16...) được ánh xạ sang màu cụ thể để giữ nhận diện nhân vật.

Lớp màu được tách thành `Palette` (màu nguyên thủy) và `Theme` (màu ngữ nghĩa). Cấu trúc `SmartColor` là POD đơn giản để dễ copy/assign, còn chuyển đổi sang GDI/GDI+ được thực hiện qua `ToCOLORREF` và `ToGdiColor`. Cách này giúp toàn bộ pipeline màu nhất quán giữa GDI và GDI+.

Mã nguồn 21: Mã nguồn Bảng màu nguyên thủy và màu ngữ nghĩa

STT	Mã nguồn
1	`namespace Palette`
2	`{`
3	`  constexpr SmartColor Transparent = {0, 0, 0, 0};`
4	`  constexpr SmartColor White = {255, 255, 255, 255};`
5	`  constexpr SmartColor GrayDarkest = {255, 33, 33, 33};`
6	`}`
7	
8	`namespace Theme`
9	`{`
10	`  constexpr SmartColor GlassWhite = {220, 255, 255, 255};`
11	`  constexpr SmartColor PanelBlueBorder = {180, 0, 150, 255};`
12	`  constexpr SmartColor TitleFill = {255, 255, 220, 0};`
13	`  constexpr SmartColor P1TurnPulse = {30, 255, 150, 0};`
14	`}`

Mã nguồn 22: Mã nguồn Tra mã màu avatar theo loại nhân vật

STT	Mã nguồn
1	`static SmartColor LookupAvatarColor(int type, int code)`
2	`{`
3	`  switch (type)`
4	`  {`
5	`  case 1: // Messi`
6	`    switch (code)`
7	`    { case 2: return Theme::MES_SkinL; case 6: return Theme::MES_HairD; }`
8	`  case 2: // Neymar`
9	`    switch (code)`
10	`    { case 2: return Theme::NEY_SkinL; case 13: return Theme::NEY_Shirt; }`

```
  default: // Ronaldo
    switch (code)
    { case 2: return Theme::CR7_SkinL; case 13: return Theme::TitleFill; }
  }
  return Theme::CR7_SkinM;
}
```

4.3.3. Action Animation System

Hệ thống animation dùng thư mục hành động theo cấu trúc:

```
Asset/models/avt_00/idle/f_0.txt ...
Asset/models/avt_00/run/f_0.txt ...
Asset/models/avt_00/sad/f_0.txt ...
Asset/models/avt_00/win/f_0.txt ...
```

Mỗi frame là một file txt. Hàm `DrawPixelAction` cập nhật frame theo thời gian, tạo cache key theo (avatar, action, frame, flip, size), fallback về frame 0 hoặc avatar mặc định khi thiếu dữ liệu.

Cụ thể, hệ thống dùng `GetTickCount64` để kiểm soát thời gian frame, đảm bảo animation ổn định kể cả khi FPS biến động. Khi thiếu frame, hàm tự reset về `f_0.txt` và cập nhật cache key mới, giảm rủi ro crash do file không tồn tại. Việc hỗ trợ `flipH` giúp tái sử dụng sprite cho hai hướng di chuyển mà không cần thêm asset.

Hình 4.2: Hình minh họa chuỗi frame action run của avt_00

Mã nguồn 23: Mã nguồn Cập nhật frame và dựng animation

STT	Mã nguồn
1	`ULONGLONG now = GetTickCount64();`
2	`if (now - state.lastFrameTime > (ULONGLONG)state.animationSpeed)`
3	`{`
4	`  state.currentFrame++;`
5	`  state.lastFrameTime = now;`
6	`}`
7	
8	`size_t cacheKey = state.avatarType;`
9	`hash_combine(cacheKey, state.currentAction);`
10	`hash_combine(cacheKey, state.currentFrame);`
11	`hash_combine(cacheKey, state.flipH);`
12	`hash_combine(cacheKey, size);`

```
13
14  std::string path = "Asset/models/avt_0" +
    std::to_string(state.avatarType) +
15              "/" + state.currentAction + "/f_"
    + std::to_string(state.currentFrame) + ".txt";
16  PixelModel model = LoadPixelModel(path);
17
18  if (!model.isLoaded && state.currentFrame > 0)
19  {
20    state.currentFrame = 0;
21    path = "Asset/models/avt_0" + std::to_string(state.avatarType) +
22              "/" + state.currentAction + "/f_0.txt";
23    model = LoadPixelModel(path);
24  }
```

Khi dựng frame, hệ thống còn vẽ bóng dưới chân và cho phép lật ngang (flipH) để tái sử dụng sprite cho hai hướng di chuyển. Cách tiếp cận này tiết kiệm asset và tăng tính động trong gameplay.

4.4. Thiết kế UI thích ứng (Responsive UI)

4.4.1. UIScaler và cơ chế tỉ lệ hóa

Mọi kích thước UI đều dựa trên baseline 1920x1080. Khi màn hình thay đổi, `UIScaler::Update` tính lại scaleX/scaleY/scaleMin. Các hàm `S()`, `SX()`, `SY()` được dùng xuyên suốt để giữ bố cục cân đối.

Mã nguồn 24: Mã nguồn Cập nhật tỉ lệ theo kích thước cửa sổ

STT	Mã nguồn
1	`void Update(int width, int height)`
2	`{`
3	`  if (width <= 0) { width = 1; }`
4	`  if (height <= 0) { height = 1; }`
5	

```
  scaleX = width / BASE_WIDTH;
  scaleY = height / BASE_HEIGHT;
  scaleMin = (std::min)(scaleX, scaleY);
}

int S(int val) { return (int)(val * scaleMin); }
int SX(int val) { return (int)(val * scaleX); }
int SY(int val) { return (int)(val * scaleY); }
```

Nhờ `scaleMin`, UI giữ tỉ lệ đồng đều, tránh méo khi màn hình quá rộng hoặc quá cao. Các panel lớn dùng `SX/SY` để tối ưu không gian hiển thị.

`UIScaler::Update` bảo vệ trường hợp `width/height` bằng 0 (cửa sổ bị thu nhỏ), từ đó tránh chia cho 0 và tránh tạo kích thước âm trong các phép nhân sau. Đây là chi tiết nhỏ nhưng giúp hệ thống không bị lỗi khi người dùng resize liên tục.

4.4.2. Hiệu ứng thị giác và nền động

Nền sân vận động là một ví dụ điển hình của procedural art. Hàm `DrawProceduralStadium` tạo nhiều lớp:

- Nền cỏ sọc ngang, vạch sân và vòng tròn giữa sân.
- Hiệu ứng "camera flash" (chỉ bật ở menu) để tạo không khí khán đài.
- Vệt gió, mây trôi, bóng bay bay lên.

Lớp tĩnh (pitch + line) được cache thành bitmap để chỉ vẽ lại khi kích thước màn hình thay đổi.

Trong `DrawProceduralStadium`, phần pitch cache được xây một lần và chỉ rebuild khi `screenWidth/screenHeight` thay đổi. Các hiệu ứng động (flash, wind, cloud, balloon) được vẽ sau cùng để giảm chi phí và tránh redraw nền quá nhiều. Cách tách tầng giúp giữ FPS ổn định ngay cả khi nhiều hiệu ứng cùng lúc.

Mã nguồn 25: Mã nguồn Cache nền sân và hiệu ứng flash

STT	Mã nguồn
1	`static Gdiplus::Bitmap *pitchCache = nullptr;`
2	`static int cachedW = 0;`
3	`static int cachedH = 0;`

```
4
5  if (!pitchCache || cachedW != screenWidth || cachedH !=
   screenHeight)
6  {
7    if (pitchCache) { delete pitchCache; pitchCache = nullptr; }
8    pitchCache = new Gdiplus::Bitmap(screenWidth, screenHeight,
   PixelFormat32bppARGB);
9    Gdiplus::Graphics gP(pitchCache);
10   gP.SetSmoothingMode(Gdiplus::SmoothingModeAntiAlias);
11   // ... vẽ pitch, line, circle ...
12   cachedW = screenWidth;
13   cachedH = screenHeight;
14 }
15
16 g.DrawImage(pitchCache, 0, 0);
17
18 if (showFlashes)
19 {
20   const int flashCount = 15;
21   for (int i = 0; i < flashCount; i++)
22   {
23     int fx = (i * 918273) % screenWidth;
24     int fy = (i * 374621) % (screenHeight / 3);
25     float phase = (i * 137) % 314 / 50.0f;
26     float rawPulse = sin(g_GlobalAnimTime * 15.0f + phase);
27     if (rawPulse > 0.92f)
28     {
29       int alpha = (int)((rawPulse - 0.92f) * 12.5f * 255);
30       alpha = max(0, min(255, alpha));
31       Gdiplus::SolidBrush flashBrush(Gdiplus::Color((BYTE)alpha,
   255, 255, 255));
```

32	`        // Vẽ dấu cộng flash (2 hình chữ nhật) tại (fx, fy)`
33	`      }`
34	`    }`
35	`}`

Nền động này giúp UI luôn có chuyển động nhẹ, tránh cảm giác tĩnh trong thời gian chờ.

4.5. Vẽ bàn cờ và các hiệu ứng gameplay

Trong gameplay, bàn cờ là lớp quan trọng nhất. Hàm `DrawGameBoard` chia làm 3 giai đoạn:

- Vẽ lưới bằng GDI (nhanh nhất cho line).
- Vẽ highlight, hiệu ứng thắng, cursor bằng GDI+ (alpha + glow).
- Vẽ quân cờ (X/O) bằng text GDI sau cùng để không bị can thiệp bởi GDI+.

Cách tách lớp trên giúp đường kẻ lưới sắc nét, đồng thời hiệu ứng dùng alpha và glow không làm mờ chữ/đường kẻ.

Trong `DrawGameBoard`, phần GDI+ được xử lý trước (highlight thắng, ô vừa đi, cursor neon) rồi mới vẽ quân cờ bằng GDI để tránh hiện tượng text bị alpha blend. Điều này cũng giải thích vì sao hệ thống tách riêng ”GIAI ĐOẠN 1” và ”GIAI ĐOẠN 2” trong mã.

Mã nguồn 26: Mã nguồn Vẽ lưới bàn cờ bằng GDI trong DrawGameBoard

STT	Mã nguồn
1	`void DrawGameBoard(Gdiplus::Graphics &g, HDC hdc, const PlayState *state, int cellSize, int offsetX, int offsetY)`
2	`{`
3	`  int size = state->boardSize;`
4	`  int boardLength = size * cellSize;`
5	
6	`  HPEN hPen = CreatePen(PS_SOLID, max(1, UIScaler::S(2)), ToCOLORREF(Palette::White));`
7	`  HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);`

```
  for (int i = 0; i <= size; ++i)
  {
    int currX = offsetX + i * cellSize;
    int currY = offsetY + i * cellSize;
    MoveToEx(hdc, offsetX, currY, NULL);
    LineTo(hdc, offsetX + boardLength, currY);
    MoveToEx(hdc, currX, offsetY, NULL);
    LineTo(hdc, currX, offsetY + boardLength);
  }
  SelectObject(hdc, hOldPen);
  DeleteObject(hPen);
}
```

Mã nguồn 27: Mã nguồn Highlight thắng và con trỏ neon

STT	Mã nguồn
1	`if (!state->winningCells.empty())`
2	`{`
3	`  float wPulse = 0.5f + sin(g_GlobalAnimTime * 10.0f) * 0.5f;`
4	`  int wAlpha = (int)(100 + wPulse * 155);`
5	`  Gdiplus::SolidBrush *winBrush = GetCachedBrush(ToGdiColor(WithAlpha(Theme::WinCellFill, (BYTE)wAlpha)));`
6	`  float pWidth = 1.5f + wPulse * 2.0f;`
7	`  Gdiplus::Pen winPen(ToGdiColor(Theme::WinCellBorder), pWidth);`
8	
9	`  for (const auto &wCell : state->winningCells)`
10	`  {`
11	`    int drawX = offsetX + wCell.second * cellSize;`
12	`    int drawY = offsetY + wCell.first * cellSize;`
13	`    g.FillRectangle(winBrush, drawX + 1, drawY + 1, cellSize - 2, cellSize - 2);`

```
    g.DrawRectangle(&winPen, drawX + 2, drawY + 2, cellSize - 4, cellSize - 4);
  }
}

if (state->status == MATCH_PLAYING)
{
  int cursorX = offsetX + state->cursorCol * cellSize;
  int cursorY = offsetY + state->cursorRow * cellSize;
  float pulse = 0.5f + sin(g_GlobalAnimTime * 8.0f) * 0.5f;
  Gdiplus::Color cursorColor = state->isP1Turn ? ToGdiColor(Palette::OrangeNormal) : ToGdiColor(Palette::CyanNormal);
  // ... glow + neon corners ...
}
```

Nhờ sự tách lớp như trên, gameplay vừa rõ ràng (grid sắc nét), vừa có hiệu ứng nhấn mạnh (highlight thắng, con trỏ neon), qua đó hỗ trợ người chơi theo dõi trạng thái.

Ngoài ra, logic toạ độ được kiểm tra lại trong mã: `row` ánh xạ Y và `col` ánh xạ X. Việc sửa này tránh lỗi highlight lệch vị trí khi tính toán nhầm trục.

4.6. Case study UI/UX từng màn hình

Phần này mô tả cách các mô-đun UI được ghép thành trải nghiệm hoàn chỉnh ở từng màn hình.

4.6.1. MenuScreen

Menu là màn hình mở đầu nên sử dụng nhiều hiệu ứng động:

- Nền sân vận động + flash khán đài (chỉ bật ở menu).
- Cúp pixel lắc nhẹ theo sin, title “CARO” pixel art.
- Các mục menu được highlight bằng màu nhấp nháy và icon bóng hai bên.

Menu sử dụng `DrawPixelTrophy`, `DrawPixelModel` (title), và `DrawPixelFootball` nhằm tạo điểm nhấn thị giác.

Các hiệu ứng dao động (cúp lắc, pulse màu) đều dựa trên `g_GlobalAnimTime`, giúp đồng bộ chuyển động giữa các thành phần và tạo cảm giác ”một nhịp” chung cho UI.

Bố cục được canh giữa toàn màn hình: phần tiêu đề đặt ở 1/4 chiều cao, danh sách menu bắt đầu từ nửa màn hình và giãn cách theo `UIScaler::SY`. Mục đang chọn được nhấn bằng màu cyan biến thiên theo `g_GlobalAnimTime` và cặp bóng hai bên để tạo hướng nhìn.

Hình 4.3: Hình minh họa MenuScreen

Mã nguồn 28: Mã nguồn RenderMenuScreen và bố cục tiêu đề

STT	Mã nguồn
1	`void RenderMenuScreen(HDC hdc, int selectedOption, int screenWidth, int screenHeight)`
2	`{`
3	`  Gdiplus::Graphics g(hdc);`
4	`  DrawProceduralStadium(g, screenWidth, screenHeight, true);`
5	`  Gdiplus::SolidBrush lightGlassBrush(Gdiplus::Color(80, 255, 255, 255));`
6	`  g.FillRectangle(&lightGlassBrush, 0, 0, screenWidth, screenHeight);`
7	
8	`  int cupYOffset = UIScaler::SY((int)(sin(g_GlobalAnimTime * 2.5f) * 10.0f));`
9	`  DrawPixelTrophy(g, screenWidth / 2, screenHeight / 4 - UIScaler::SY(85) + cupYOffset, UIScaler::S(100));`
10	

```
  static PixelModel titleModel;
  static std::map<int, Gdiplus::Color> titlePalette;
  if (!titleModel.isLoaded) {
    titleModel = LoadPixelModel("Asset/models/bg/title_caro.txt");
    titlePalette[1] = ToGdiColor(Theme::TitleBorder);
    titlePalette[2] = ToGdiColor(Theme::TitleFill);
    titlePalette[3] = ToGdiColor(Theme::TitleShadow); }
  DrawPixelModel(g, titleModel, screenWidth / 2, screenHeight / 4 +
UIScaler::SY(20), UIScaler::S(500), titlePalette);
}
```

4.6.2. PlayScreen

PlayScreen tập trung vào tính rõ ràng và phản hồi thị giác:

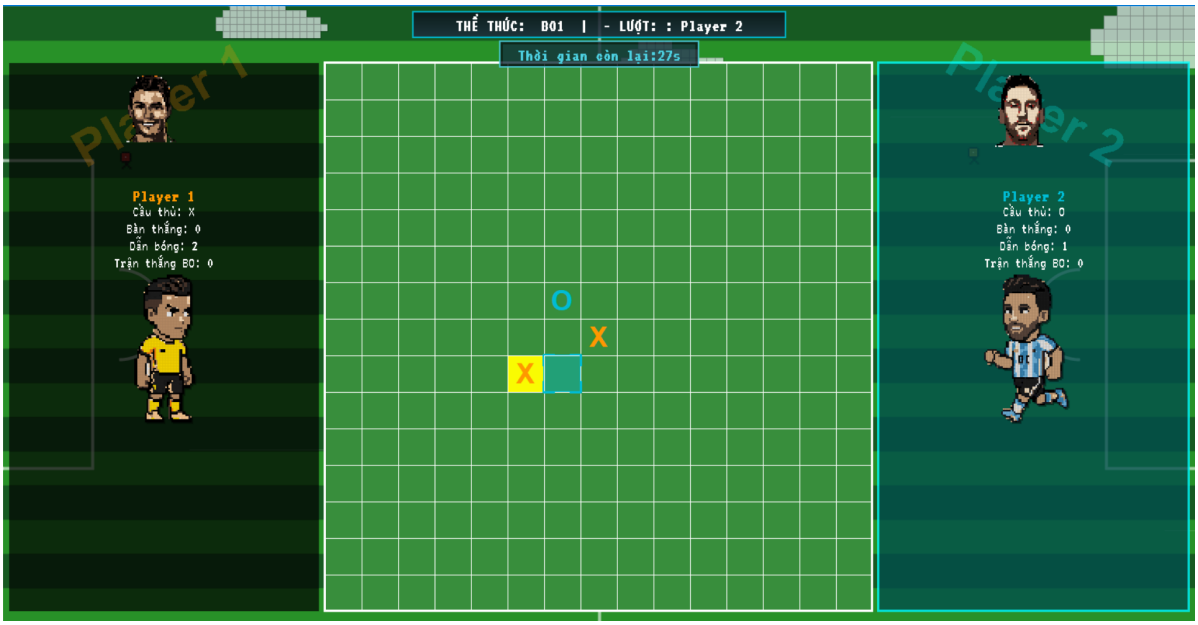
- Lưới cờ vẽ bằng GDI để giữ sắc nét.
- Cursor neon đổi màu theo lượt (cam cho P1, cyan cho P2).
- Highlight nước đi cuối và chuỗi thắng với hiệu ứng pulse.
- Animation cầu thủ sử dụng `DrawPixelAction` (idle/run/win/sad).

Phần logic cập nhật thời gian và trạng thái cũng làm nền cho UI (đếm giờ, thông báo thắng).

Trên PlayScreen, các chuỗi hiển thị như `play_format` và `play_turn` được lấy qua `GetText` nhằm đảm bảo đa ngôn ngữ. Việc dựng text score theo `state->targetScore` và `state->isP1Turn` giúp UI luôn đồng bộ với PlayState.

Bố cục gồm hai tab trái/phải cho thông tin người chơi, ở giữa là bàn cờ. Kích thước bàn cờ được tính động theo vùng trống (min giữa chiều rộng và chiều cao trừ biên), sau đó quy đổi thành `cellSize`. Trên cùng là scoreboard và đồng hồ đếm thời gian dạng "glass panel" có viền phát sáng theo lượt.

Hình 4.4: Hình minh họa PlayScreen



Mã nguồn 29: Mã nguồn RenderPlayScreen: tính kích thước và vẽ bàn cờ

STT	Mã nguồn
1	`void RenderPlayScreen(HDC hdc, const PlayState *state, int screenWidth, int screenHeight, const GameConfig *config)`
2	`{`
3	`  Gdiplus::Graphics g(hdc);`
4	`  DrawProceduralStadium(g, screenWidth, screenHeight);`
5	
6	`  int availableHeight = screenHeight - UIScaler::SY(120);`
7	`  int availableWidth = screenWidth - UIScaler::SX(300);`
8	`  int maxBoardSize = availableWidth < availableHeight ? availableWidth : availableHeight;`
9	`  int dynamicCellSize = maxBoardSize / state->boardSize;`
10	`  int boardPixelSize = state->boardSize * dynamicCellSize;`
11	`  int startX = (screenWidth - boardPixelSize) / 2;`
12	`  int startY = (screenHeight - boardPixelSize) / 2 + UIScaler::SY(40);`
13	
14	`  DrawGameBoard(g, hdc, state, dynamicCellSize, startX, startY);`

```
15
16    int scoreW = UIScaler::SX(600);
17    int scoreH = UIScaler::SY(45);
18    int scoreX = (screenWidth - scoreW) / 2;
19    int scoreY = UIScaler::SY(10);
20    std::wstring formatText = GetText("play_format") + L" BO" +
std::to_wstring(state->targetScore);
21    std::wstring turnText = GetText("play_turn") + L": " +
(state->isP1Turn ? state->p1.name : state->p2.name);
22    std::wstring fullScoreText = formatText + L" | " + turnText;
23    COLORREF turnColor = state->isP1Turn ?
ToCOLORREF(Palette::OrangeNormal) : ToCOLORREF(Palette::CyanNormal);
24    float pulseScore = 0.6f + sin(g_GlobalAnimTime * 6.0f) * 0.4f;
25    BYTE scoreAlpha = (BYTE)(130 + pulseScore * 125);
26    Gdiplus::LinearGradientBrush scoreBg(Gdiplus::Point(scoreX,
scoreY), Gdiplus::Point(scoreX, scoreY + scoreH),
Gdiplus::Color(210, 10, 15, 25), Gdiplus::Color(210, 30, 45, 65));
27    g.FillRectangle(&scoreBg, scoreX, scoreY, scoreW, scoreH);
28    Gdiplus::Pen scoreBorder(Gdiplus::Color(scoreAlpha,
GetRValue(turnColor), GetGValue(turnColor), GetBValue(turnColor)),
2.5f);
29    g.DrawRectangle(&scoreBorder, scoreX, scoreY, scoreW, scoreH);
30  }
```

4.6.3. Pause Game

Pause menu là lớp phủ trực tiếp lên gameplay, nhằm giữ ngữ cảnh trong khi vẫn giảm độ tập trung vào bàn cờ. Màn hình gồm hai khối chính: khung menu bên trái và cột hướng dẫn điều khiển bên phải. Các mục như Resume, BGM, Volume, Save, Exit được trình bày theo danh sách dọc, trong đó volume có thanh trượt trực quan.

Lớp overlay dùng `Theme::GlassWhite` để giảm độ tương phản nền, tạo hiệu ứng ”mờ” nhưng không làm mất hoàn toàn bối cảnh game. Điều này giúp người chơi vẫn nhận biết trạng thái ván đang tạm dừng.

Khi tạm dừng, hệ thống dùng nền kính trắng mờ để giảm nhiễu thị giác; banner “Pause” và icon còi (whistle) được vẽ bằng pixel art để giữ đồng nhất phong cách.

Hình 4.5: Hình minh họa Pause Game

Mã nguồn 30: Mã nguồn Lớp phủ Pause Menu trong RenderPlayScreen

STT	Mã nguồn
1	`if (state->status == MATCH_PAUSED)`
2	`{`
3	`  Gdiplus::SolidBrush shadowBrush2(Gdiplus::Color(100, 255, 255, 255));`
4	`  g.FillRectangle(&shadowBrush2, 0, 0, screenWidth, screenHeight);`
5	
6	`  int menuW = UIScaler::SX(580), menuH = UIScaler::SY(500);`
7	`  int guideW = UIScaler::SX(350);`
8	`  int gap = UIScaler::SX(25);`
9	`  int totalW = (g_CurrentSubMenu == SUB_MAIN) ? (menuW + gap + guideW) : menuW;`
10	`  int totalStartX = (screenWidth - totalW) / 2;`
11	
12	`  int menuX = totalStartX;`
13	`  int menuY = (screenHeight - menuH) / 2;`
14	`  Gdiplus::SolidBrush bgBrush(ToGdiColor(Theme::GlassWhite));`
15	`  g.FillRectangle(&bgBrush, menuX, menuY, menuW, menuH);`

```
  Gdiplus::Pen yellowBorder(ToGdiColor(Theme::PanelYellowBorder), 4.0f);
  g.DrawRectangle(&yellowBorder, menuX, menuY, menuW, menuH);
  DrawPixelBanner(g, hdc, GetText("pause_title").c_str(), menuX + menuW / 2, menuY + UIScaler::SY(40), menuW - UIScaler::SX(20), ToCOLORREF(Palette::White), RGB(255, 180, 0), "Asset/models/bg/whistle.txt");
}
```

4.6.4. MatchConfigScreen

Màn hình cấu hình trận đấu có hai trang, áp dụng UI điều hướng rõ ràng:

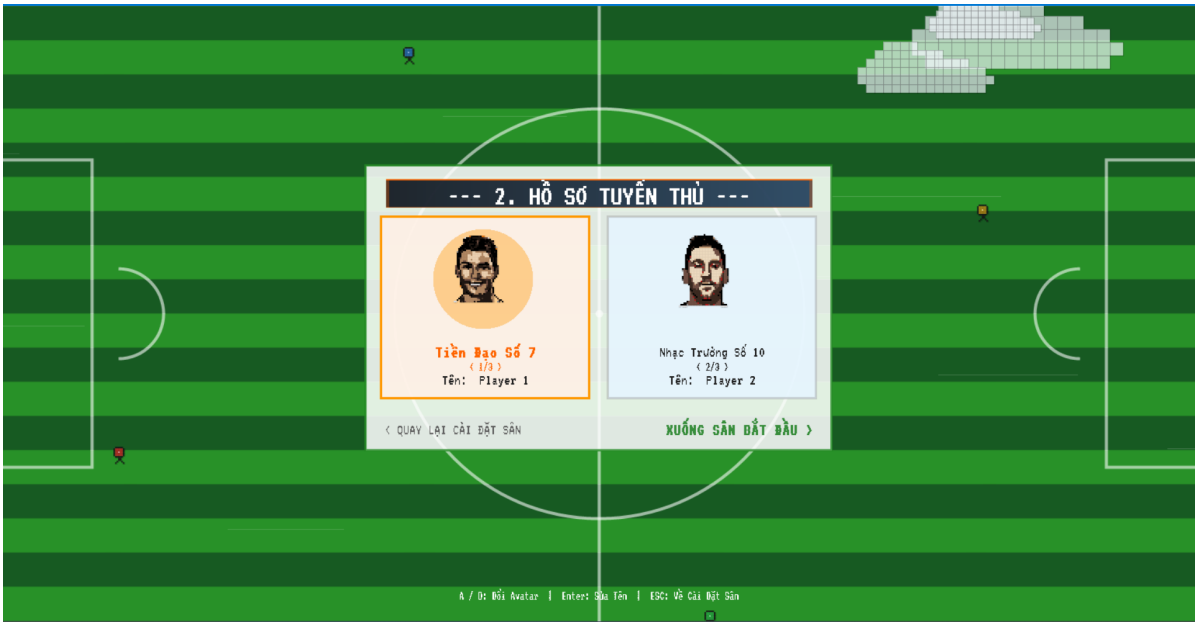
- Trang 1: cấu hình chế độ, PvP/PvE, độ khó, thời gian, số ván.
- Trang 2: cấu hình avatar và tên người chơi với input trực tiếp.

Màu sắc và font thay đổi theo lựa chọn để nhấn mạnh mục đang focus; mỗi giá trị được hiển thị trong khung riêng, tạo cảm giác "form" rõ ràng.

Layout chính là một panel kính giữa màn hình, chia hai cột (nhãn và giá trị). Ở trang 2, mỗi người chơi là một "card" riêng với avatar, tên, và hiệu ứng glow khi đang được chọn.

Hình 4.6: Hình minh họa MatchConfigScreen - trang 1

Hình 4.7: Hình minh họa MatchConfigScreen - trang 2



Mã nguồn 31: Mã nguồn RenderMatchConfigScreen: panel và banner

STT	Mã nguồn
1	`void RenderMatchConfigScreen(HDC hdc, int selectedOption, const PlayState *config, int screenWidth, int screenHeight)`
2	`{`
3	`  Gdiplus::Graphics g(hdc);`
4	`  DrawProceduralStadium(g, screenWidth, screenHeight);`
5	
6	`  Gdiplus::SolidBrush whitePanel(ToGdiColor(Theme::GlassWhite));`
7	`  int panelW = UIScaler::SX(750);`
8	`  int panelH = UIScaler::SY(500);`
9	`  int panelX = (screenWidth - panelW) / 2;`
10	`  int panelY = (screenHeight - panelH) / 2 - UIScaler::SY(10);`
11	`  g.FillRectangle(&whitePanel, panelX, panelY, panelW, panelH);`
12	`  Gdiplus::Pen panelPen(ToGdiColor(Theme::PanelGreenBorder), 3.0f);`
13	`  g.DrawRectangle(&panelPen, panelX, panelY, panelW, panelH);`
14	`  DrawPixelBanner(g, hdc, GetText("config_page1").c_str(), screenWidth / 2, panelY + UIScaler::SY(50), panelW - UIScaler::SX(40), ToCOLORREF(Palette::White), RGB(0, 100, 255), "Asset/models/bg/gears.txt");`

```
}
```

4.6.5. SettingScreen

SettingScreen là ví dụ điển hình của layout dạng bảng và thanh trượt:

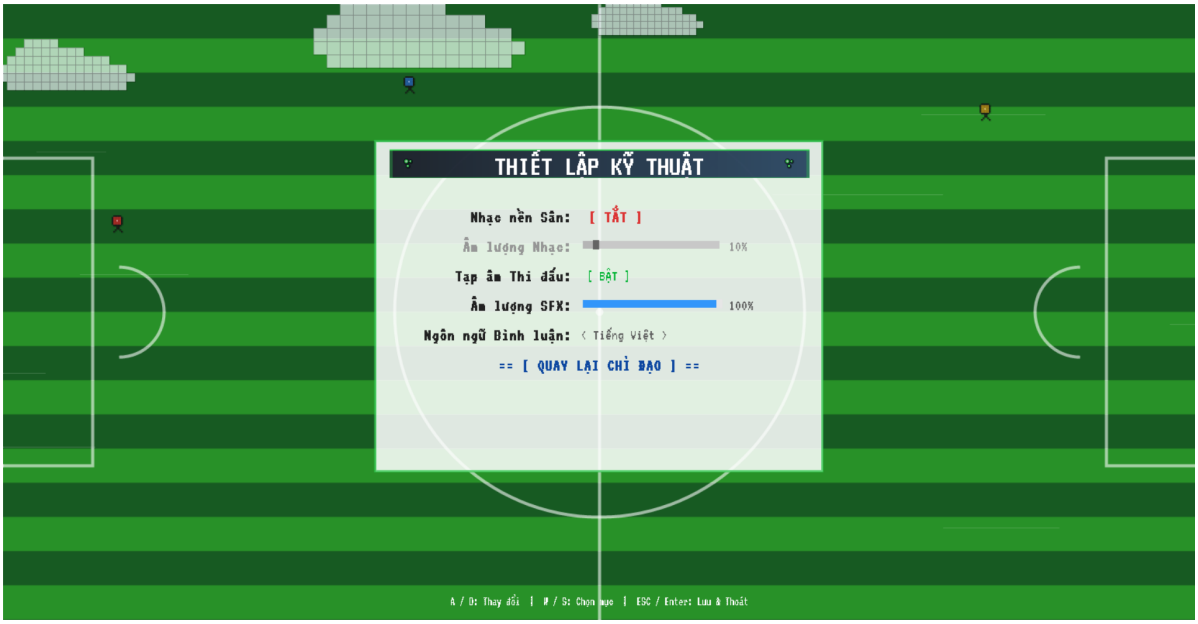
- Panel kính trắng, viền màu xanh.
- Banner tiêu đề có icon bánh răng.
- Thanh volume vẽ bằng GDI+, có bar track, fill, thumb.
- Mục bị disable sẽ đổi màu xám để giảm độ nổi bật.

Các mục bật/tắt được trình bày dưới dạng nhãn trạng thái, trong khi thanh trượt thể hiện mức âm lượng bằng tỉ lệ chiều dài.

Hệ thống chia panel thành hai cột: nhãn (trái) và giá trị (phải). Thanh trượt âm lượng dùng GDI+ để vẽ nền, phần fill và thumb, đồng thời đổi màu khi mục đang được chọn.

Các màu của thanh trượt lấy từ `Theme::BarTrack`, `Theme::BarFillSelected` và `Theme::BarFillNormal`, giúp nhất quán với bảng màu tổng thể. Khi tắt BGM/SFX, màu mục chuyển xám để biểu đạt trạng thái disable.

Hình 4.8: Hình minh họa SettingScreen



Mã nguồn 32: Mã nguồn RenderSettingScreen: panel và thanh volume

STT	Mã nguồn

```
void RenderSettingScreen(HDC hdc, const GameConfig *config, int selectedOption, int screenWidth, int screenHeight)
{
  Gdiplus::Graphics g(hdc);
  DrawProceduralStadium(g, screenWidth, screenHeight);

  int panelW = UIScaler::SX(720);
  int panelH = UIScaler::SY(580);
  int panelX = (screenWidth - panelW) / 2;
  int panelY = (screenHeight - panelH) / 2 - UIScaler::SY(10);
  Gdiplus::SolidBrush whitePanel(ToGdiColor(Theme::GlassWhite));
  g.FillRectangle(&whitePanel, panelX, panelY, panelW, panelH);
  DrawPixelBanner(g, hdc, GetText("setting_title").c_str(), screenWidth / 2, panelY + UIScaler::SY(40), panelW - UIScaler::SX(20), ToCOLORREF(Palette::White), RGB(50, 220, 80), "Asset/models/bg/gears.txt");
}
```

4.6.6. LoadGameScreen

Màn hình load game chia thành ba vùng: danh sách slot, thông tin slot, và hành động. Trạng thái tương tác được quản lý rõ ràng:

- `MODE_SELECT_SLOT`: chọn slot ở cột trái.
- `MODE_SELECT_ACTION`: chọn hành động (Load/Rename/Delete/Back).
- `MODE_EDIT_NAME`: nhập tên slot mới.

Điểm UX quan trọng: nếu slot trống sẽ hiển thị thông báo ngắn, tránh thao tác không cần thiết. Các panel được chia cột bằng layout tính theo `UIScaler` nên luôn cân đối.

Màn hình gồm ba cột: danh sách slot, thông tin chi tiết và cột hành động/đổi tên. Trạng thái chọn được tô nền bằng màu trong suốt để phân biệt giữa "đang chọn slot" và "đang chọn hành động".

Các màu nhấn cho slot lấy từ `Theme::SlotSelected` và `Theme::SlotNormal`, đảm bảo đồng bộ với palette chung. Đây là chi tiết nhỏ nhưng giúp UI không bị "lạc tông" giữa các màn hình.

Hình 4.9: Hình minh họa LoadGameScreen

Mã nguồn 33: Mã nguồn Che do tuong tac va RenderLoadGameScreen

STT	Mã nguồn
1	`enum LoadScreenMode`
2	`{ MODE_SELECT_SLOT, MODE_SELECT_ACTION, MODE_EDIT_NAME };`
3	
4	`void RenderLoadGameScreen(HDC hdc, int selectedOption, const std::wstring &statusMessage, int screenWidth, int screenHeight)`
5	`{`
6	`  Gdiplus::Graphics g(hdc);`
7	`  DrawProceduralStadium(g, screenWidth, screenHeight);`
8	
9	`  int panelW = UIScaler::SX(900);`
10	`  int panelH = UIScaler::SY(540);`
11	`  int panelX = (screenWidth - panelW) / 2;`
12	`  int panelY = (screenHeight - panelH) / 2;`
13	`  Gdiplus::SolidBrush bgBrush(ToGdiColor(Theme::GlassWhite));`
14	`  g.FillRectangle(&bgBrush, panelX, panelY, panelW, panelH);`

```
  DrawPixelBanner(g, hdc, GetText("save_title").c_str(), screenWidth / 2, panelY + UIScaler::SY(45), panelW - UIScaler::SX(40), ToCOLORREF(Palette::White), RGB(0, 150, 255), "Asset/models/bg/cassette.txt");
}
```

4.6.7. AboutScreen

AboutScreen thể hiện phong cách trình bày thông tin:

- Nền sân + lớp kính mờ.
- Banner tiêu đề theo ngôn ngữ.
- Bảng thành viên với header nền xanh, grid line rõ ràng.
- Dòng giảng viên hướng dẫn và công nghệ.

Màu sắc thiên sáng, thể hiện tông sáng nhẹ nhàng hơn so với các màn hình gameplay.

Bảng thành viên được vẽ như một bảng dữ liệu: có header nền xanh, chia cột, kẻ grid line bằng GDI+ để đảm bảo độ đều. Các dòng chữ căn giữa theo từng cột.

Hình 4.10: Hình minh họa AboutScreen

Mã nguồn 34: Mã nguồn RenderAboutScreen: banner va bang thanh vien

STT	Mã nguồn
1	`void RenderAboutScreen(HDC hdc, int screenWidth, int screenHeight)`

```
{
  Gdiplus::Graphics g(hdc);
  DrawProceduralStadium(g, screenWidth, screenHeight);
  DrawPixelBanner(g, hdc, GetText("about_title").c_str(), screenWidth / 2, UIScaler::SY(80), UIScaler::SX(500), ToCOLORREF(Palette::White), ToCOLORREF(Palette::CyanNormal));

  int tableW = UIScaler::SX(600);
  int tableH = UIScaler::SY(175);
  int tableX = (screenWidth - tableW) / 2;
  int tableY = UIScaler::SY(395);
  Gdiplus::Pen gridPen(Gdiplus::Color(100, 100, 100, 100), 1.0f);
  g.DrawLine(&gridPen, (INT)tableX, (INT)tableY, (INT)(tableX + tableW), (INT)tableY);
}
```

4.6.8. GuildScreen

GuildScreen chia thành 3 trang hướng dẫn, có chấm phân trang và mũi tên điều hướng:

- Trang 1: so sánh Caro và TicTacToe (hai cột).
- Trang 2: hướng dẫn di chuyển và thao tác.
- Trang 3: giải thích hệ thống AI, thời gian, mục tiêu, lưu trận.

Điểm nhấn UX là phân trang bằng dot + aura, giúp người dùng biết mình đang ở đâu trong chuỗi hướng dẫn.

Tiêu đề và màu banner thay đổi theo trang (cam, cyan, xanh lá), đồng thời hệ thống hiển thị mũi tên điều hướng ở hai bên để gợi ý thao tác.

Hình 4.11: Hình minh họa GuildScreen

Mã nguồn 35: Mã nguồn RenderGuildScreen: banner va phan trang

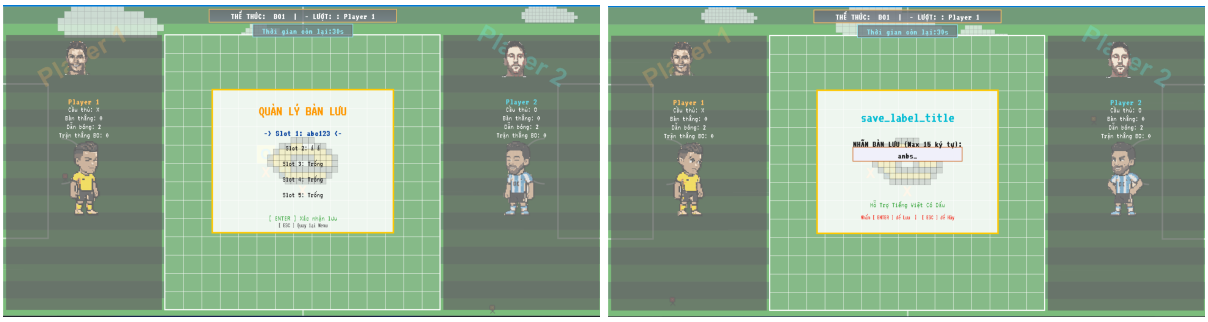
STT	Mã nguồn
1	`void RenderGuildScreen(HDC hdc, int screenWidth, int screenHeight, int currentPage)`
2	`{`
3	`  Gdiplus::Graphics g(hdc);`
4	`  DrawProceduralStadium(g, screenWidth, screenHeight);`
5	`  COLORREF bannerColors[] = { ToCOLORREF(Palette::OrangeNormal), ToCOLORREF(Palette::CyanNormal), ToCOLORREF(Palette::GreenNormal) };`
6	`  std::wstring titles[] = { GetText("guild_tab1"), GetText("guild_tab2"), GetText("guild_tab3") };`
7	`  DrawPixelBanner(g, hdc, titles[currentPage].c_str(), screenWidth / 2, UIScaler::SY(70), UIScaler::SX(500), ToCOLORREF(Palette::White), bannerColors[currentPage]);`
8	`}`

4.6.9. SaveGameScreen

SaveGameScreen có layout tương tự LoadGameScreen nhưng ưu tiên thao tác ghi dữ liệu. Màn hình chia thành ba vùng: danh sách slot, thông tin slot đang chọn, và vùng nhập tên bản lưu. UX mục tiêu là giảm thao tác nhầm: slot trống sẽ hiển thị nhãn ”Empty” rõ ràng, slot đã có sẽ hiển thị tên + thời gian lưu gần nhất.

Khung nhập tên dùng `Theme::SaveBoxBorder` để tạo viền cam, giúp người chơi nhận biết vùng focus. Tên được giới hạn độ dài ở tầng lưu dữ liệu, nên UI có thể hiển thị thông báo nếu vượt quá giới hạn mà không làm lỗi file.

Ở chế độ nhập tên, vùng input được tô viền nổi và con trỏ nhấp nháy, đảm bảo người chơi nhận biết trạng thái đang nhập. Khi lưu thành công, thông báo trạng thái hiển thị ngắn ở góc dưới để xác nhận mà không phá bố cục.



Hình 4.12: Hình minh họa SaveGameScreen

Mã nguồn 36: Mã nguồn RenderSaveGameScreen: bố cục và banner

STT	Mã nguồn
1	`void RenderSaveGameScreen(HDC hdc, int selectedOption, const std::wstring &statusMessage, int screenWidth, int screenHeight)`
2	`{`
3	`  Gdiplus::Graphics g(hdc);`
4	`  DrawProceduralStadium(g, screenWidth, screenHeight);`
5	`  int panelW = UIScaler::SX(900);`
6	`  int panelH = UIScaler::SY(540);`
7	`  int panelX = (screenWidth - panelW) / 2;`
8	`  int panelY = (screenHeight - panelH) / 2;`
9	`  Gdiplus::SolidBrush bgBrush(ToGdiColor(Theme::GlassWhite));`
10	`  g.FillRectangle(&bgBrush, panelX, panelY, panelW, panelH);`
11	`  DrawPixelBanner(g, hdc, GetText("save_title").c_str(), screenWidth / 2, panelY + UIScaler::SY(45), panelW - UIScaler::SX(40), ToCOLORREF(Palette::White), RGB(0, 150, 255), "Asset/models/bg/cassette.txt");`
12	`}`

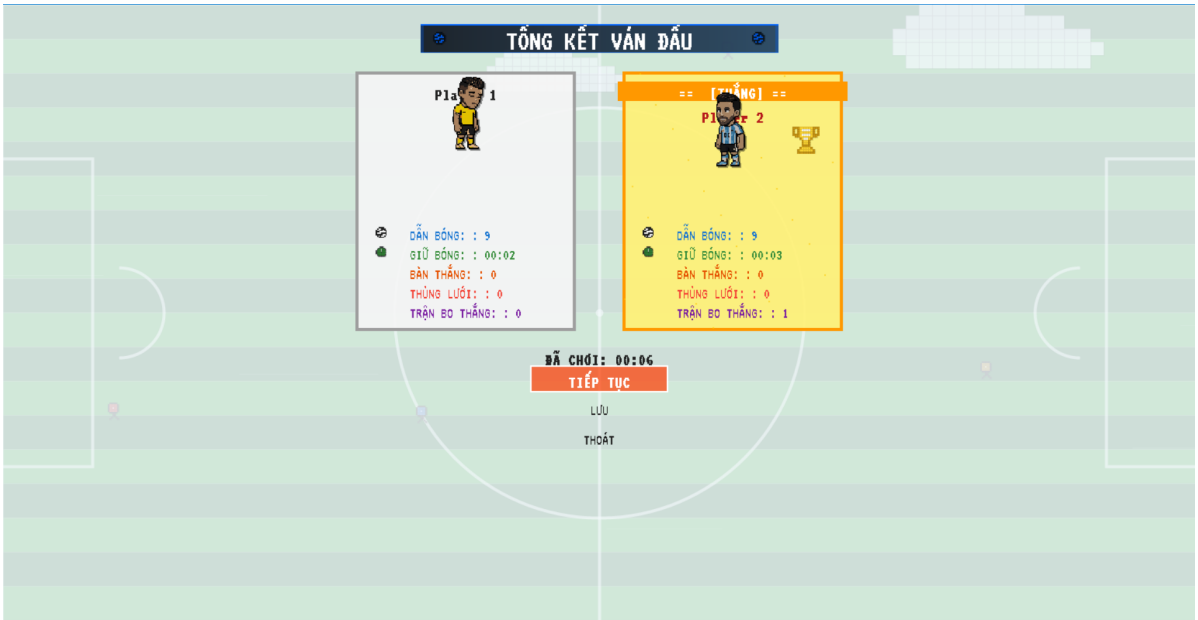
4.6.10. SummaryScreen

SummaryScreen là màn hình tổng kết sau ván/series, tập trung hiển thị thông tin cốt lõi: người thắng, tỷ số, thời gian, và các nút hành động (chơi lại, về menu). UI dùng banner màu nổi và hiệu ứng glow theo màu người thắng để nhấn mạnh kết quả.

Các màu victory banner và highlight kết quả lấy theo Theme (`P1TurnBorder` hoặc `P2TurnBorder`), nên người chơi nhận biết nhanh bên thắng mà không cần đọc chữ.

Phần số liệu được căn giữa theo chiều dọc, dùng font lớn để đảm bảo đọc nhanh. Các nút điều hướng được bố trí dưới cùng và tách biệt bằng khoảng đệm lớn để tránh bấm nhầm.

Hình 4.13: Hình minh họa SummaryScreen



Mã nguồn 37: Mã nguồn RenderSummaryScreen: banner và vùng kết quả

STT	Mã nguồn
1	`void RenderSummaryScreen(HDC hdc, const PlayState *state, int screenWidth, int screenHeight)`
2	`{`
3	`  Gdiplus::Graphics g(hdc);`
4	`  DrawProceduralStadium(g, screenWidth, screenHeight);`
5	`  int panelW = UIScaler::SX(800);`
6	`  int panelH = UIScaler::SY(420);`
7	`  int panelX = (screenWidth - panelW) / 2;`

```
  int panelY = (screenHeight - panelH) / 2;
  Gdiplus::SolidBrush bgBrush(ToGdiColor(Theme::GlassWhite));
  g.FillRectangle(&bgBrush, panelX, panelY, panelW, panelH);
  DrawPixelBanner(g, hdc, GetText("summary_title").c_str(), screenWidth / 2, panelY + UIScaler::SY(45), panelW - UIScaler::SX(40), ToCOLORREF(Palette::White), RGB(255, 180, 0), "Asset/models/bg/trophy.txt");
}
```

4.7. Tối ưu hóa và quản lý tài nguyên

Hệ thống có cơ chế dọn cache tập trung để tránh rò rỉ bộ nhớ. Hàm `ClearUICaches` giải phóng toàn bộ bitmap cache và brush cache khi cần (ví dụ khi đổi độ phân giải lớn hoặc thoát game).

Hàm này dọn tuần tự từng nhóm cache (model, avatar, football, trophy, clock, action, brush). Việc dọn theo nhóm giúp tránh bỏ sót và đảm bảo bộ nhớ đồ họa không bị giữ lại sau khi đổi scene lớn hoặc khi tắt game.

Mã nguồn 38: Mã nguồn Dọn cache UI tập trung

```
void ClearUICaches()
{
  for (auto &pair : g_ModelCache) { if (pair.second) delete pair.second; }
  g_ModelCache.clear();

  for (auto &pair : g_AvatarCache) { if (pair.second) delete pair.second; }
  g_AvatarCache.clear();

  for (auto &pair : g_BrushCache) { if (pair.second) delete pair.second; }
  g_BrushCache.clear();
}
```

Ngoài ra, mô hình “data-driven pixel art” giúp giảm kích thước asset ảnh, đồng thời

cho phép thay đổi palette nhanh theo theme mà không cần sửa lại sprite.

4.8. Vòng lặp ứng dụng và điều phối input

Phần này mô tả vòng lặp chính của ứng dụng (message pump + update/render) và cơ chế throttling input để tránh spam phím. Hai cơ chế này quyết định tính ổn định FPS và cảm giác điều khiển trong UI.

4.8.1. Message pump, dt và giới hạn bước nhảy thời gian

Vòng lặp chính xử lý toàn bộ tin nhắn đang chờ, sau đó mới chạy update và render. Thời gian khung hình `dt` được tính từ `high_resolution_clock` và bị chặn ở ngưỡng 0.1s để tránh "nhảy vọt" khi người dùng kéo/di chuyển cửa sổ. Điều này giữ logic ổn định và tránh cập nhật quá lớn gây vỡ animation.

Mã nguồn 39: Vòng lặp chính: xử lý message và giới hạn dt

STT	Mã nguồn
1	`while (bRunning) {`
2	`  while (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE)) {`
3	`  if (msg.message == WM_QUIT) { bRunning = false; break; }`
4	`  TranslateMessage(&msg);  DispatchMessage(&msg);`
5	`  }`
6	`  if (!bRunning) break;`
7	`  auto frameStart = std::chrono::high_resolution_clock::now();`
8	`  std::chrono::duration<double> elapsed = frameStart - lastTime;`
9	`  lastTime = frameStart;`
10	`  double dt = elapsed.count();`
11	`  if (dt > 0.1) dt = 0.1;`
12	`  g_GlobalAnimTime += (float)dt;`
13	`}`

4.8.2. Throttling input và chống autorepeat quá nhanh

Ở menu và các màn hình điều hướng, phím di chuyển được giới hạn tốc độ để tránh "nhảy" nhiều mục khi người dùng giữ phím. Hệ thống tách nhấn thường và autorepeat,

áp dụng ngưỡng 80ms cho nhấn tay và 150ms cho phím giữ. Cách này giữ cảm giác điều khiển mượt và dễ kiểm soát.

Mã nguồn 40: Throttling phím điều hướng trong MenuScreen

STT	Mã nguồn
1	`bool isRepeat = (wParam & 0x20000) != 0;`
2	`WPARAM rawKey = wParam & 0xFFFF;`
3	`static ULONGLONG lastMoveTime = 0;`
4	`ULONGLONG now = GetTickCount64();`
5	`bool canMove = (now - lastMoveTime > (ULONGLONG)(isRepeat ? 150 : 80));`
6	`if (rawKey == 'W' || rawKey == 'w' || rawKey == VK_UP) {`
7	`  if (!canMove) return false;`
8	`  selectedOption--;`
9	`  if (!isRepeat) PlaySFX("sfx_move");`
10	`  lastMoveTime = now;`
11	`}`

CHƯƠNG 5. LUẬT CHƠI VÀ CƠ CHẾ KIỂM TRA

5.1. Luật và biến thể

Chương này tập trung mô tả hệ luật của trò chơi, cách kiểm tra hợp lệ của nước đi và cơ chế phán xét kết quả theo thời gian thực. Toàn bộ logic luật chơi nằm trong mô-đun GameRules và GameEngine, trong khi trạng thái được lưu ở PlayState. Thiết kế tách lớp giúp luật chơi độc lập với UI, đảm bảo có thể tái sử dụng cho nhiều chế độ và kích thước bàn cờ.

5.1.1. Caro 5 quân và Tic-Tac-Toe

Dự án hỗ trợ hai chế độ chơi được định nghĩa bằng PlayMode: MODE_CARO (Gomoku/Caro 5 quân) và MODE_TTT (Tic-Tac-Toe 3 quân). Hai chế độ dùng chung một engine, chỉ khác tham số độ dài chuỗi thắng (winLength). Nhờ đó, quy tắc thắng được thay đổi mà không cần nhân đôi code.

Mã nguồn 41: Xác định độ dài chuỗi thắng theo chế độ chơi

STT	Mã nguồn
1	`const int winLength = (state->gameMode == MODE_CARO) ? 5 : 3;`

Bàn cờ lưu trong ma trận hai chiều `board[MAX_BOARD_SIZE][MAX_BOARD_SIZE]` với kích thước thực tế theo `boardSize`. Mỗi ô có giá trị `CELL_EMPTY`, `CELL_PLAYER1` hoặc `CELL_PLAYER2`, cho phép thuật toán kiểm tra thắng hoạt động nhất quán trên mọi kích thước. Ở chế độ Caro, giá trị mặc định là 15x15; trong khi Tic-Tac-Toe thường dùng 3x3, nhưng thuật toán vẫn giữ nguyên logic.

5.1.2. Kiểm tra hợp lệ và biên

Trước khi đặt quân, hệ thống bắt buộc kiểm tra hai điều kiện: (1) tọa độ nằm trong phạm vi bàn cờ, (2) ô đang trống. Việc kiểm tra biên không dùng hằng số cứng mà dựa vào `boardSize`, giúp luật chơi tự thích nghi khi người chơi thay đổi kích thước bàn cờ.

Mã nguồn 42: Mã nguồn kiểm tra hợp lệ nước đi

STT	Mã nguồn
1	`bool isValidMove(const PlayState* state, int row, int col) {`
2	`  if (row < 0 || row >= state->boardSize || col < 0 || col >= state->boardSize)`
3	`  return false;`
4	`  return state->board[row][col] == CELL_EMPTY;`
5	`}`

Thiết kế này đảm bảo không thể đánh tràn biên, đồng thời tránh ghi đè quân đã có. Đây là tầng bảo vệ đầu tiên trước khi engine chấp nhận cập nhật trạng thái trận đấu.

5.1.3. Trạng thái tối thiểu phục vụ kiểm tra

Để việc kiểm tra thắng và kết thúc ván diễn ra chính xác, PlayState duy trì các trường dữ liệu tối thiểu: kích thước bàn cờ, ma trận trạng thái, nước đi cuối, danh sách ô thắng và bộ đếm số nước đi của mỗi người chơi. Những trường này giúp thuật toán kiểm tra thắng và điều kiện hòa chạy nhanh mà không phải quét lại toàn bộ bàn cờ.

Mã nguồn 43: Các trường dữ liệu phục vụ kiểm tra thắng và hòa

STT	Mã nguồn
1	`int boardSize = 15;`
2	`int board[MAX_BOARD_SIZE][MAX_BOARD_SIZE] = {0};`
3	`int lastMoveRow = -1;`
4	`int lastMoveCol = -1;`
5	`std::vector<std::pair<int, int>> winningCells;`
6	`int movesCount;  // nằm trong PlayerInfo2 của từng người chơi`

5.2. Thuật toán xác định thắng

5.2.1. Quét 4 hướng tối ưu

Thuật toán phán xét thắng sử dụng ô vừa đặt làm điểm neo và quét theo bốn hướng cơ bản: ngang, dọc, chéo chính và chéo phụ. Mỗi hướng được quét hai chiều (tiến và lùi) để gom toàn bộ chuỗi liên tiếp cùng màu. Nhờ chỉ quét từ nước đi cuối, chi phí tính toán gần như tuyến tính theo độ dài chuỗi thắng thay vì theo kích thước bàn cờ, và dừng sớm ngay khi phát hiện thắng.

Mã nguồn 44: Mã nguồn quét 4 hướng từ nước đi cuối

STT	Mã nguồn
1	`static const int dr[] = { 0, 1, 1,  1 };`
2	`static const int dc[] = { 1, 0, 1, -1 };`
3	`for (int dir = 0; dir < 4; dir++) {`
4	`  std::vector<std::pair<int, int>> line;`
5	`  line.push_back({ lastRow, lastCol });`
6	`  // Quét tiến`
7	`  for (int step = 1; step < size; step++) { …}`
8	`  // Quét lùi`
9	`  for (int step = 1; step < size; step++) { …}`
10	`  if ((int)line.size() >= winLength) { …}`
11	`}`

Thuật toán dừng ngay khi phát hiện chuỗi đủ dài. Điều này giảm độ trễ khi bàn cờ lớn và đảm bảo việc kiểm tra thắng vẫn mượt trong thời gian thực.

5.2.3. Kiểm tra điều kiện hợp lệ trước khi quét

Trước khi bắt đầu quét, hệ thống xác thực nước đi cuối có nằm trong biên và có quân thực sự hay không. Điều này tránh các phép quét thừa và ngăn lỗi khi trạng thái chưa được khởi tạo đầy đủ (ví dụ lúc vừa reset bàn hoặc chưa có nước đi).

Mã nguồn 45: Chặn sớm khi nước đi cuối không hợp lệ

STT	Mã nguồn
1	`if (lastRow < 0 || lastRow >= size || lastCol < 0 || lastCol >= size)`
2	`  return -1;`
3	`const int currentPiece = state->board[lastRow][lastCol];`
4	`if (currentPiece == CELL_EMPTY)`
5	`  return -1;`

5.2.2. Truy vết winningCells cho highlight

Hàm `checkWinCondition()` hỗ trợ tham số tùy chọn `outWinCells`. Khi phát hiện thắng, toàn bộ chuỗi liên tiếp được trả về, không bị cắt ở đúng `winLength`. Điều này

xử lý đúng trường hợp người chơi đặt quân ở giữa, nối hai đoạn rời thành một chuỗi dài hơn. Danh sách ô thắng được lưu trong PlayState và dùng trực tiếp bởi lớp render để highlight.

Mã nguồn 46: Trả về toàn bộ chuỗi thắng và lưu trong PlayState

STT	Mã nguồn
1	`if ((int)line.size() >= winLength) {`
2	`  if (outWinCells) *outWinCells = line;`
3	`  return currentPiece;`
4	`}`

Khi vẽ bàn cờ, UIComponents sử dụng danh sách này để tạo hiệu ứng highlight động trên đúng các ô thắng, đảm bảo trực quan và nhất quán với luật chơi.

Mã nguồn 47: Highlight các ô thắng dựa trên winningCells

STT	Mã nguồn
1	`if (!state->winningCells.empty()) {`
2	`  for (const auto &wCell : state->winningCells) {`
3	`    int drawX = offsetX + wCell.second * cellSize;`
4	`    int drawY = offsetY + wCell.first * cellSize;`
5	`    g.FillRectangle(winBrush, drawX + 1, drawY + 1, cellSize - 2, cellSize - 2);`
6	`    g.DrawRectangle(&winPen, drawX + 2, drawY + 2, cellSize - 4, cellSize - 4);`
7	`  }`
8	`}`

5.3. Trạng thái kết thúc đặc biệt

5.3.1. Hòa và chặn nước đi

Ván đấu kết thúc hòa nếu toàn bộ bàn cờ đã đầy mà không xuất hiện chuỗi thắng. Thay vì quét lại toàn bộ bàn cờ để kiểm tra còn ô trống hay không, hệ thống dùng tổng số nước đi của hai người chơi, giúp tối ưu thời gian kiểm tra.

Mã nguồn 48: Kiểm tra điều kiện hòa bằng bộ đếm nước đi

STT	Mã nguồn
1	`if (state->p1.movesCount + state->p2.movesCount == size * size)`
2	`  return CELL_EMPTY;`
3	`return -1;`

Khi `checkWinCondition()` trả về trạng thái thắng hoặc hòa, GameEngine lập tức kết thúc ván đấu, cập nhật trạng thái và người thắng trong PlayState.

Mã nguồn 49: Cập nhật trạng thái ván đấu sau khi kiểm tra thắng

STT	Mã nguồn
1	`int winStatus = checkWinCondition(state, row, col, &state->winningCells);`
2	`if (winStatus != -1) {`
3	`  state->status = MATCH_FINISHED;`
4	`  state->winner = winStatus;`
5	`} else {  switchTurn(state); }`

5.3.2. Cơ chế BO (Best-of) và thắng series

Ngoài luật thắng từng ván, hệ thống còn hỗ trợ luật thắng theo series BO (BO1/BO3/BO5) thông qua `targetScore`. Sau mỗi ván thắng, engine kiểm tra số bàn thắng cần để kết thúc series. Nếu đã đủ, `matchWins` được tăng, và `totalWins` được reset để bắt đầu series tiếp theo.

Mã nguồn 50: Xác định thắng series BO trong processMove

STT	Mã nguồn
1	`int winRequired = (state->targetScore + 1) / 2;`
2	`if (state->targetScore > 1) {  winRequired = state->targetScore; }`
3	`bool seriesWon = (state->p1.totalWins >= winRequired || state->p2.totalWins >= winRequired);`
4	`if (seriesWon) { …}`

Cách tính `winRequired` đảm bảo BO1 cần 1 ván, BO3 cần 2 ván, BO5 cần 3 ván. Với các thiết lập `targetScore` khác, hệ thống vẫn hoạt động nhất quán vì dùng `targetScore` làm mục tiêu trực tiếp.

5.3.3. Timeout và đồng bộ TimeSystem

Hệ thống thời gian được tách riêng trong TimeSystem, sử dụng cơ chế tích lũy `dt` để đếm giây chính xác và tránh sai số khi khung hình không đều. Mỗi khi đủ một giây, `timeRemaining` giảm đi một đơn vị. Khi về 0, hệ thống báo timeout để xử lý đổi lượt.

Mã nguồn 51: Đếm ngược theo giây bằng bộ tích lũy thời gian

STT	Mã nguồn
1	`timeAccumulator += dt;`
2	`if (timeAccumulator >= 1.0) {`
3	`  timeAccumulator -= 1.0;`
4	`  state->timeRemaining--;`
5	`  if (state->timeRemaining <= 0) { state->timeRemaining = 0; }`
6	`  return true;`
7	`}`
8	`return false;`

PlayScreen đồng bộ timeout bằng cách lắng nghe cờ trả về từ `UpdateCountdown()`. Khi hết thời gian, hệ thống phát âm thanh timeout và đổi lượt ngay lập tức. Điều này biến thời gian thành một ràng buộc luật chơi trực tiếp, không chỉ là hiển thị giao diện.

Mã nguồn 52: Xử lý timeout và đổi lượt trong vòng lặp chơi

STT	Mã nguồn
1	`if (UpdateCountdown(state, dt)) {`
2	`  if (state->timeRemaining <= 0) {`
3	`  PlaySFX("sfx_timeout");`
4	`  switchTurn(state);`
5	`  }`
6	`}`

Mỗi khi đổi lượt, `ResetTimer()` sẽ đọc `maxTurnTime` của người chơi hiện tại để thiết lập lại thời gian đếm ngược. Cách làm này hỗ trợ luật chơi bất đối xứng (hai người chơi có giới hạn thời gian khác nhau) và đảm bảo đồng bộ giữa luật chơi và hiển thị.

CHƯƠNG 6. BỘ MÁY ĐIỀU PHỐI TRẬN ĐẤU

6.1. Điều phối lượt đánh

Bộ máy điều phối ván đấu được tập trung tại GameEngine. Mục tiêu của lớp này là đóng gói toàn bộ pipeline xử lý nước đi thành một điểm vào duy nhất, giúp UI (PlayScreen) chỉ phát lệnh hành động và nhận kết quả, không can thiệp trực tiếp vào luật. Cách tổ chức này tách rõ ba lớp trách nhiệm: GameRules phán xét, GameEngine điều phối, TimeSystem quản lý thời gian. Nhờ vậy, mọi cập nhật đều nhất quán, tránh sai lệch trạng thái khi có nhiều thao tác đồng thời (đặt quân, lưu lịch sử, cập nhật điểm, kiểm tra thắng, đổi lượt).

6.1.1. Luồng xử lý: Nhập lệnh → Luật → Cập nhật trạng thái

Hàm `processMove()` là luồng chuẩn cho một nước đi: kiểm tra hợp lệ, ghi quân lên bàn, cập nhật nước đi cuối, lưu lịch sử, xóa redo (nếu có), cập nhật thống kê, sau đó kiểm tra thắng hoặc đổi lượt. Chỉ khi toàn bộ trạng thái đã cập nhật xong, hệ thống mới phát âm thanh hoặc chuyển màn hình. Thiết kế này giúp việc phục hồi (undo/redo) và đồng bộ thời gian không bị lệch, đồng thời tránh hiện tượng "bán cập nhật" khi có lỗi giữa chừng.

Để đảm bảo tính nguyên tử, các cập nhật quan trọng được thực hiện theo thứ tự cố định: (1) cập nhật bàn cờ và nước đi cuối, (2) ghi lịch sử và xóa redo, (3) cập nhật thống kê, (4) phán xét thắng/thua, (5) đổi lượt hoặc kết thúc ván. Thứ tự này phản ánh đúng logic vận hành của ván đấu và bảo toàn tính nhất quán khi undo/redo.

Mã nguồn 53: Luồng xử lý một nước đi trong processMove

STT	Mã nguồn
1	`if (state->status != MATCH_PLAYING || !isValidMove(state, row, col))`
2	`  return false;`
3	`state->board[row][col] = state->isP1Turn ? CELL_PLAYER1 : CELL_PLAYER2;`

```
state->lastMoveRow = row;
state->lastMoveCol = col;
state->matchHistory.push_back({ row, col });
state->redoStack.clear();
if (state->isP1Turn) state->p1.movesCount++; else state->p2.movesCount++;
int winStatus = checkWinCondition(state, row, col, &state->winningCells);
if (winStatus != -1) {
  if (winStatus == CELL_PLAYER1) state->p1.totalWins++;
  else if (winStatus == CELL_PLAYER2) state->p2.totalWins++;
  int winRequired = (state->targetScore + 1) / 2;
  if (state->targetScore > 1) { winRequired = state->targetScore; }
  bool seriesWon = (state->p1.totalWins >= winRequired || state->p2.totalWins >= winRequired);
  if (seriesWon) { …}
  state->status = MATCH_FINISHED;
  state->winner = winStatus;
  PlaySFX("sfx_whistle");   PlaySFX("sfx_crowd"); PlaySFX("sfx_siu");
} else { switchTurn(state); }
```

Pipeline này đảm bảo nguyên tắc "nguồn sự thật duy nhất": trạng thái luôn được cập nhật tập trung ở GameEngine, tránh sai lệch do cập nhật phân tán. Ngoài ra, việc lưu `lastMoveRow/Col` ngay tại thời điểm đặt quân giúp các bước kiểm tra thắng và highlight luôn bám đúng nước đi cuối.

6.1.2. Đồng bộ lượt và biến isP1Turn

Biến `isP1Turn` trong PlayState đóng vai trò nguồn sự thật duy nhất cho lượt chơi. Mọi hệ thống phụ (UI, AI, TimeSystem) đều đọc biến này thay vì tự suy diễn. Hàm `switchTurn()` đảo lượt và reset thời gian theo cấu hình, đảm bảo đồng bộ tuyệt đối giữa luật chơi và bộ đếm giờ. Cơ chế này cũng giúp việc undo/redo khôi phục đúng lượt hiện tại mà không cần suy luận lại từ lịch sử.

Mã nguồn 54: Đổi lượt và đồng bộ timer

STT	Mã nguồn
1	`void switchTurn(PlayState* state) {`
2	`  state->isP1Turn = !state->isP1Turn;`
3	`  state->timeRemaining = state->countdownTime;`
4	`  ResetTimer(state);`
5	`}`

6.2. Điều phối match series

6.2.1. BO1/BO3/BO5 và mục tiêu thắng

Quy tắc BO đã được trình bày ở chương luật chơi. Tại đây chỉ tập trung vào cách engine hiện thực hóa việc cộng dồn và chốt series. Sau mỗi ván thắng, GameEngine tăng `totalWins`, tính ngưỡng `winRequired` từ `targetScore` và kiểm tra đã đủ điều kiện thắng series hay chưa. Khi thắng series, `matchWins` tăng và `totalWins` được reset về 0 để chuẩn bị cho series tiếp theo. Việc tách biệt này giúp luật chơi rõ ràng, còn engine chỉ chịu trách nhiệm cập nhật trạng thái.

Mã nguồn 55: Xác định thắng series trong processMove

STT	Mã nguồn
1	`int winRequired = (state->targetScore + 1) / 2;`
2	`if (state->targetScore > 1) { winRequired = state->targetScore; }`
3	`bool seriesWon = (state->p1.totalWins >= winRequired ||` `state->p2.totalWins >= winRequired);`
4	`if (seriesWon) { …}`

Cách tính này hoạt động nhất quán với mọi cấu hình `targetScore` và đảm bảo trạng thái series luôn đồng bộ với số ván đã thắng của từng người chơi.

6.2.2. Reset hiệp và bảo toàn tỷ số

Khi chuyển sang một round mới trong cùng series, `startNextRound()` reset bàn cờ, lịch sử nước đi và dữ liệu thời gian, nhưng giữ nguyên điểm series. Điều này giúp kết quả từng ván tách biệt khỏi tiến trình series. Hệ thống cũng đưa con trỏ về giữa bàn và xóa danh sách ô thắng để bắt đầu ván mới sạch sẽ. Ngoài ra, bảng băm trạng thái (nếu dùng cho AI) được xóa để tránh dùng lại dữ liệu của ván trước.

Mã nguồn 56: Khởi tạo round mới, giữ nguyên tiến trình series

STT	Mã nguồn
1	`state->timeRemaining = state->countdownTime;`
2	`state->isP1Turn = true;`
3	`state->status = MATCH_PLAYING;`
4	`state->winner = -1;`
5	`resetPlayerForRound(state->p1);`
6	`resetPlayerForRound(state->p2);`
7	`state->lastMoveRow = -1;`
8	`state->lastMoveCol = -1;`
9	`state->winningCells.clear();`
10	`state->matchHistory.clear();`
11	`state->redoStack.clear();`
12	`state->matchDuration = 0.0f;`
13	`for (int r = 0; r < MAX_BOARD_SIZE; r++)`
14	`for (int c = 0; c < MAX_BOARD_SIZE; c++)`
15	`state->board[r][c] = CELL_EMPTY;`
16	`state->cursorRow = state->boardSize / 2;`
17	`state->cursorCol = state->boardSize / 2;`
18	`clearTranspositionTable();`
19	`ResetTimer(state);`

Việc reset bộ đếm lượt được thực hiện ở PlayerEngineer, tách rõ dữ liệu ván (movesCount) khỏi dữ liệu series (totalWins, matchWins).

Mã nguồn 57: Reset thống kê lượt, giữ nguyên tiến trình BO

STT	Mã nguồn
1	`void resetPlayerForRound(PlayerInfo2& player) {`
2	`player.movesCount = 0;`
3	`// totalWins và matchWins được giữ nguyên`
4	`}`

6.3. Undo/Redo và lịch sử

6.3.1. Stack lưu nước đi

Lịch sử nước đi được quản lý bằng hai vector đóng vai trò stack: `matchHistory` (stack chính) và `redoStack` (stack phục hồi). Khi đặt nước mới, hệ thống push vào history và xóa redo để tránh sai lệch nhánh. Cơ chế này tương đương với mô hình lệnh: mỗi nước đi là một lệnh có thể hoàn tác hoặc phục hồi.

Mã nguồn 58: Ghi lịch sử và hủy nhánh redo khi có nước mới

STT	Mã nguồn
1	`state->matchHistory.push_back({ row, col });`
2	`state->redoStack.clear();`

6.3.2. Khôi phục trạng thái và đồng hồ

Undo không chỉ cập nhật bàn cờ mà còn phải đồng bộ lượt chơi, bộ đếm nước đi, trạng thái thắng và timer. Ở chế độ PvE, hệ thống hoàn tác hai nước cùng lúc để luôn trả lượt về người chơi. Điều này tránh tình huống người chơi phải chờ bot đi ngay sau khi hoàn tác.

Mã nguồn 59: Undo theo cơ chế 2 bước trong PvE

STT	Mã nguồn
1	`int popCount = (state->matchType == MATCH_PVE`
2	`&& state->matchHistory.size() >= 2) ? 2 : 1;`
3	`for (int i = 0; i < popCount; i++) {`
4	`if (state->matchHistory.empty()) break;`
5	`auto last = state->matchHistory.back();`
6	`state->matchHistory.pop_back();`
7	`state->redoStack.push_back(last);`
8	`state->board[last.first][last.second] = CELL_EMPTY;`
9	`state->isP1Turn = !state->isP1Turn;`
10	`if (state->isP1Turn) state->p1.movesCount--; else state->p2.movesCount--;`
11	`}`

Sau khi hoàn tác, hệ thống phải cập nhật lại nước đi cuối để phục vụ việc highlight

và kiểm tra thắng ở các thao tác tiếp theo. Nếu lịch sử rỗng, nước đi cuối được đặt về trạng thái chưa khởi tạo.

Mã nguồn 60: Cập nhật nước đi cuối sau Undo

STT	Mã nguồn
1	`if (state->matchHistory.empty()) {`
2	`  state->lastMoveRow = -1;`
3	`  state->lastMoveCol = -1;`
4	`} else {`
5	`  auto last = state->matchHistory.back();`
6	`  state->lastMoveRow = last.first;`
7	`  state->lastMoveCol = last.second;`
8	`}`

Nếu ván đã kết thúc mà người chơi thực hiện Undo, hệ thống mở lại trạng thái chơi và điều chỉnh điểm thắng để nhất quán với lịch sử, sau đó reset timer cho lượt hiện tại. Ngoài ra, `winningCells` được xóa để tránh highlight cũ ảnh hưởng lên ván đã mở lại.

Mã nguồn 61: Reopen ván đấu sau Undo

STT	Mã nguồn
1	`if (state->status == MATCH_FINISHED) {`
2	`  state->status = MATCH_PLAYING;`
3	`  if (state->winner == CELL_PLAYER1) state->p1.totalWins--;`
4	`  else if (state->winner == CELL_PLAYER2) state->p2.totalWins--;`
5	`  state->winner = -1;`
6	`}`
7	`state->winningCells.clear();`
8	`ResetTimer(state);`

6.4. Hệ thống thời gian

6.4.1. Đếm ngược lượt và thời gian giữ bóng

Hệ thống thời gian quản lý hai trục độc lập: đếm ngược cho từng lượt (`timeRemaining`) và tổng thời gian giữ bóng (`totalTimePossessed`). Tại vòng lặp

PlayScreen, mỗi khung hình cộng dồn thời gian cho người đang tới lượt, đồng thời cập nhật đếm ngược để kiểm soát timeout. Nhờ vậy, luật chơi vẫn chuẩn xác ngay cả khi tốc độ khung hình dao động.

Mã nguồn 62: Cộng dồn thời gian giữ bóng và kiểm soát timeout

STT	Mã nguồn
1	`if (state->status == MATCH_PLAYING) {`
2	`  state->matchDuration += (float)dt;`
3	`  if (state->isP1Turn) { state->p1.totalTimePossessed += (float)dt; }`
4	`  else { state->p2.totalTimePossessed += (float)dt; }`
5	`  if (state->matchType == MATCH_PVE) { …}`
6	`  else if (UpdateCountdown(state, dt)) {`
7	`  if (state->timeRemaining <= 0) { PlaySFX("sfx_timeout"); switchTurn(state); }`
8	`  }`
9	`}`

Việc tách riêng hai loại thời gian giúp luật chơi (timeout) và thống kê sau trận (possession) không ảnh hưởng lẫn nhau, đồng thời cung cấp dữ liệu phân tích cho người chơi.

6.4.2. Quỹ thời gian toàn trận và khởi tạo match

Khi khởi tạo match mới, `initNewMatch()` thiết lập đầy đủ thông số chế độ, kích thước bàn, thời gian lượt, độ khó bot và điểm mục tiêu BO. Đồng thời, nếu người chơi bật chế độ tính giờ toàn trận, hệ thống sẽ khởi tạo quỹ thời gian cho mỗi bên ngay tại đây. Hàm này cũng khởi tạo thông tin người chơi, xóa bảng băm AI và gọi `startNextRound()` để đảm bảo trạng thái ban đầu sạch.

Mã nguồn 63: Khởi tạo trận mới và quỹ thời gian toàn trận

STT	Mã nguồn
1	`state->gameMode = mode;`
2	`state->matchType = type;`
3	`state->boardSize = boardSize;`
4	`state->countdownTime = countdownTime;`

5	`state->difficulty = difficulty;`
6	`state->targetScore = targetScore;`
7	`state->isMatchTimed = (totalTime > 0);`
8	`state->p1TotalTimeLeft = totalTime * 60;`
9	`state->p2TotalTimeLeft = totalTime * 60;`
10	`initPlayer(state->p1, state->p1.name, state->p1.avatarPath, 'X', (float)countdownTime);`
11	`initPlayer(state->p2, state->p2.name, state->p2.avatarPath, 'O', (float)countdownTime);`
12	`clearTranspositionTable();`
13	`startNextRound(state);`

6.3.3. Phục hồi redo và phán xét lại trạng thái

Redo thực hiện ngược với Undo: lấy nước đi từ `redoStack`, đặt lại lên bàn, cập nhật lịch sử, rồi kiểm tra thắng. Nếu phát hiện kết thúc ván, hệ thống cập nhật trạng thái và người thắng; nếu chưa kết thúc thì đổi lượt bình thường.

Mã nguồn 64: Phục hồi nước đi trong redoMove

STT	Mã nguồn
1	`if (state->redoStack.empty()) return;`
2	`int popCount = (state->matchType == MATCH_PVE`
3	`&& state->redoStack.size() >= 2) ? 2 : 1;`
4	`for (int i = 0; i < popCount; i++) {`
5	`if (state->redoStack.empty()) break;`
6	`auto nextMove = state->redoStack.back();`
7	`state->redoStack.pop_back();`
8	`state->board[nextMove.first][nextMove.second] = state->isP1Turn ? CELL_PLAYER1 : CELL_PLAYER2;`
9	`state->lastMoveRow = nextMove.first;`
10	`state->lastMoveCol = nextMove.second;`
11	`state->matchHistory.push_back(nextMove);`
12	`if (state->isP1Turn) state->p1.movesCount++; else state->p2.movesCount++;`

```
  int winStatus = checkWinCondition(state, nextMove.first, nextMove.second, &state->winningCells);
  if (winStatus != -1) { state->status = MATCH_FINISHED; state->winner = winStatus; }
  else { state->isP1Turn = !state->isP1Turn; }
}
ResetTimer(state);
```

CHƯƠNG 7. TRÍ TUỆ NHÂN TẠO

7.1. Tổng quan kiến trúc AI

Trí tuệ nhân tạo (AI) của trò chơi được hiện thực trong mô-đun BotAI. Hệ thống cung cấp 3 mức độ khó: dễ (ngẫu nhiên), trung bình (đánh giá heuristic cục bộ), và khó (tìm kiếm alpha-beta có bảng băm trạng thái). Mỗi mức độ đều dùng chung cấu trúc dữ liệu bàn cờ nhưng khác chiến lược ra quyết định.

Về mặt nguyên lý, AI được thiết kế theo ba tầng: (1) tầng chọn nước đi nhanh để đảm bảo phản hồi tức thì, (2) tầng đánh giá cục bộ để có quyết định chiến thuật hợp lý, và (3) tầng tìm kiếm sâu để nhìn trước nhiều nước. Việc chia tầng giúp cân bằng giữa chất lượng và thời gian phản hồi, phù hợp đặc thù ván cờ Caro có không gian trạng thái rất lớn.

Mã nguồn 65: Điểm vào chính của AI theo mức độ khó

STT	Mã nguồn
1	`void calculateAIMove(const PlayState* state, int difficulty, int& outRow, int& outCol) {`
2	`  initZobrist();`
3	`  if (difficulty == 1) { randomMove(state, outRow, outCol); return; }`
4	`  if (difficulty == 2) { simpleMove(state, outRow, outCol); return; }`
5	`  int size = state->boardSize;`
6	`  int winLen = (state->gameMode == MODE_CARO) ? 5 : 3;`
7	`  int depth = 5;`
8	`  // Sao chép bàn cờ sang virtualBoard để thử nước đi`
9	`  int virtualBoard[MAX_BOARD_SIZE][MAX_BOARD_SIZE];`
10	`  for (int r = 0; r < MAX_BOARD_SIZE; r++)`

```
    for (int c = 0; c < MAX_BOARD_SIZE; c++) { virtualBoard[r][c] = state->board[r][c]; }
  uint64_t rootHash = buildHash(virtualBoard, size);
  std::vector<MoveCandidate> moves;
  moves.reserve(64);
  for (int r = 0; r < size; r++) {
    for (int c = 0; c < size; c++) {
      if (virtualBoard[r][c] != CELL_EMPTY || !isCandidate(virtualBoard, size, r, c)) continue;
      int atk = scoreMove(virtualBoard, size, winLen, r, c, CELL_PLAYER2);
      int def = scoreMove(virtualBoard, size, winLen, r, c, CELL_PLAYER1);
      moves.push_back({ r, c, atk + def * 2 });
    }
  }
  if (moves.empty()) { randomMove(state, outRow, outCol); return; }
  sort(moves.begin(), moves.end(), [](const MoveCandidate& a, const MoveCandidate& b) { return a.score > b.score; });
  // Ưu tiên nước thắng ngay hoặc chặn thua ngay
  for (const auto& m : moves) {
    virtualBoard[m.r][m.c] = CELL_PLAYER2;
    if (hasImmediateWin(virtualBoard, size, winLen, CELL_PLAYER2)) { outRow = m.r; outCol = m.c; return; }
    virtualBoard[m.r][m.c] = CELL_EMPTY;
  }
  for (const auto& m : moves) {
    virtualBoard[m.r][m.c] = CELL_PLAYER1;
    if (hasImmediateWin(virtualBoard, size, winLen, CELL_PLAYER1)) { outRow = m.r; outCol = m.c; return; }
    virtualBoard[m.r][m.c] = CELL_EMPTY;
  }
  int bestScore = INT_MIN;
```

```
  outRow = moves[0].r;
  outCol = moves[0].c;
  for (const auto& m : moves) {
    uint64_t newHash = rootHash ^ zobristFor(m.r, m.c, CELL_PLAYER2);
    virtualBoard[m.r][m.c] = CELL_PLAYER2;
    int score = alphaBeta(virtualBoard, size, winLen, depth - 1, INT_MIN, INT_MAX, false, newHash);
    virtualBoard[m.r][m.c] = CELL_EMPTY;
    if (score > bestScore) { bestScore = score; outRow = m.r; outCol = m.c; }
  }
}
```

Ba chiến lược này được thiết kế để cân bằng giữa tốc độ phản hồi và chất lượng nước đi. Dễ đảm bảo phản hồi tức thì, trung bình có tính chiến thuật cục bộ, còn khó có khả năng nhìn trước nhiều nước nhờ cây tìm kiếm.

7.1.2. Gọi AI bất đồng bộ để giữ mượt giao diện

Ở chế độ PvE, việc tìm nước đi có thể tốn thời gian (alpha-beta + bảng băm). Để tránh block luồng render, PlayScreen chạy AI bằng `std::async` và chỉ áp dụng kết quả khi `future` báo sẵn sàng. Cách này giúp UI tiếp tục cập nhật animation và đồng hồ trong lúc bot đang tính.

Mã nguồn 66: Chạy AI bất đồng bộ và nhận kết quả an toàn

```
if (state->status == MATCH_PLAYING && state->matchType == MATCH_PVE && !state->isP1Turn) {
  if (!g_AIsCalculating) {
    g_AIsCalculating = true;
    PlayState localState = *state;
    g_AIFuture = std::async(std::launch::async, [localState]() {
      int r, c;
      calculateAIMove(&localState, localState.difficulty, r, c);
```

```
      return std::make_pair(r, c); });
  } else if (g_AIFuture.valid() && g_AIFuture.wait_for(std::chrono::seconds(0)) == std::future_status::ready) {
    auto bestMove = g_AIFuture.get();
    g_AIsCalculating = false;
    if (processMove(state, bestMove.first, bestMove.second)) {
      PlaySFX("sfx_place");
    }
  }
}
```

7.2. Đánh giá heuristic (đánh giá cục bộ)

7.2.1. Nhận dạng mẫu công/thủ theo 4 hướng

Hệ thống đánh giá một vị trí dựa trên việc phân tích chuỗi quân liên tiếp theo 4 hướng: ngang, dọc, chéo chính, chéo phụ. Hàm `analyzeDirection()` đếm số quân liên tiếp và số đầu hở thực sự. Đầu hở chỉ được tính khi ô kế tiếp nằm trong bàn và trống, không tính biên hoặc quân địch.

Các bước của thuật toán phân tích hướng như sau:

- Bước 1: lấy ô đang xét làm trung tâm, khởi tạo `count = 1`, `openEnds = 0`.
- Bước 2: quét tiến theo (dr, dc) để đếm chuỗi liên tiếp cùng quân, dừng khi gặp biên hoặc ô khác quân.
- Bước 3: nếu ô dừng lại vẫn nằm trong bàn và trống, tăng `openEnds`.
- Bước 4: quét lùi theo (-dr, -dc) tương tự bước 2.
- Bước 5: kiểm tra đầu hở phía lùi và tăng `openEnds` nếu hợp lệ.

Thiết kế này phản ánh đúng bản chất chiến thuật Caro: chuỗi dài và hai đầu hở là nguy hiểm nhất; chuỗi bị chặn ở biên hoặc bởi quân đối thủ có giá trị thấp hơn.

Mã nguồn 67: Phân tích chuỗi và đầu hở theo một hướng

```
LineInfo analyzeDirection(
```

```
  const int board[MAX_BOARD_SIZE][MAX_BOARD_SIZE],
  int size, int row, int col, int dr, int dc, int player) {
  LineInfo res = { 1, 0 };
  // Hướng tiến
  int r = row + dr, c = col + dc;
  while (isInBounds(r, c, size) && board[r][c] == player) {
    res.count++; r += dr; c += dc;
  }
  if (isInBounds(r, c, size) && board[r][c] == CELL_EMPTY) res.openEnds++;
  // Hướng lùi
  r = row - dr; c = col - dc;
  while (isInBounds(r, c, size) && board[r][c] == player) {
    res.count++; r -= dr; c -= dc;
  }
  if (isInBounds(r, c, size) && board[r][c] == CELL_EMPTY) res.openEnds++;
  return res;
}
```

Cách làm này đảm bảo phân biệt rõ chuỗi bị chặn bởi biên bàn và chuỗi bị chặn bởi quân đối phương, giúp AI đánh giá chính xác mức độ nguy hiểm của một thế cờ.

7.2.2. Trọng số chuỗi mở và chuỗi bị chặn

Hàm `evaluateLine()` ánh xạ (count, openEnds) sang điểm số. Chuỗi có hai đầu hở được ưu tiên cao vì khả năng mở rộng lớn, chuỗi bị chặn cả hai đầu có điểm 0 vì không còn giá trị tấn công. Với chuỗi đạt độ dài thắng, điểm số được đẩy lên mức cực lớn để ưu tiên tuyệt đối.

Lý do dùng bảng trọng số cố định là để AI phản ứng nhanh, không cần đánh giá toàn bộ bàn cờ theo cách tốn kém. Trong thực tế, chỉ một vài mẫu cơ bản (2, 3, 4 quân liên tiếp với số đầu hở khác nhau) đã đủ mô tả mức độ nguy hiểm của thế cờ.

Các bước tính điểm cho một chuỗi cụ thể:

- Bước 1: nếu `count` đạt hoặc vượt `winLen`, trả về điểm cực lớn để ưu tiên thắng.
- Bước 2: nếu `openEnds = 0`, trả về 0 vì chuỗi bị chặn hoàn toàn.

- Bước 3: xác định chuỗi có hai đầu hở hay một đầu hở.
- Bước 4: ánh xạ (`count`, `openEnds`) vào trọng số tương ứng.

Mã nguồn 68: Bảng trọng số đánh giá một chuỗi

STT	Mã nguồn
1	`if (line.count >= winLen) return 1000000;`
2	`if (line.openEnds == 0) return 0;`
3	`bool open2 = (line.openEnds == 2);`
4	`switch (line.count) {`
5	`  case 4: return open2 ? 50000 : 10000;`
6	`  case 3: return open2 ? 5000 : 1000;`
7	`  case 2: return open2 ? 500 : 100;`
8	`  case 1: return open2 ? 50 : 10;`
9	`  default: return 0;`
10	`}`

7.2.3. Tính điểm một nước đi theo công/thủ

Điểm của một ô ứng cử viên được tính bằng tổng điểm bốn hướng. Ở chế độ trung bình và khó, AI chấm điểm cả tấn công (đặt cho mình) và phòng thủ (đặt để chặn đối phương), sau đó cộng theo trọng số để xếp hạng nước đi.

Cơ chế này có vai trò như một bộ lọc chiến thuật: nếu điểm phòng thủ cao, AI sẽ ưu tiên chặn đối thủ; nếu điểm tấn công cao, AI sẽ ưu tiên mở rộng chuỗi của mình. Đây là nền tảng cho cả `simpleMove` và danh sách ứng cử viên trong alpha-beta.

Quy trình chấm điểm một ô ứng cử viên:

- Bước 1: với 4 hướng cơ bản, gọi `analyzeDirection` để lấy `count` và `openEnds`.
- Bước 2: chuyển từng hướng sang điểm bằng `evaluateLine`.
- Bước 3: cộng điểm 4 hướng để ra tổng điểm tấn công hoặc phòng thủ.
- Bước 4: kết hợp điểm công/thủ để xếp hạng nước đi.

Mã nguồn 69: Tính điểm một ô theo 4 hướng

STT	Mã nguồn

```
int scoreMove(
  const int board[MAX_BOARD_SIZE][MAX_BOARD_SIZE],
  int size, int winLen, int row, int col, int player) {
  static const int dirs[4][2] = { {0,1}, {1,0}, {1,1}, {1,-1} };
  int total = 0;
  for (int d = 0; d < 4; d++) {
    LineInfo line = analyzeDirection(board, size, row, col, dirs[d][0], dirs[d][1], player);
    total += evaluateLine(line, winLen);
  }
  return total;
}
```

7.3. Tìm kiếm cây quyết định

7.3.1. Minimax và cắt tỉa alpha-beta

Ở mức khó, AI dùng thuật toán alpha-beta trên cây nước đi. Hàm `alphaBeta()` nhận bàn cờ, độ sâu, và khoảng giá trị `[alpha, beta]`. Nếu `beta` không thể vượt qua `alpha`, nhánh được cắt bỏ để tiết kiệm thời gian.

Quy trình alpha-beta trong dự án gồm các bước chính:

- Bước 1: tra cứu Transposition Table (TT) để lấy kết quả đã tính ở độ sâu tương đương; nếu đủ tin cậy thì trả về ngay.
- Bước 2: nếu đạt độ sâu 0, trả về điểm đánh giá cục bộ bằng `computeIncrementalScore`.
- Bước 3: kiểm tra thắng ngay (quiescence nhẹ) để ưu tiên trạng thái kết thúc rõ ràng.
- Bước 4: sinh danh sách ứng cử viên, sắp xếp theo điểm, giới hạn nhánh.
- Bước 5: thử từng nước đi, gọi đệ quy alpha-beta, cập nhật `alpha` hoặc `beta`.
- Bước 6: nếu `beta <= alpha` thì cắt tỉa nhánh còn lại.
- Bước 7: lưu kết quả vào TT với cờ biên phù hợp (EXACT/LOWER/UPPER).

Thuật toán này giúp giảm số lượng node phải duyệt, từ đó đạt độ sâu lớn hơn trong cùng thời gian, phù hợp với nhịp độ chơi thời gian thực. Nhờ các bước này, AI có thể đạt độ sâu 5 trong thời gian thực tế mà không làm gián đoạn trải nghiệm người chơi.

Mã nguồn 70: Hàm alpha-beta đầy đủ (có TT và cắt tỉa)

STT	Mã nguồn
1	`int alphaBeta(int board[MAX_BOARD_SIZE][MAX_BOARD_SIZE], int size, int winLen, int depth, int alpha, int beta, bool isMax, uint64_t hash) {`
2	`  TTEntry* tte = ttLookup(hash);`
3	`  if (tte->hash == hash && tte->depth >= depth) {`
4	`  if (tte->flag == TT_EXACT) return tte->score;`
5	`  if (tte->flag == TT_LOWER && tte->score > alpha) alpha = tte->score;`
6	`  if (tte->flag == TT_UPPER && tte->score < beta) beta = tte->score;`
7	`  if (alpha >= beta) return tte->score;`
8	`  }`
9	`  if (depth == 0) return computeIncrementalScore(board, size, winLen);`
10	`  if (hasImmediateWin(board, size, winLen, CELL_PLAYER2)) return 900000 + depth;`
11	`  if (hasImmediateWin(board, size, winLen, CELL_PLAYER1)) return -900000 - depth;`
12	`  std::vector<MoveCandidate> moves; moves.reserve(64);`
13	`  int curPlayer = isMax ? CELL_PLAYER2 : CELL_PLAYER1;`
14	`  int oppPlayer = isMax ? CELL_PLAYER1 : CELL_PLAYER2;`
15	`  for (int r = 0; r < size; r++) {`
16	`  for (int c = 0; c < size; c++) {`
17	`  if (board[r][c] != CELL_EMPTY || !isCandidate(board, size, r, c)) continue;`
18	`  int atk = scoreMove(board, size, winLen, r, c, curPlayer);`
19	`  int def = scoreMove(board, size, winLen, r, c, oppPlayer);`
20	`  moves.push_back({ r, c, atk + def });`
21	`  }`

```
  }
  if (moves.empty()) return computeIncrementalScore(board, size, winLen);
  sort(moves.begin(), moves.end(), [](const MoveCandidate& a, const MoveCandidate& b) { return a.score > b.score; });
  static constexpr int MAX_BRANCHES = 12;
  if ((int)moves.size() > MAX_BRANCHES) moves.resize(MAX_BRANCHES);
  int originalAlpha = alpha;
  int best = isMax ? INT_MIN : INT_MAX;
  for (const auto& m : moves) {
    uint64_t newHash = hash ^ zobristFor(m.r, m.c, isMax ? CELL_PLAYER2 : CELL_PLAYER1);
    board[m.r][m.c] = isMax ? CELL_PLAYER2 : CELL_PLAYER1;
    int val = alphaBeta(board, size, winLen, depth - 1, alpha, beta, !isMax, newHash);
    board[m.r][m.c] = CELL_EMPTY;
    if (isMax) { if (val > best) best = val; if (best > alpha) alpha = best; }
    else { if (val < best) best = val; if (best < beta) beta = best; }
    if (beta <= alpha) break;
  }
  TTFlag flag;
  if (best <= originalAlpha) flag = TT_UPPER;
  else if (best >= beta) flag = TT_LOWER;
  else flag = TT_EXACT;
  ttStore(hash, best, depth, flag);
  return best;
}
```

7.3.2. Sắp xếp nước đi

Trước khi tìm kiếm, danh sách ứng cử viên được sắp xếp giảm dần theo điểm công + thủ. Nếu nước tốt được duyệt trước, alpha-beta sẽ cắt tỉa nhiều hơn, tăng hiệu suất.

Đây là bước quan trọng để giảm thời gian tìm kiếm: khi nước mạnh được xét trước, `alpha` hoặc `beta` nhanh chóng thu hẹp khoảng giá trị, giúp cắt bỏ nhiều nhánh yếu.

Mã nguồn 71: Tạo và sắp xếp danh sách ứng cử viên

STT	Mã nguồn
1	`std::vector<MoveCandidate> moves;`
2	`moves.reserve(64);`
3	`for (int r = 0; r < size; r++) {`
4	`  for (int c = 0; c < size; c++) {`
5	`  if (virtualBoard[r][c] != CELL_EMPTY || !isCandidate(virtualBoard, size, r, c)) continue;`
6	`  int atk = scoreMove(virtualBoard, size, winLen, r, c, CELL_PLAYER2);`
7	`  int def = scoreMove(virtualBoard, size, winLen, r, c, CELL_PLAYER1);`
8	`  moves.push_back({ r, c, atk + def * 2 });`
9	`  }`
10	`}`
11	`sort(moves.begin(), moves.end(), [](const MoveCandidate& a, const MoveCandidate& b) { return a.score > b.score; });`

7.3.3. Ưu tiên thắng ngay và chặn thua

Ở nút gốc, AI kiểm tra nước thắng ngay lập tức cho mình, sau đó kiểm tra nước chặn thắng của đối phương. Nếu phát hiện, trả về ngay mà không cần chạy alpha-beta, đảm bảo phản xạ chiến thuật chính xác.

Mục tiêu của bước này là loại bỏ các trường hợp hiển nhiên: nếu có nước thắng tức thì, mọi tìm kiếm sâu hơn là không cần thiết; nếu đối thủ có nước thắng ngay, AI phải chặn lập tức để tránh thua.

Mã nguồn 72: Ưu tiên nước thắng hoặc chặn thua tại gốc

STT	Mã nguồn
1	`for (const auto& m : moves) {`
2	`  virtualBoard[m.r][m.c] = CELL_PLAYER2;`
3	`  if (hasImmediateWin(virtualBoard, size, winLen, CELL_PLAYER2)) {`
4	`  outRow = m.r; outCol = m.c;`

```
      virtualBoard[m.r][m.c] = CELL_EMPTY;
      return;
    }
    virtualBoard[m.r][m.c] = CELL_EMPTY;
  }
  for (const auto& m : moves) {
    virtualBoard[m.r][m.c] = CELL_PLAYER1;
    if (hasImmediateWin(virtualBoard, size, winLen, CELL_PLAYER1)) {
      outRow = m.r; outCol = m.c;
      virtualBoard[m.r][m.c] = CELL_EMPTY;
      return;
    }
    virtualBoard[m.r][m.c] = CELL_EMPTY;
  }
```

7.4. Tối ưu và tăng tốc

7.4.1. Zobrist hashing

Zobrist hashing mã hóa trạng thái bàn cờ thành một số 64-bit bằng phép XOR. Mỗi cặp (ô, quân) được gán một số ngẫu nhiên; khi đặt hoặc xóa quân, hash mới chỉ cần XOR thêm một giá trị, rất nhanh.

Lý do dùng Zobrist là vì trạng thái bàn cờ thay đổi liên tục khi thử nước đi trong alpha-beta. Việc tính lại hash toàn bàn sẽ tốn $O(n^2)$, còn Zobrist cho phép cập nhật $O(1)$ bằng XOR.

Trình tự hoạt động của Zobrist:

- Bước 1: khởi tạo bảng số ngẫu nhiên cho từng cặp (ô, quân).
- Bước 2: xây hash ban đầu bằng XOR các ô đang có quân.
- Bước 3: khi thử một nước đi, chỉ cần XOR thêm (hoặc XOR lại) giá trị tương ứng.

Mã nguồn 73: Khởi tạo bảng Zobrist và sinh hash

STT	Mã nguồn
1	`mt19937_64 rng(0xDEADBEEFCAFEBABEULL);`
2	`for (int r = 0; r < MAX_BOARD_SIZE; r++)`
3	`  for (int c = 0; c < MAX_BOARD_SIZE; c++)`
4	`  for (int p = 0; p < 2; p++)`
5	`  zobristTable[r][c][p] = rng();`
6	`uint64_t h = 0;`
7	`if (board[r][c] != CELL_EMPTY) h ^= zobristFor(r, c, board[r][c]);`

7.4.2. Transposition Table

Transposition Table lưu lại kết quả đánh giá của các trạng thái đã duyệt, giúp tránh tính lại ở những nhánh trùng lặp. Bảng có kích thước 2^{20} entry. Mỗi entry lưu hash, điểm, độ sâu và cờ biên (exact/lower/upper).

Trong cây tìm kiếm, cùng một trạng thái có thể xuất hiện qua nhiều thứ tự nước đi khác nhau. TT giúp tái sử dụng kết quả cũ, giảm đáng kể số node phải duyệt. Cờ biên được dùng để cập nhật `alpha/beta` đúng mức và đảm bảo tính an toàn của cắt tỉa.

Các bước sử dụng TT:

- Bước 1: tra bảng bằng hash, nếu có kết quả đủ độ sâu thì dùng ngay.
- Bước 2: nếu không đủ, tiếp tục tìm kiếm và cập nhật kết quả mới.
- Bước 3: lưu kết quả kèm cờ biên để lần sau sử dụng đúng ngữ cảnh.

Mã nguồn 74: Lưu và tra cứu bảng TT

STT	Mã nguồn
1	`TTEntry* tte = ttLookup(hash);`
2	`if (tte->hash == hash && tte->depth >= depth) {`
3	`  if (tte->flag == TT_EXACT) return tte->score;`
4	`  if (tte->flag == TT_LOWER && tte->score > alpha) alpha = tte->score;`
5	`  if (tte->flag == TT_UPPER && tte->score < beta) beta = tte->score;`
6	`  if (alpha >= beta) return tte->score;`
7	`}`

8	`TTFlag flag;`
9	`if (best <= originalAlpha) flag = TT_UPPER;`
10	`else if (best >= beta) flag = TT_LOWER;`
11	`else flag = TT_EXACT;`
12	`ttStore(hash, best, depth, flag);`

7.4.3. Lọc ứng cử viên và giới hạn nhánh

Để giảm không gian tìm kiếm, AI chỉ xét các ô trống nằm trong bán kính 2 ô quanh những quân đã có. Đồng thời, số lượng nước đi mỗi tầng bị giới hạn ở `MAX_BRANCHES = 12`. Nhờ đó, độ sâu 5 vẫn chạy được trong thời gian thực.

Lọc ứng cử viên dựa trên quan sát thực tế: các nước đi xa vùng đang có quân hiếm khi mang lại lợi ích chiến thuật tức thời. Giới hạn nhánh giúp chi phí tìm kiếm tăng tuyến tính theo độ sâu thay vì bùng nổ theo số ô trống toàn bàn.

Trình tự lọc và giới hạn:

- Bước 1: bỏ qua các ô không nằm gần vùng đã có quân.
- Bước 2: chấm điểm và sắp xếp các ô còn lại.
- Bước 3: giữ lại tối đa `MAX_BRANCHES` nước đi tốt nhất cho mỗi tầng.

Mã nguồn 75: Lọc ứng cử viên theo bán kính và giới hạn nhánh

STT	Mã nguồn
1	`if (board[r][c] != CELL_EMPTY || !isCandidate(board, size, r, c)) continue;`
2	`static constexpr int MAX_BRANCHES = 12;`
3	`if ((int)moves.size() > MAX_BRANCHES) moves.resize(MAX_BRANCHES);`

CHƯƠNG 8. MÔ-ĐUN HỆ THỐNG PHỤ TRỢ

8.1. Lưu trữ và serialization

Hệ thống lưu/đọc ván đấu được triển khai trong mô-đun `SaveLoadSystem`. Mục tiêu là lưu trữ đầy đủ trạng thái trận đấu (PlayState) dưới dạng binary để tốc độ đọc/ghi nhanh, đồng thời có khả năng tương thích phiên bản và chống lỗi file [2]. Cấu trúc dữ liệu lưu giữ bao gồm các trường POD (game mode, thời gian, bàn cờ, lượt, v.v.) và các trường động (thông tin người chơi, lịch sử nước đi, redo stack, danh sách ô thắng).

Thiết kế ưu tiên tốc độ I/O: dữ liệu được ghi theo thứ tự tuyến tính, hạn chế chuyển đổi trung gian. Việc dùng định dạng binary giúp file nhỏ hơn so với JSON/XML, giảm thời gian đọc/ghi và ít gây phân mảnh dữ liệu trong quá trình lưu liên tục. Hệ thống cũng đảm bảo tính nhất quán bằng cách coi việc ghi/đọc như một giao thức tuần tự: chỉ cần sai lệch thứ tự hoặc độ dài là quá trình đọc sẽ dừng sớm và trả về lỗi, tránh gây hỏng trạng thái trong bộ nhớ.

8.1.1. Binary Save: magic, version và chuỗi Unicode

File lưu bắt đầu bằng `SAVE_MAGIC` và `SAVE_VERSION` nhằm xác thực tính hợp lệ và cho phép nâng cấp cấu trúc theo thời gian. Chuỗi tiếng Việt được lưu bằng `std::wstring` để đảm bảo hiển thị đúng dấu, trong khi avatar path dùng `std::string`. Các hàm hỗ trợ `WriteWStr/ReadWStr` và `WriteStr/ReadStr` giúp thống nhất quy ước ghi độ dài trước khi ghi dữ liệu thô.

Tổ hợp `magic + version` đóng vai trò như chữ ký file và mốc phát triển định dạng. Nhờ đó, hệ thống có thể từ chối các file không thuộc trò chơi hoặc file bị sửa tay. Việc lưu chuỗi theo định dạng `[len][data]` giúp tránh đọc tràn, đồng thời giữ được ký tự Unicode đầy đủ. Các giới hạn độ dài (4096 ký tự) đóng vai trò bộ lọc an toàn trong trường hợp file bị hỏng hoặc bị chèn dữ liệu rác.

Mã nguồn 76: Ghi/đọc chuỗi Unicode kèm độ dài

STT	Mã nguồn
1	`static void WriteWStr(std::ofstream& f, const std::wstring& s) {`

```
  uint32_t len = (uint32_t)s.size();
  f.write(reinterpret_cast<const char*>(&len), sizeof(len));
  if (len > 0) {
    f.write(reinterpret_cast<const char*>(s.data()), len * sizeof(wchar_t));
  }
}
static bool ReadWStr(std::ifstream& f, std::wstring& s) {
  uint32_t len = 0;
  if (!f.read(reinterpret_cast<char*>(&len), sizeof(len))) return false;
  if (len > 4096) return false;
  s.resize(len);
  if (len > 0 && !f.read(reinterpret_cast<char*>(&s[0]), len * sizeof(wchar_t))) return false;
  return true;
}
```

Độ dài chuỗi được giới hạn (4096 ký tự) để tránh file lỗi hoặc dữ liệu rác gây tràn bộ nhớ khi đọc.

8.1.2. Serialize PlayerInfo2 và các vector động

Đối tượng người chơi `PlayerInfo2` chứa cả dữ liệu tĩnh (tên, avatar, ký hiệu quân) và dữ liệu động theo ván (wins, số nước đi, thời gian). Hệ thống ghi theo thứ tự cố định; khi đọc sẽ dựa vào `SAVE_VERSION` để quyết định có tồn tại các trường mới hay không. Với các mảng động như `winningCells`, `matchHistory`, `redoStack`, hệ thống ghi số lượng phần tử rồi ghi mảng thô.

Việc tuần tự hóa `PlayerInfo2` tách rõ phần "dữ liệu nhận diện" (name, avatar, piece) và phần "thống kê" (wins, moves, time). Nhờ đó, khi bổ sung trường mới ở phiên bản sau, hệ thống vẫn có thể gán giá trị mặc định trong quá trình đọc để tránh rác bộ nhớ. Các vector động được ghi bằng `WriteVec` với độ dài trước, giúp phục hồi chính xác số phần tử và tránh đọc sai biên.

Mã nguồn 77: Serialize PlayerInfo2 theo phiên bản

STT	Mã nguồn

```
static void WritePlayer(std::ofstream& f, const PlayerInfo2& p) {
  WriteWStr(f, p.name);
  WriteStr(f, p.avatarPath);
  f.write(&p.piece, sizeof(p.piece));
  f.write(reinterpret_cast<const char*>(&p.totalWins), sizeof(p.totalWins));
  f.write(reinterpret_cast<const char*>(&p.matchWins), sizeof(p.matchWins));
  f.write(reinterpret_cast<const char*>(&p.movesCount), sizeof(p.movesCount));
  f.write(reinterpret_cast<const char*>(&p.maxTurnTime), sizeof(p.maxTurnTime));
  f.write(reinterpret_cast<const char*>(&p.totalTimePossessed), sizeof(p.totalTimePossessed));
}
static bool ReadPlayer(std::ifstream& f, PlayerInfo2& p, uint32_t version) {
  if (!ReadWStr(f, p.name)) return false;
  if (!ReadStr(f, p.avatarPath)) return false;
  if (!f.read(&p.piece, sizeof(p.piece))) return false;
  if (!f.read(reinterpret_cast<char*>(&p.totalWins), sizeof(p.totalWins))) return false;
  if (version >= 5) { if (!f.read(reinterpret_cast<char*>(&p.matchWins), sizeof(p.matchWins))) return false; } else { p.matchWins = 0; }
  if (!f.read(reinterpret_cast<char*>(&p.movesCount), sizeof(p.movesCount))) return false;
  if (!f.read(reinterpret_cast<char*>(&p.maxTurnTime), sizeof(p.maxTurnTime))) return false;
  if (version >= 5) {
   if (!f.read(reinterpret_cast<char*>(&p.totalTimePossessed), sizeof(p.totalTimePossessed))) return false; }
   else { p.totalTimePossessed = 0.0f; }
  return true;
}
```

Cách đọc có điều kiện theo phiên bản giúp các bản lưu cũ vẫn mở được mà không

gây crash. Đây là cơ chế then chốt để duy trì tính tương thích ngược khi game mở rộng tính năng.

8.1.3. Save/Load toàn bộ PlayState

Hàm `SaveMatchData` ghi toàn bộ `PlayState` vào file binary, từ thông tin meta (tên bản lưu, timestamp) đến dữ liệu trận đấu. Ngược lại, `LoadMatchData` đọc dữ liệu, kiểm tra `SAVE_MAGIC` và `SAVE_VERSION`, sau đó lần lượt khôi phục trạng thái.

Điểm quan trọng là các trường POD được ghi theo đúng thứ tự mà mã nguồn định nghĩa trong `SaveMatchData` (tuần tự cố định). Điều này giúp thao tác đọc nhanh (sequential read) và dễ kiểm soát lỗi. Khi xảy ra lỗi đọc ở bất kỳ bước nào, hàm trả về `false` để tầng trên quyết định xử lý (ví dụ: báo file lỗi hoặc tự reset ván).

Mã nguồn 78: Ghi trạng thái trận đấu ra file nhị phân

STT	Mã nguồn
1	`bool SaveMatchData(const PlayState* state, const std::wstring& filename) {`
2	`  CreateDirectoryW(L"Asset", NULL);`
3	`  CreateDirectoryW(L"Asset/save", NULL);`
4	`  std::ofstream file(filename, std::ios::binary);`
5	`  if (!file.is_open()) return false;`
6	`  file.write(reinterpret_cast<const char*>(&SAVE_MAGIC), sizeof(SAVE_MAGIC));`
7	`  file.write(reinterpret_cast<const char*>(&SAVE_VERSION), sizeof(SAVE_VERSION));`
8	`  WriteWStr(file, state->saveName);`
9	`  // Tự tạo timestamp nếu chưa có`
10	`  if (state->saveTimestamp.empty()) { …}`
11	`  WriteWStr(file, ts);`
12	`  file.write(reinterpret_cast<const char*>(&state->gameMode), sizeof(state->gameMode));`
13	`  file.write(reinterpret_cast<const char*>(&state->matchType), sizeof(state->matchType));`
14	`  file.write(reinterpret_cast<const char*>(&state->isP1Turn), sizeof(state->isP1Turn));`
15	`  file.write(reinterpret_cast<const char*>(&state->countdownTime), sizeof(state->countdownTime));`

```
  file.write(reinterpret_cast<const char*>(&state->timeRemaining), sizeof(state->timeRemaining));
  file.write(reinterpret_cast<const char*>(&state->boardSize), sizeof(state->boardSize));
  file.write(reinterpret_cast<const char*>(state->board), sizeof(state->board));
  file.write(reinterpret_cast<const char*>(&state->cursorRow), sizeof(state->cursorRow));
  file.write(reinterpret_cast<const char*>(&state->cursorCol), sizeof(state->cursorCol));
  file.write(reinterpret_cast<const char*>(&state->lastMoveRow), sizeof(state->lastMoveRow));
  file.write(reinterpret_cast<const char*>(&state->lastMoveCol), sizeof(state->lastMoveCol));
  file.write(reinterpret_cast<const char*>(&state->status), sizeof(state->status));
  file.write(reinterpret_cast<const char*>(&state->winner), sizeof(state->winner));
  file.write(reinterpret_cast<const char*>(&state->difficulty), sizeof(state->difficulty));
  file.write(reinterpret_cast<const char*>(&state->targetScore), sizeof(state->targetScore));
  file.write(reinterpret_cast<const char*>(&state->matchDuration), sizeof(state->matchDuration));
  file.write(reinterpret_cast<const char*>(&state->p1TotalTimeLeft), sizeof(state->p1TotalTimeLeft));
  file.write(reinterpret_cast<const char*>(&state->p2TotalTimeLeft), sizeof(state->p2TotalTimeLeft));
  file.write(reinterpret_cast<const char*>(&state->isMatchTimed), sizeof(state->isMatchTimed));
  WritePlayer(file, state->p1);
  WritePlayer(file, state->p2);
  WriteVec(file, state->winningCells);
  WriteVec(file, state->matchHistory);
  WriteVec(file, state->redoStack);
  file.close();
```

```
  return true;
}
```

Khi đọc dữ liệu, hệ thống trả về `false` nếu file lỗi hoặc không đúng phiên bản yêu cầu. Việc đọc theo thứ tự cố định đảm bảo mọi trường dữ liệu được phục hồi chính xác.

Mã nguồn 79: Đọc và khôi phục trạng thái trận đấu

```
bool LoadMatchData(PlayState* state, const std::wstring& filename) {
  std::ifstream file(filename, std::ios::binary);
  if (!file.is_open()) return false;
  uint32_t magic = 0, version = 0;
  file.read(reinterpret_cast<char*>(&magic), sizeof(magic));
  file.read(reinterpret_cast<char*>(&version), sizeof(version));
  if (magic != SAVE_MAGIC || version < 3) { file.close(); return false; }
  if (!ReadWStr(file, state->saveName)) return false;
  if (version >= 4) { if (!ReadWStr(file, state->saveTimestamp)) return false; } else { state->saveTimestamp = L"N/A (Bản cũ)"; }
  // Đọc lần lượt các trường POD và dữ liệu động
  if (!file.read(reinterpret_cast<char*>(&state->gameMode), sizeof(state->gameMode))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->matchType), sizeof(state->matchType))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->isP1Turn), sizeof(state->isP1Turn))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->countdownTime), sizeof(state->countdownTime))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->timeRemaining), sizeof(state->timeRemaining))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->boardSize), sizeof(state->boardSize))) return false;
  if (!file.read(reinterpret_cast<char*>(state->board), sizeof(state->board))) return false;
```

```
  if (!file.read(reinterpret_cast<char*>(&state->cursorRow), sizeof(state->cursorRow))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->cursorCol), sizeof(state->cursorCol))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->lastMoveRow), sizeof(state->lastMoveRow))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->lastMoveCol), sizeof(state->lastMoveCol))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->status), sizeof(state->status))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->winner), sizeof(state->winner))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->difficulty), sizeof(state->difficulty))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->targetScore), sizeof(state->targetScore))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->matchDuration), sizeof(state->matchDuration))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->p1TotalTimeLeft), sizeof(state->p1TotalTimeLeft))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->p2TotalTimeLeft), sizeof(state->p2TotalTimeLeft))) return false;
  if (!file.read(reinterpret_cast<char*>(&state->isMatchTimed), sizeof(state->isMatchTimed))) return false;
  if (!ReadPlayer(file, state->p1, version)) return false;
  if (!ReadPlayer(file, state->p2, version)) return false;
  if (!ReadVec(file, state->winningCells)) return false;
  if (!ReadVec(file, state->matchHistory)) return false;
  if (!ReadVec(file, state->redoStack)) return false;
  file.close();
  return true;
}
```

8.1.4. Hệ thống slot và metadata

Game hỗ trợ tối đa `MAX_SAVE_SLOTS` slot. Mỗi slot là một file `Asset/save/slot_X.bin`. Ngoài việc lưu/đọc, hệ thống còn hỗ trợ kiểm tra tồn tại, xóa, đổi tên, và trích xuất metadata phục vụ giao diện danh sách bản lưu.

Metadata được đọc nhanh từ phần đầu file (magic, version, name, timestamp, mode, type) nhằm giảm chi phí I/O khi hiển thị danh sách. Điều này tránh phải load toàn bộ `PlayState` cho mỗi slot. Với các bản lưu cũ, hệ thống gắn nhãn "Bản cũ" và cung cấp timestamp mặc định, giúp UI vẫn hiển thị ổn định.

Bảng 8.1: Cấu hình slot lưu game và metadata

Thành phần	**Giá trị/Nguồn**	**Ghi chú**
Số slot tối đa	`MAX_SAVE_SLOTS = 5`	Giới hạn cố định trong `GameConstants.h`
Tên hiển thị tối đa	`MAX_SAVE_NAME_LEN = 15`	Khi đổi tên, hệ thống cắt theo giới hạn này
Đường dẫn slot	`Asset/save/slot_N.bin`	Sinh bởi `GetSavePath(slot)`
Metadata chính	name, timestamp, mode, type, p1Wins, p2Wins, difficulty, boardSize, version	Đọc nhanh để hiển thị danh sách
Thao tác hỗ trợ	Check, delete, rename, đọc metadata	Phục vụ màn hình Load/Save

Mã nguồn 80: Truy xuất metadata bản lưu để hiển thị danh sách

STT	**Mã nguồn**
1	`struct SaveMetadata {`
2	`  std::wstring name;`
3	`  std::wstring timestamp;`
4	`  int mode;  int type;`
5	`  int p1Wins;  int p2Wins;`
6	`  int difficulty;  int boardSize;`
7	`  bool exists;  uint32_t version;`
8	`};`
9	`std::wstring GetSavePath(int slot) { return L"Asset/save/slot_" + std::to_wstring(slot) + L".bin"; }`
10	`bool CheckSaveExists(int slot) {`

```
  DWORD dwAttrib = GetFileAttributesW(GetSavePath(slot).c_str());
  return (dwAttrib != INVALID_FILE_ATTRIBUTES && !(dwAttrib & FILE_ATTRIBUTE_DIRECTORY));
}
```

Trích xuất metadata giúp giao diện hiển thị nhanh mà không cần load toàn bộ trạng thái, giảm chi phí đọc I/O khi mở màn hình Load Game.

8.2. Đa phương tiện

8.2.1. AudioSystem: preload và phát âm thanh

Hệ thống âm thanh sử dụng API MCI (Media Control Interface) của Windows để phát nhạc nền (BGM) và hiệu ứng (SFX). Để kiểm soát volume ổn định, SFX được mở bằng type `mpegvideo` và preload một lần khi khởi động game. Sau đó, PlaySFX chỉ phát bằng alias thay vì mở file lặp lại, giúp giảm độ trễ khi thao tác.

MCI phù hợp cho dự án desktop nhẹ vì có sẵn trong Windows và không cần thư viện ngoài. Việc preload tạo "alias" cố định cho từng hiệu ứng, cho phép hệ thống gọi phát tức thời mà không cần truy xuất file trên đĩa mỗi lần. Luồng SFX được khởi tạo bằng `std::call_once` để đảm bảo chỉ có một worker nền.

Mã nguồn 81: Preload các SFX và khởi tạo luồng nền

```
void InitAudioSystem() {
  EnsureSFXThread();
  PreLoad("Asset/audio/move.wav", "sfx_move");
  PreLoad("Asset/audio/select.wav", "sfx_select");
  PreLoad("Asset/audio/DatCo.wav", "sfx_place");
  PreLoad("Asset/audio/Tiengcoi.wav", "sfx_whistle");
  PreLoad("Asset/audio/MatLuot.wav", "sfx_timeout");
  PreLoad("Asset/audio/select.wav", "sfx_error");
  PreLoad("Asset/audio/siuuu.wav", "sfx_success");
  PreLoad("Asset/audio/siuuu.wav", "sfx_siu");
  PreLoad("Asset/audio/TiengKhanGia_Het_Tran.wav", "sfx_crowd");
}
```

Bảng 8.2: Danh sách SFX preload và alias

Alias	File âm thanh	Tác vụ
sfx_move	Asset/audio/move.wav	Di chuyển con trỏ
sfx_select	Asset/audio/select.wav	Chọn menu
sfx_place	Asset/audio/DatCo.wav	Đặt quân
sfx_whistle	Asset/audio/Tiengcoi.wav	Còi hiệu ứng
sfx_timeout	Asset/audio/MatLuot.wav	Hết lượt / timeout
sfx_error	Asset/audio/select.wav	Báo lỗi thao tác
sfx_success	Asset/audio/siuuu.wav	Thắng ván / lưu thành công
sfx_siu	Asset/audio/siuuu.wav	Khán đài cổ vũ
sfx_crowd	Asset/audio/Tieng..Tran.wav	Khán giả hết trận

8.2.2. SFX bất đồng bộ và cơ chế chống spam

Để tránh giật khi phát âm thanh liên tục, các yêu cầu SFX được đẩy vào hàng đợi và xử lý bởi một worker thread. Hệ thống đặt cooldown 50ms theo alias để tránh spam, giới hạn hàng đợi tối đa 32 yêu cầu, đồng thời chỉ cập nhật volume khi có thay đổi để giảm số lệnh MCI.

Hàng đợi SFX giúp tách biệt luồng render/logic khỏi thao tác I/O âm thanh. Cơ chế `mutex + condition_variable` đảm bảo thread nền ngủ khi không có yêu cầu và chỉ thức dậy khi có lệnh mới. Việc kiểm soát volume theo alias tránh gửi lệnh "setaudio" liên tục, giảm overhead và hạn chế trễ âm thanh. Khi alias chưa sẵn sàng, worker tự động open lại để đảm bảo lần phát đầu không bị rớt.

Mã nguồn 82: Hàng đợi SFX và xử lý luồng nền

STT	Mã nguồn
1	`void SFXWorker() {`
2	`  while (true) {`
3	`  SFXRequest req;`
4	`  { // Lấy yêu cầu từ queue`
5	`  std::unique_lock<std::mutex> lock(g_SFXMutex);`
6	`  g_SFXCond.wait(lock, []() { return !g_SFXQueue.empty() || !g_SFXRunning; });`
7	`  if (!g_SFXRunning && g_SFXQueue.empty()) break;`
8	`  if (g_SFXQueue.empty()) continue;`

```
        req = g_SFXQueue.front();
        g_SFXQueue.pop();
    }
    if (req.command == SFXCommand::Preload) {
        mciSendStringA(("close " + req.alias).c_str(), NULL, 0, NULL);
        bool opened = OpenSFXAlias(req.path, req.alias);
        { std::lock_guard<std::mutex> lock(g_SFXMutex); g_SFXReady[req.alias] = opened; }
        continue;
    }
    if (req.command == SFXCommand::Stop) {
        mciSendStringA(("stop " + req.alias).c_str(), NULL, 0, NULL);
        continue;
    }
    if (req.command != SFXCommand::Play) continue;
    bool isReady = false; std::string path;
    { std::lock_guard<std::mutex> lock(g_SFXMutex);
        auto readyIt = g_SFXReady.find(req.alias);
        isReady = (readyIt != g_SFXReady.end() && readyIt->second);
        auto pathIt = g_SFXPaths.find(req.alias);
        if (pathIt != g_SFXPaths.end()) path = pathIt->second; }
    if (!isReady && !path.empty()) {
        mciSendStringA(("close " + req.alias).c_str(), NULL, 0, NULL);
        bool opened = OpenSFXAlias(path, req.alias);
        { std::lock_guard<std::mutex> lock(g_SFXMutex); g_SFXReady[req.alias] = opened; }
    }
    ULONGLONG now = GetTickCount64();
    auto lastTimeIt = g_LastSFXTime.find(req.alias);
    if (lastTimeIt != g_LastSFXTime.end() && (now - lastTimeIt->second) < 50ULL) continue;
```

```
    g_LastSFXTime[req.alias] = now;
    auto lastVolIt = g_LastSFXVolume.find(req.alias);
    int lastVol = (lastVolIt == g_LastSFXVolume.end()) ? -1 : lastVolIt->second;
    if (lastVol != req.volume) {
      std::string volCmd = "setaudio " + req.alias + " volume to " + std::to_string(req.volume);
      mciSendStringA(volCmd.c_str(), NULL, 0, NULL);
      g_LastSFXVolume[req.alias] = req.volume;
    }
    mciSendStringA(("stop " + req.alias).c_str(), NULL, 0, NULL);
    mciSendStringA(("play " + req.alias + " from 0").c_str(), NULL, 0, NULL);
  }
}
```

8.2.3. BGM: lặp vô hạn và quản lý volume

Nhạc nền được phát theo cơ chế repeat và cập nhật âm lượng dựa trên cấu hình toàn cục. Để tránh gửi lệnh không cần thiết, hệ thống chỉ cập nhật volume khi có sự thay đổi.

Vòng đời BGM gồm 3 bước: đóng nhạc cũ (nếu có), mở file mới, thiết lập volume và lặp vô hạn. Cách làm này ngăn rò rỉ handle MCI khi người chơi đổi màn hình hoặc thay đổi bài nhạc. Khi thoát game, `ShutdownAudioSystem` sẽ dừng thread và gọi `close all` để giải phóng tài nguyên.

Bảng 8.3: Cấu hình BGM và quy tắc cập nhật âm lượng

Thành phần	**Giá trị/Nguồn**	**Ghi chú**
Điều kiện phát BGM	`g_Config.isBgmEnabled`	Nếu tắt thì bỏ qua `PlayBGM`
Alias BGM	`bgm`	Dùng lệnh MCI mở/phát/đóng
Âm lượng	`g_Config.bgmVolume * 10`	Scale sang thang MCI (0–1000)
Lặp vô hạn	`play bgm repeat`	Tự lặp khi hết nhạc
Tối ưu cập nhật	`lastBgmVol`	Chỉ gửi lệnh khi volume đổi

Mã nguồn 83: Phát nhạc nền và cập nhật volume

STT	Mã nguồn
1	`void PlayBGM(const std::string& filepath) {`
2	`  if (!g_Config.isBgmEnabled) return;`
3	`  StopBGM();`
4	`  std::string cmd = "open " + filepath + "type mpegvideo alias bgm";`
5	`  mciSendStringA(cmd.c_str(), NULL, 0, NULL);`
6	`  UpdateBGMVolume(true);`
7	`  mciSendStringA("play bgm repeat", NULL, 0, NULL);`
8	`}`
9	`void UpdateBGMVolume(bool force) {`
10	`  static int lastBgmVol = -1;`
11	`  int vol = g_Config.bgmVolume * 10;`
12	`  if (!force && lastBgmVol == vol) return;`
13	`  std::string volCmd = "setaudio bgm volume to " + std::to_string(vol);`
14	`  mciSendStringA(volCmd.c_str(), NULL, 0, NULL);`
15	`  lastBgmVol = vol;`
16	`}`

8.3. Ngôn ngữ

Hệ thống đa ngôn ngữ nằm trong `Localization`, sử dụng file text dạng `key=value`. Khi tải, toàn bộ cặp khóa-giá trị được đưa vào `map`. Việc chuyển đổi UTF-8 sang `wstring` đảm bảo ký tự tiếng Việt hiển thị đúng trong UI.

Thiết kế này giúp nội dung giao diện tách khỏi mã nguồn: khi cần thay đổi câu chữ hoặc bổ sung ngôn ngữ mới, chỉ cần chỉnh file `Asset/lang`. Mã nguồn chỉ làm nhiệm vụ tra cứu, không cần sửa đổi từng chuỗi literal trong UI.

Ví dụ trích một phần nội dung `key=value` trong file ngôn ngữ:

Mã nguồn 84: Ví dụ định dạng `key=value` trong file ngôn ngữ

STT	Mã nguồn
1	`menu_play=Chơi nhanh`
2	`menu_load=Tiếp tục`

3	menu_settings=Cài đặt
4	match_turn=Lượt của bạn
5	result_draw=Hòa

Bảng 8.4: Danh sách ngôn ngữ và file bản địa hóa

Mã	Enum	File	Ghi chú
VI	APP_LANG_VI	Asset/lang/vi.txt	UTF-8 → wstring
EN	APP_LANG_EN	Asset/lang/en.txt	UTF-8 → wstring

Mã nguồn 85: Nạp từ điển ngôn ngữ và chuyển UTF-8

STT	Mã nguồn
1	`static std::wstring UTF8ToWString(const std::string& str) {`
2	`  if (str.empty()) return L"";`
3	`  int size_needed = MultiByteToWideChar(CP_UTF8, 0, &str[0], (int)str.size(), NULL, 0);`
4	`  std::wstring wstrTo(size_needed, 0);`
5	`  MultiByteToWideChar(CP_UTF8, 0, &str[0], (int)str.size(), &wstrTo[0], size_needed);`
6	`  return wstrTo;`
7	`}`
8	`void LoadLanguageFile(Language lang) {`
9	`  g_dictionary.clear();`
10	`  std::string filepath = (lang == APP_LANG_VI) ? "Asset/lang/vi.txt" : "Asset/lang/en.txt";`
11	`  std::ifstream file(filepath);`
12	`  if (!file.is_open()) return;`
13	`  std::string line;`
14	`  while (std::getline(file, line)) {`
15	`    size_t pos = line.find('=');`
16	`    if (pos != std::string::npos) {`
17	`      std::string key = line.substr(0, pos);`

```
        std::string val = line.substr(pos + 1);
        g_dictionary[key] = UTF8ToWString(val);
      }
    }
}
```

Khi UI truy vấn một khóa chưa tồn tại, hệ thống sẽ tự cache chính khóa đó dưới dạng `wstring`. Cơ chế này giảm chi phí chuyển đổi lặp lại trong các vòng lặp render.

Ngoài ra, hành vi "fallback về chính key" giúp phát hiện lỗi dịch: nếu một key bị thiếu trong file ngôn ngữ, UI sẽ hiển thị key đó thay vì chuỗi rỗng, hỗ trợ kiểm thử nhanh và tránh mất thông tin.

Mã nguồn 86: Truy xuất text với cache khóa thiếu

STT	Mã nguồn
1	`std::wstring GetText(const std::string& key) {`
2	`  auto it = g_dictionary.find(key);`
3	`  if (it != g_dictionary.end()) return it->second;`
4	`  std::wstring wkey = UTF8ToWString(key);`
5	`  g_dictionary[key] = wkey;`
6	`  return wkey;`
7	`}`

8.4. Thời gian

Hệ thống thời gian phục vụ đếm ngược theo lượt, hiển thị dạng `MM:SS`, và hỗ trợ reset theo người chơi hiện tại. Biến `timeAccumulator` tích lũy `dt` để giảm sai số và chỉ trừ thời gian khi đủ một giây thực.

Cách tích lũy `dt` tránh tình trạng trừ thời gian theo từng frame (dễ gây lệch khi FPS không ổn định). Khi đủ 1 giây, hệ thống mới giảm `timeRemaining`, giúp đồng hồ đếm ngược sát với thời gian thực hơn.

8.4.1. Cập nhật bộ đếm theo dt

Mã nguồn 87: Cập nhật đếm ngược và phát hiện hết lượt

STT	Mã nguồn
1	`bool UpdateCountdown(PlayState* state, double dt) {`
2	`  if (state->timeRemaining <= 0) return true;`
3	`  timeAccumulator += dt;`
4	`  if (timeAccumulator >= 1.0) {`
5	`  timeAccumulator -= 1.0;`
6	`  state->timeRemaining--;`
7	`  if (state->timeRemaining <= 0) { state->timeRemaining = 0; }`
8	`  return true;`
9	`  }`
10	`  return false;`
11	`}`

8.4.2. Reset theo lượt và cơ chế failsafe

Khi chuyển lượt, bộ đếm thời gian được reset theo `maxTurnTime` của người chơi hiện tại. Nếu giá trị này bị lỗi (<=0) do dữ liệu cũ hoặc bộ nhớ chưa khởi tạo, hệ thống tự động dùng `countdownTime` hoặc mặc định 30 giây để tránh game bị treo thời gian.

Điều này đặc biệt quan trọng khi load bản lưu cũ: nếu phiên bản trước chưa lưu `maxTurnTime`, hệ thống sẽ tự khôi phục giá trị hợp lệ và ghi lại vào state để những lần sau chạy ổn định.

Mã nguồn 88: Reset bộ đếm kèm kiểm tra an toàn

STT	Mã nguồn
1	`void ResetTimer(PlayState* state) {`
2	`  float maxFloatTime = state->isP1Turn ? state->p1.maxTurnTime : state->p2.maxTurnTime;`
3	`  if (maxFloatTime <= 0.0f) {`
4	`  maxFloatTime = (state->countdownTime > 0) ? static_cast<float>(state->countdownTime) : 30.0f;`
5	`  if (state->isP1Turn) { state->p1.maxTurnTime = maxFloatTime; } else { state->p2.maxTurnTime = maxFloatTime; }`
6	`  }`
7	`  state->timeRemaining = static_cast<int>(maxFloatTime);`

8	`  state->countdownTime = state->timeRemaining;`
9	`  timeAccumulator = 0.0;`
10	`}`

8.4.3. Hiển thị thời gian dạng MM:SS

Mã nguồn 89: Định dạng thời gian hiển thị

STT	Mã nguồn
1	`std::string GetTimeDisplay(PlayState* state) {`
2	`  int minutes = state->timeRemaining / 60;`
3	`  int seconds = state->timeRemaining % 60;`
4	`  std::ostringstream oss;`
5	`  oss << std::setfill('0') << std::setw(2) << minutes << ":" << std::setfill('0') << std::setw(2) << seconds;`
6	`  return oss.str();`
7	`}`

8.5. Cấu hình hệ thống

Các thiết lập cơ bản (BGM/SFX bật tắt, âm lượng, ngôn ngữ) được lưu trong `GameConfig` và được đọc/ghi bằng `ConfigLoader`. Nếu file cấu hình chưa tồn tại, hệ thống tự tạo giá trị mặc định nhằm đảm bảo game luôn khởi động được.

Việc lưu cấu hình dạng binary giúp quá trình tải nhanh và cấu trúc dữ liệu gọn. Cấu hình được nạp ngay khi mở game để các mô-đun khác (Audio, Localization) có thể dựa vào giá trị thiết lập hiện hành. Thiết kế này cũng tránh trường hợp game chạy với trạng thái nửa cấu hình khi người dùng chưa vào phần Settings.

Mã nguồn 90: Thiết lập mặc định và nạp cấu hình

STT	Mã nguồn
1	`static void SetDefaultConfig(GameConfig* config) {`
2	`  config->isBgmEnabled = true;`
3	`  config->bgmVolume = 50;`
4	`  config->isSfxEnabled = true;`

```
  config->sfxVolume = 50;
  config->currentLang = APP_LANG_VI;
}
void LoadConfig(GameConfig* config, const std::string& filepath) {
  std::ifstream file(filepath, std::ios::binary);
  if (!file.is_open()) { SetDefaultConfig(config); return; }
  file.read(reinterpret_cast<char*>(config), sizeof(GameConfig));
  file.close();
}
```

Việc lưu cấu hình cũng ở dạng binary để đơn giản hóa thao tác và giảm thời gian I/O.

Mã nguồn 91: Lưu cấu hình hiện hành

```
void SaveConfig(const GameConfig* config, const std::string& filepath) {
  std::ofstream file(filepath, std::ios::binary);
  if (file.is_open()) {
    file.write(reinterpret_cast<const char*>(config), sizeof(GameConfig));
    file.close();
  }
}
```

CHƯƠNG 9. TỔNG KẾT

9.1. Kết quả đạt được

Dự án đã hoàn thiện một trò chơi Gomoku/TTT có đầy đủ vòng đời trận đấu: cấu hình, chơi, tạm dừng, lưu/đọc, và tổng kết. Hệ thống logic tách lớp rõ ràng, các mô-đun phụ trợ (âm thanh, ngôn ngữ, thời gian, cấu hình) hoạt động ổn định, và phần đồ họa/UX đảm bảo tính trực quan, phản hồi tốt trong thời gian thực.

Các mục tiêu chính đã đạt: hỗ trợ nhiều chế độ chơi (PvP/PvE), tùy biến kích thước bàn cờ, quản lý lượt và thời gian, lưu/đọc đa slot, và trải nghiệm UI theo phong cách pixel art kết hợp procedural graphics.

9.2. Các hạng mục đã hoàn thiện

Logic trò chơi: Cơ chế luật chơi, kiểm tra thắng/hòa, xử lý lượt, và thống kê được triển khai nhất quán trên mọi kích thước bàn cờ và chế độ (Caro/TTT).

Đồ họa và UI/UX: Hệ thống RenderAPI dựa trên Win32 GDI/GDI+ với bộ đệm kép, cache bitmap, palette động, và các màn hình UI hoàn chỉnh (Menu, Play, Pause, Settings, Load/Save, Summary...).

Mô-đun phụ trợ: Hệ thống Save/Load dạng binary có versioning; AudioSystem hỗ trợ SFX async và BGM; Localization đọc file key=value; TimeSystem xử lý đếm ngược và hiển thị thời gian; ConfigLoader đọc/ghi thiết lập.

Bảng 9.1: Ma trận chức năng các mô-đun phụ trợ

Mô-đun	**Tệp chính**	**Chức năng**
SaveLoadSystem	`SaveLoadSystem.h/.cpp`	Lưu/tải binary, versioning, metadata slot
AudioSystem	`AudioSystem.h/.cpp`	Preload SFX, BGM lặp, kiểm soát volume
Localization	`Localization.h/.cpp`	Đọc `key=value`, UTF-8 → `wstring`
TimeSystem	`TimeSystem.h/.cpp`	Đếm ngược, reset theo lượt, hiển thị `MM:SS`
ConfigLoader	`ConfigLoader.h/.cpp`	Nạp/lưu `GameConfig` dạng binary

9.3. Dấu ấn kỹ thuật nổi bật

Dự án sử dụng cơ chế định nghĩa trạng thái ván đấu tập trung trong `PlayState`, giúp các hệ thống (game logic, UI, save/load) dùng chung nguồn dữ liệu và giảm sai lệch. Với AI, hệ thống áp dụng alpha-beta pruning và Zobrist hashing trong quá trình đánh giá nước đi, giúp giảm nhánh tìm kiếm và tăng tốc độ tra cứu trạng thái. Các hằng số cốt lõi được gom về `GameConstants` để đồng bộ hóa kích thước bàn cờ, giá trị ô cờ và giới hạn slot lưu.

Mã nguồn 92: Các hằng số nền tảng cho bàn cờ và lưu game

STT	Mã nguồn
1	`#define GOMOKU_SIZE 15`
2	`#define WIN_CONDITION 5`
3	`#define MAX_BOARD_SIZE 20`
4	`#define CELL_EMPTY 0`
5	`#define CELL_PLAYER1 1`
6	`#define CELL_PLAYER2 2`
7	`#define MAX_SAVE_SLOTS 5`
8	`#define MAX_SAVE_NAME_LEN 15`

Bảng 9.2: Tóm tắt các trường quan trọng trong PlayState

Trường	Kiểu/nhóm	Vai trò (và lưu/đọc)
`gameMode`, `matchType`	enum	Chế độ ván đấu (được lưu)
`p1`, `p2`	PlayerInfo2	Thông tin người chơi, thống kê (được lưu)
`boardSize`, `board`	int + ma trận	Kích thước và trạng thái bàn cờ (được lưu)
`isP1Turn`	bool	Lượt hiện tại (được lưu)
`countdownTime`, `timeRemaining`	int	Thời gian lượt (được lưu)
`cursorRow`, `cursorCol`	int	Con trỏ UI (được lưu để phục hồi)
`lastMoveRow`, `lastMoveCol`	int	Highlight nước đi cuối (được lưu)
`winningCells`	vector pair	Chuỗi thắng (được lưu)
`matchHistory`, `redoStack`	vector pair	Lịch sử và hoàn tác (được lưu)
`status`, `winner`	enum/int	Trạng thái ván và kết quả (được lưu)
`difficulty`, `targetScore`	int	Thiết lập PvE/BO (được lưu)
`matchDuration`, `p1TotalTimeLeft`, `p2TotalTimeLeft`, `isMatchTimed`	float/int/bool	Thời gian toàn trận (được lưu)
`saveName`, `saveTimestamp`	wstring	Metadata bản lưu (được lưu)

Cấu trúc `PlayState` bao phủ cả dữ liệu logic và dữ liệu hiển thị (cursor, winningCells, matchHistory...), nhờ đó UI phản ánh đúng trạng thái trận đấu và save/load có thể phục hồi đầy đủ ván đang chơi.

Mã nguồn 93: Trạng thái ván đấu tập trung trong PlayState

STT	Mã nguồn
1	`int boardSize = 15;`
2	`int board[MAX_BOARD_SIZE][MAX_BOARD_SIZE] = {0};`
3	`int lastMoveRow = -1;`
4	`int lastMoveCol = -1;`
5	`std::vector<std::pair<int, int>> winningCells;`
6	`std::vector<std::pair<int, int>> matchHistory;`

```
std::vector<std::pair<int, int>> redoStack;
bool isP1Turn = true;
```

TÀI LIỆU THAM KHẢO

[1] Microsoft. "Windows gdi." Truy cap ngay: 20/04/2026. [Online]. Available: https://learn.microsoft.com/windows/win32/gdi/windows-gdi

[2] Microsoft. "C++ language reference." Truy cap ngay: 20/04/2026. [Online]. Available: https://learn.microsoft.com/cpp/cpp/cpp-language-reference